Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial

Optimasi Penataan Kendaraan pada Geladak Kapal

dengan Local Search



Disusun Oleh Tim Guele:

Zhafira Kayla Nafisa	18224018
Kayla Fiyaza Nawal Zaghbi	18224054
Nabilla Eka Putri Sunarto	18224056

PROGRAM STUDI SISTEM & TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO & INFORMATIKA KOMPUTASI

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI **1**
DAFTAR TABEL **2**
BAB I
PENDAHULUAN **3**
1.1 Deskripsi Permasalahan 3
BAB II
PEMBAHASAN **5**
2.1 Representasi Struktur Data 5
2.2 Pemilihan Objective Function 7
2.3 Pemilihan Fungsi Initial State 13
2.4 Pemilihan Fungsi Neighbor 15
2.5 Heuristik yang Digunakan 23
2.6 Implementasi Algoritma Local Search 24
2.6.1 Steepest Ascent Hill Climbing 24
2.6.2 Stochastic Hill-Climbing 27
2.6.3 Hill Climbing with Sideways Move 31
2.6.4 Random Restart Hill-Climbing 36
2.6.5 Simulated Annealing 41
2.6.6 Genetic Algorithm 46
2.7 Hasil Eksperimen 51
2.7.1 Hasil Eksperimen Steepest Ascent Hill Climbing 51
2.7.2 Hasil Eksperimen Stochastic Hill-Climbing 54
2.7.3 Hasil Eksperimen Hill Climbing with Sideways Move 57
2.7.4 Hasil Eksperimen Random Restart Hill-Climbing 60
2.7.5 Hasil Eksperimen Simulated Annealing 63
2.7.6 Hasil Eksperimen Genetic Algorithm 66
2.8 Analisis Hasil Eksperimen 85
2.8.1 Analisis Pertanyaan 1 85
2.8.2 Analisis Pertanyaan 2 89
2.8.3 Analisis Pertanyaan 3 92
2.8.4 Analisis Pertanyaan 4 94
2.8.5 Analisis Pertanyaan 5 97
BAB III
KESIMPULAN DAN SARAN **99**
Pembagian Tugas **102**
REFERENSI **103**
LAMPIRAN **104**

DAFTAR TABEL

Tabel 1. Representasi Struktur Data 1.....6
Tabel 2. Representasi Struktur Data 2..... 6
Tabel 3. Implementasi Objective Function.....10
Tabel 4. Implementasi Inisialisasi State Awal..... 14
Tabel 5. Implementasi Fungsi Neighbor.....17
Tabel 6. Implementasi Steepest Ascent Hill-Climbing.....25
Tabel 7. Implementasi Stochastic Hill-Climbing.....28
Tabel 8. Implementasi Hill-Climbing with Sideways Move.....32
Tabel 9. Implementasi Random Restart Hill-Climbing.....37
Tabel 10. Implementasi Simulated Annealing..... 43
Tabel 11. Implementasi Genetic Algorithm..... 48
Tabel 12. Hasil Eksperimen Steepest Ascent Hill Climbing..... 52
Tabel 13. Hasil Eksperimen Stochastic Hill-Climbing.....55
Tabel 14. Hasil Eksperimen Hill Climbing with Sideways Move.....58
Tabel 15. Hasil Eksperimen Random Restart Hill-Climbing..... 61
Tabel 16. Hasil Eksperimen Simulated Annealing.....64
Tabel 17. Hasil Eksperimen Skenario 1 Konfigurasi 1 Genetic Algorithm..... 67
Tabel 18. Hasil Eksperimen Skenario 1 Konfigurasi 2 Genetic Algorithm.....70
Tabel 19. Hasil Eksperimen Skenario 1 Konfigurasi 3 Genetic Algorithm.....73
Tabel 20. Hasil Eksperimen Skenario 2 Konfigurasi 1 Genetic Algorithm..... 77
Tabel 21. Hasil Eksperimen Skenario 2 Konfigurasi 2 Genetic Algorithm.....80
Tabel 22. Hasil Eksperimen Skenario 2 Konfigurasi 3 Genetic Algorithm..... 83
Tabel 23. Analisis Eksperimen Pertanyaan 1..... 86
Tabel 24. Perbandingan Total ShippingFee Akhir Tiap Algoritma.....90
Tabel 25. Analisis Eksperimen Pertanyaan 2.....91
Tabel 26. Perbandingan Durasi Pencarian Tiap Algoritma.....93
Tabel 27. Analisis Eksperimen Pertanyaan 3..... 94
Tabel 28. Perbandingan Hasil Fee Tiap Algoritma.....95
Tabel 29. Analisis Eksperimen Pertanyaan 4.....96
Tabel 30. Analisis Eksperimen Pertanyaan 5.....98

BAB I
PENDAHULUAN

1.1 Deskripsi Permasalahan

PT Intim Line mengoperasikan kapal kargo pengangkut kendaraan dan menerima daftar kendaraan dari beberapa dealer pada setiap jadwal keberangkatan. Setiap kendaraan memiliki dimensi, berat, tarif pengiriman, dan deadline yang berbeda-beda. Permasalahan ini bertujuan untuk membuat konfigurasi penempatan kendaraan paling optimal pada sebuah geladak kapal kargo, yaitu menentukan kendaraan yang akan diangkut beserta posisi dan orientasinya di dalam geladak, dengan tujuan memaksimalkan ShippingFee pada kapal. Namun, terdapat beberapa kondisi yang harus dipenuhi:

- Kendaraan dan geladak kapal direpresentasikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang. Kendaraan memiliki atribut id, *Dimensions* (*width* dan *length* dalam satuan grid), *Orientation* (*horizontal* atau *vertical*), *ShippingFee* (juta rupiah), *Weight* (unit), dan ETA (sisa hari). Kapal memiliki atribut *Dimensions* (*width* dan *length* dalam satuan grid) dan *MaxCapacity* (unit).
- Kendaraan dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam kapal, sehingga suatu kendaraan dapat berada di luar (seluruh bagian kendaraan di luar kapal).
- Koordinat berupa bilangan bulat.
- Inisialisasi *state* awal dilakukan secara random.
- Perpindahan yang dibolehkan untuk setiap iterasi adalah sebagai berikut:
    - Menukar dua kendaraan, baik di dalam maupun di luar kapal
    - Memindahkan sebuah kendaraan ke suatu koordinat yang berbeda
    - Mengubah orientasi kendaraan
- Kendaraan tidak boleh saling bertumpuk (*no overlapping*), yaitu tidak boleh ada satu sel grid geladak yang ditempati oleh lebih dari satu kendaraan.
- Kendaraan harus berada di dalam batas geladak, yaitu seluruh sel grid yang ditempati suatu kendaraan harus berada di dalam dimensi geladak kapal.

- Kapal tidak boleh mengangkut melebihi kapasitasnya.
- Representasi solusi (*state* akhir) harus dapat menunjukkan keadaan semua kendaraan yang berada di dalam maupun di luar kapal, keadaan geladak pada kapal, dan *value* pada *state* tersebut.
- Untuk menyelesaikan permasalahan ini, rencana harus diimplementasikan menggunakan 3 algoritma *local search*: salah satu algoritma *hill-climbing*, *simulated annealing*, dan *genetic algorithm*.
- Program dibuat menggunakan bahasa Python dan harus dapat memberikan visualisasi *state* awal, *state* akhir, dan hasil eksperimen.

BAB II
PEMBAHASAN

2.1 Representasi Struktur Data

Pada implementasi rencana algoritma pencarian untuk permasalahan penataan kendaraan pada geladak kapal, struktur data utama yang digunakan adalah variabel *state*. Variabel *state* merepresentasikan konfigurasi penempatan seluruh kendaraan, baik yang diangkut di dalam geladak maupun yang berada di luar kapal. Struktur data ini disusun dalam bentuk *dictionary*, dengan id kendaraan sebagai key dan *dictionary* berisi status penempatan kendaraan tersebut sebagai value. Setiap *value* memiliki empat pasangan *key-value*, yaitu *in_ship* (menyimpan status apakah kendaraan berada di dalam kapal), x dan y (menyimpan koordinat sel awal kendaraan pada geladak), serta *Orientation* (menyimpan orientasi kendaraan, yaitu Horizontal atau Vertical).

Geladak kapal direpresentasikan sebagai grid dua dimensi berukuran W × L, dengan W adalah lebar dan L adalah panjang geladak. Koordinat x berjalan sepanjang lebar geladak (0 sampai W − 1) dan koordinat y berjalan sepanjang panjang geladak (0 sampai L − 1). Kendaraan dengan ukuran w × l pada orientasi Horizontal akan menempati sel dari x sampai x + w − 1 dan dari y sampai y + l − 1. Pada orientasi Vertical, lebar dan panjang kendaraan ditukar sehingga kendaraan menempati sel dari x sampai x + l − 1 dan dari y sampai y + w − 1. Kendaraan yang berada di luar kapal memiliki nilai in_ship bernilai false serta x dan y bernilai null. Untuk keperluan evaluasi *objective function*, *state* tersebut diubah sementara menjadi grid bilangan bulat berukuran W × L yang menyimpan banyak kendaraan yang menempati setiap sel, sehingga sel yang bernilai lebih dari 1 menandakan terjadinya tumpang tindih.

Berikut ini adalah contoh struktur data yang menjadi input untuk jalannya program dalam format JSON:

Tabel 1. Representasi Struktur Data 1

File: `input1.json`


```
{
  "Ship": {"Dimensions": {"w": 6, "l": 8}, "MaxCapacity": 110},
  "Vehicles": [
    {"id": "K01", "Dimensions": {"w": 3, "l": 7}, "ShippingFee": 28,
"Weight": 49, "ETA": 5},
    {"id": "K02", "Dimensions": {"w": 3, "l": 7}, "ShippingFee": 27,
"Weight": 50, "ETA": 4},
    {"id": "K03", "Dimensions": {"w": 2, "l": 6}, "ShippingFee": 12,
"Weight": 21, "ETA": 4},
    {"id": "K04", "Dimensions": {"w": 2, "l": 4}, "ShippingFee": 7,
"Weight": 10, "ETA": 2},
    {"id": "K05", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 26,
"Weight": 23, "ETA": 1},
    {"id": "K06", "Dimensions": {"w": 2, "l": 4}, "ShippingFee": 8,
"Weight": 10, "ETA": 2},
    {"id": "K07", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 10,
"Weight": 16, "ETA": 3}
  ]
}
```


Tabel 2. Representasi Struktur Data 2

File: `input2.json`


```
{
  "Ship": {"Dimensions": {"w": 10, "l": 20}, "MaxCapacity": 500},
  "Vehicles": [
    {"id": "V01", "Dimensions": {"w": 2, "l": 6}, "ShippingFee": 16,
"Weight": 28, "ETA": 1},
    {"id": "V02", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 28,
"Weight": 19, "ETA": 4},
    {"id": "V03", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 11,
"Weight": 12, "ETA": 7},
    {"id": "V04", "Dimensions": {"w": 3, "l": 7}, "ShippingFee": 28,
"Weight": 50, "ETA": 1},
    {"id": "V05", "Dimensions": {"w": 2, "l": 6}, "ShippingFee": 14,
"Weight": 23, "ETA": 3},
    {"id": "V06", "Dimensions": {"w": 2, "l": 4}, "ShippingFee": 9,
"Weight": 11, "ETA": 5},
    {"id": "V07", "Dimensions": {"w": 2, "l": 4}, "ShippingFee": 7,
"Weight": 12, "ETA": 1},
    {"id": "V08", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 12,
"Weight": 16, "ETA": 5},
    {"id": "V09", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 14,
```

```
"Weight": 13, "ETA": 2},
    {"id": "V10", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 27,
"Weight": 18, "ETA": 2},
    {"id": "V11", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 13,
"Weight": 17, "ETA": 7},
    {"id": "V12", "Dimensions": {"w": 3, "l": 7}, "ShippingFee": 27,
"Weight": 50, "ETA": 3},
    {"id": "V13", "Dimensions": {"w": 3, "l": 10}, "ShippingFee": 39,
"Weight": 113, "ETA": 2},
    {"id": "V14", "Dimensions": {"w": 2, "l": 6}, "ShippingFee": 14,
"Weight": 22, "ETA": 3},
    {"id": "V15", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 27,
"Weight": 24, "ETA": 1},
    {"id": "V16", "Dimensions": {"w": 3, "l": 10}, "ShippingFee": 42,
"Weight": 87, "ETA": 6},
    {"id": "V17", "Dimensions": {"w": 2, "l": 4}, "ShippingFee": 7,
"Weight": 10, "ETA": 3},
    {"id": "V18", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 14,
"Weight": 15, "ETA": 5},
    {"id": "V19", "Dimensions": {"w": 2, "l": 6}, "ShippingFee": 13,
"Weight": 23, "ETA": 7},
    {"id": "V20", "Dimensions": {"w": 2, "l": 4}, "ShippingFee": 9,
"Weight": 9, "ETA": 1},
    {"id": "V21", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 28,
"Weight": 21, "ETA": 1},
    {"id": "V22", "Dimensions": {"w": 3, "l": 10}, "ShippingFee": 35,
"Weight": 118, "ETA": 6},
    {"id": "V23", "Dimensions": {"w": 2, "l": 4}, "ShippingFee": 7,
"Weight": 11, "ETA": 4},
    {"id": "V24", "Dimensions": {"w": 3, "l": 7}, "ShippingFee": 35,
"Weight": 63, "ETA": 2},
    {"id": "V25", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 11,
"Weight": 13, "ETA": 5},
    {"id": "V26", "Dimensions": {"w": 2, "l": 5}, "ShippingFee": 12,
"Weight": 15, "ETA": 4},
    {"id": "V27", "Dimensions": {"w": 3, "l": 7}, "ShippingFee": 35,
"Weight": 58, "ETA": 2},
    {"id": "V28", "Dimensions": {"w": 2, "l": 4}, "ShippingFee": 10,
"Weight": 9, "ETA": 4},
    {"id": "V29", "Dimensions": {"w": 2, "l": 4}, "ShippingFee": 7,
"Weight": 8, "ETA": 1},
    {"id": "V30", "Dimensions": {"w": 2, "l": 6}, "ShippingFee": 13,
"Weight": 22, "ETA": 6}
  ]
}
```

2.2 Pemilihan Objective Function

Objective function berperan sebagai alat ukur yang memberi nilai pada setiap *state*, sehingga algoritma *local search* dapat membedakan konfigurasi yang lebih baik dari yang lebih buruk. Pada permasalahan ini, kapal ingin mengangkut kendaraan dengan total `ShippingFee` setinggi mungkin, tetapi tetap harus mematuhi tiga batasan yaitu kendaraan tidak boleh bertumpuk, kendaraan harus berada di dalam geladak, dan total berat tidak boleh melebihi kapasitas kapal. Karena itu, fungsi yang dirancang harus mendorong pencarian ke arah keuntungan yang besar sekaligus menjauhkannya dari konfigurasi yang melanggar batasan.

Data yang digunakan dalam perhitungan berasal dari file input JSON, yaitu ukuran geladak (W dan L), kapasitas kapal (`MaxCapacity`), serta daftar kendaraan dengan id, dimensi, `ShippingFee` , Weight, dan ETA masing-masing.

Kelompok kami memilih *objective function* yang dimaksimalkan, artinya *state* dengan nilai lebih tinggi dianggap lebih baik. Nilai tersebut tersusun dari empat komponen dengan tingkat kepentingan yang berbeda:

1. Penalti Pelanggaran (`violation`): Setiap pelanggaran batasan dihitung dalam satuan yang bisa dijumlahkan, yaitu banyak sel geladak yang terisi lebih dari satu kendaraan (`overlap_cells`), banyak sel kendaraan yang keluar dari geladak (`out_of_bound_cells`), dan selisih berat di atas kapasitas (`overweight`). Jumlah ketiganya dikalikan bobot P, yang besarnya sama dengan total `ShippingFee` semua kendaraan ditambah 1. Dengan bobot sebesar ini, satu satuan pelanggaran saja sudah menghapus seluruh keuntungan yang mungkin didapat, sehingga algoritma akan selalu berusaha memperbaiki pelanggaran terlebih dahulu.
2. Total `ShippingFee` (`total_fee`): Komponen inti yang mewakili tujuan permasalahan, yaitu jumlah `ShippingFee` dari kendaraan yang ditempatkan di geladak.

3. Prioritas ETA (`eta_term`): Untuk setiap kendaraan di geladak, nilai 1/ETA dijumlahkan lalu dikalikan 1/(2n), dengan n adalah banyak kendaraan dalam input. Komponen ini hanya berpengaruh ketika dua *state* memiliki total `ShippingFee` yang sama. Pada kondisi itu, *state* yang membawa kendaraan dengan sisa waktu lebih sedikit akan dipilih.
4. Kerapatan Posisi (`position_sum`): Koordinat x + y dari setiap kendaraan di geladak dijumlahkan, dikalikan ε = 1/(2n(W + L)), lalu dikurangkan dari nilai. Komponen ini membuat kendaraan cenderung berkumpul di satu sudut geladak, sehingga sisa ruang kosong tidak terpecah-pecah dan lebih mudah dimasuki kendaraan lain.

Secara matematis, nilai *objective* dihitung sebagai berikut:

$$Objective\,Value = Total\,ShippingFee - (P \times Pelanggaran) + (1/(2n) \times \Sigma\, 1/ETA) - (\varepsilon \times \Sigma\,(x + y))$$

Dengan:

$$P = \Sigma\, ShippingFee\ seluruh\ kendaraan + 1$$

$$Pelanggaran = overlap_cells + out_of_bound_cells + overweight$$

$$\varepsilon = 1 / (2n \times (W + L))$$

Besar bobot pada komponen ETA dan posisi tidak dipilih secara sembarang. ETA dalam input selalu bilangan bulat minimal 1, sehingga 1/ETA tidak pernah melebihi 1. Jika seluruh n kendaraan ada di geladak, jumlahnya paling besar n, dan setelah dikalikan 1/(2n) hasilnya paling besar 0,5. Di sisi lain, setiap kendaraan yang berada di geladak memiliki x + y yang lebih kecil dari W + L, sehingga komponen posisi juga selalu di bawah 0,5. Kedua komponen ini bila digabung tidak pernah mencapai 1. Karena `ShippingFee` berupa bilangan bulat, dua *state* dengan total `ShippingFee` berbeda pasti selisihnya minimal 1, sehingga komponen ETA dan posisi tidak akan pernah membalik urutan keduanya.

Hal yang sama berlaku untuk penalti. *State* valid paling buruk adalah geladak kosong dengan nilai 0. Sebaliknya, *state* dengan satu satuan pelanggaran saja bernilai paling tinggi total `ShippingFee` dikurangi P ditambah 0,5, yaitu −0,5. Jadi tidak ada *state* yang melanggar batasan yang bisa mengalahkan *state* yang valid.

Dengan susunan ini, pencarian pertama-tama akan menghilangkan pelanggaran, kemudian menaikkan total `ShippingFee`, dan baru memakai ETA serta posisi untuk memilih di antara *state* yang setara. Perlu dicatat bahwa nilai yang ditampilkan sebagai hasil akhir eksperimen adalah total `ShippingFee` dari *state* akhir beserta keterangan valid atau tidaknya, sebab komponen ETA dan posisi hanya alat bantu pencarian dan tidak memiliki makna bisnis.

Fungsi ini diimplementasikan sebagai `calculate_objective` pada file `objective.py`, dengan bobot dihitung terpisah di fungsi `calculate_weights`.

Tabel 3. Implementasi *Objective Function*

Kategori	Nama	Deskripsi
Parameter	`state`	Representasi konfigurasi penempatan kendaraan yang akan dihitung skornya.
	`data`	Data input berisi dimensi geladak, kapasitas kapal, dan daftar kendaraan.
	`detail`	Penanda apakah fungsi mengembalikan rincian perhitungan (*True*) atau hanya nilai *objective* (*False*).
Variabel	`W, L`	Lebar dan panjang geladak kapal.
	`capacity`	Kapasitas angkut maksimum kapal.
	`vehicle`	*Dictionary* pemetaan id kendaraan ke data kendaraan.
	`penalty`	Bobot penalti P, yaitu total ShippingFee seluruh kendaraan ditambah 1.
	`eta_scale`	Bobot komponen ETA, yaitu 1/(2n).
	`epsilon`	Bobot komponen posisi, yaitu 1/(2n(W + L)).

	`grid`	Grid berukuran W × L yang menyimpan banyak kendaraan pada setiap sel.
	`total_fee`	Total ShippingFee kendaraan di dalam geladak.
	`eta_sum`	Jumlah 1/ETA kendaraan di dalam geladak.
	`total_eta`	Jumlah ETA kendaraan di dalam geladak, untuk menghitung rata-rata ETA.
	`loaded`	Jumlah kendaraan di dalam geladak.
	`total_weight`	Total berat kendaraan di dalam geladak.
	`out_of_bound_cells`	Jumlah sel kendaraan yang berada di luar batas geladak.
	`position_sum`	Jumlah koordinat x + y kendaraan di dalam geladak.
	`Overlap_cells`	Jumlah sel yang ditempati lebih dari satu kendaraan.
	`overweight`	Kelebihan berat terhadap kapasitas kapal.
	`violation`	Total pelanggaran, yaitu overlap_cells + `out_of_bound_cells` + `overweight`.
	`eta_term`	Nilai komponen ETA, yaitu eta_scale × eta_sum.
	`objective`	Hasil akhir perhitungan skor *objective function*.
Method	`calculate_objective`	Menghitung skor *objective function* dari suatu *state*.
	`calculate_weights`	Menghitung bobot penalti, bobot ETA, dan bobot posisi berdasarkan data input.
	`get_size`	Menghitung lebar dan panjang kendaraan sesuai orientasinya.

Algoritma

```
import json

def load_input(path):
```

```
    with open(path) as f:
        return json.load(f)


def get_size(vehicle, orientation):
    w, l = vehicle["Dimensions"]["w"], vehicle["Dimensions"]["l"]
    return (w, l) if orientation == "Horizontal" else (l, w)

def calculate_weights(data):
    W = data["Ship"]["Dimensions"]["w"]
    L = data["Ship"]["Dimensions"]["l"]
    n = len(data["Vehicles"])
    if any(v["ETA"] < 1 for v in data["Vehicles"]):
        raise ValueError("ETA harus bilangan bulat minimal 1")
    penalty = sum(v["ShippingFee"] for v in data["Vehicles"]) + 1
     eta_scale = 1 / (2 * n)           # total suku ETA paling besar
0.5
    epsilon = 1 / (2 * n * (W + L))  # total suku posisi di bawah 0.5
    return penalty, eta_scale, epsilon

def calculate_objective(state, data, detail=False):
    W = data["Ship"]["Dimensions"]["w"]
    L = data["Ship"]["Dimensions"]["l"]
    capacity = data["Ship"]["MaxCapacity"]
    vehicles = {v["id"]: v for v in data["Vehicles"]}
    penalty, eta_scale, epsilon = calculate_weights(data)

    grid = [[0] * L for _ in range(W)]
    total_fee = 0
    eta_sum = 0
    total_eta = 0
    loaded = 0
    total_weight = 0
    out_of_bound_cells = 0
    position_sum = 0
    for vid, s in state.items():
        if not s["in_ship"]:
            continue
        v = vehicles[vid]
        width, length = get_size(v, s["Orientation"])
        total_fee += v["ShippingFee"]
        eta_sum += 1 / v["ETA"]
        total_eta += v["ETA"]
        loaded += 1
        total_weight += v["Weight"]
        position_sum += s["x"] + s["y"]
        for i in range(s["x"], s["x"] + width):
            for j in range(s["y"], s["y"] + length):
                if 0 <= i < W and 0 <= j < L:
```

```
                    grid[i][j] += 1
                else:
                    out_of_bound_cells += 1

    overlap_cells = sum(max(0, c - 1) for col in grid for c in col)
    overweight = max(0, total_weight - capacity)

    violation = overlap_cells + out_of_bound_cells + overweight
    eta_term = eta_scale * eta_sum
     objective = total_fee - penalty * violation + eta_term - epsilon
* position_sum
    if detail:
        return {
            "total_fee": total_fee,
            "eta_term": eta_term,
            "loaded_vehicles": loaded,
            "average_eta": total_eta / loaded if loaded else 0,
            "total_weight": total_weight,
            "overlap_cells": overlap_cells,
            "out_of_bound_cells": out_of_bound_cells,
            "overweight": overweight,
            "penalty_weight": penalty,
            "epsilon": epsilon,
            "position_sum": position_sum,
            "valid": violation == 0,
            "objective": objective,
        }
    return objective
```

2.3 Pemilihan Fungsi Initial State

Sesuai ketentuan tugas, state awal dibangkitkan secara acak melalui fungsi generate_random_state pada file util.py. Fungsi ini memproses kendaraan satu per satu. Pertama, setiap kendaraan diberi orientasi acak, Horizontal atau Vertical. Jika dengan orientasi tersebut kendaraan tidak muat di geladak, orientasinya langsung dibalik. Selanjutnya, setiap kendaraan memiliki peluang 50% untuk ditempatkan di geladak. Kendaraan yang ditempatkan akan mendapat koordinat x dan y acak, dengan rentang yang sudah dibatasi agar seluruh badan kendaraan tetap berada di dalam geladak. Kendaraan yang tidak ditempatkan dicatat berada di luar kapal.

Pada tahap ini, program sengaja tidak memeriksa tumpang tindih antarkendaraan maupun total berat. Akibatnya, state awal hampir selalu melanggar batasan, terutama pada input berukuran besar ketika belasan kendaraan diletakkan secara acak di geladak yang sama. Pilihan ini diambil agar state awal benar-benar acak sesuai spesifikasi, sementara tugas memperbaiki pelanggaran diserahkan kepada objective function melalui komponen penalti dan kepada algoritma pencarian itu sendiri.

Fungsi generate_random_state menerima parameter seed yang mengatur pembangkit bilangan acak. Dengan seed yang sama, fungsi ini selalu menghasilkan state awal yang identik. Kelompok kami memanfaatkan sifat ini untuk dua tujuan. Pertama, setiap algoritma dijalankan dengan seed 1, 2, dan 3 untuk ketiga percobaannya, sehingga semua algoritma mulai dari titik awal yang sama dan hasilnya dapat dibandingkan secara adil. Kedua, karena tiap percobaan memakai seed berbeda, konsistensi setiap algoritma tetap dapat diuji dari beberapa titik awal. Seed juga membuat setiap eksperimen dapat diulang dengan hasil yang sama saat demonstrasi.

Tabel 4. Implementasi Inisialisasi *State* Awal

Kategori	Nama	Deskripsi
Parameter	`data`	Data input berisi ukuran geladak dan daftar kendaraan.
	`seed`	Bilangan pengatur pembangkit acak, agar state awal bisa dibangkitkan ulang dengan hasil yang sama.
Variabel	`rng`	Pembangkit bilangan acak yang dibuat dari seed.
	`W, L`	Lebar dan panjang geladak.
	`state`	Dictionary hasil inisialisasi yang berisi status setiap kendaraan.
	`orientation`	Orientasi acak yang dipilih untuk kendaraan.
	`Width, length`	Ukuran kendaraan sesuai orientasi terpilih.

	`x, y`	Koordinat acak kendaraan di geladak, dipilih agar kendaraan tidak keluar batas.
Konstanta	`ORIENTATIONS`	Daftar orientasi yang mungkin, yaitu Horizontal dan Vertical.
Method	`generate_random_state`	Membangkitkan state awal secara acak. Setiap kendaraan punya peluang 50% untuk ditempatkan di geladak pada posisi acak.
	`fits_in_deck`	Memeriksa apakah kendaraan dengan orientasi tertentu muat di geladak.
	`get_size`	Menentukan lebar dan panjang kendaraan sesuai orientasinya.

Algoritma

```
import random
from objective import get_size

ORIENTATIONS = ["Horizontal", "Vertical"]


def fits_in_deck(vehicle, orientation, W, L):
    width, length = get_size(vehicle, orientation)
    return width <= W and length <= L


def generate_random_state(data, seed=None):
    rng = random.Random(seed)
    W = data["Ship"]["Dimensions"]["w"]
    L = data["Ship"]["Dimensions"]["l"]

    state = {}
    for v in data["Vehicles"]:
        orientation = rng.choice(ORIENTATIONS)

        if not fits_in_deck(v, orientation, W, L):
             orientation = "Vertical" if orientation == "Horizontal"
else "Horizontal"

          if rng.random() < 0.5 and fits_in_deck(v, orientation, W,
L):
            width, length = get_size(v, orientation)
            x = rng.randint(0, W - width)
            y = rng.randint(0, L - length)
                state[v["id"]] = {"in_ship": True, "x": x, "y": y,
```

```
"Orientation": orientation}
        else:
                state[v["id"]] = {"in_ship": False, "x": None, "y":
None, "Orientation": orientation}

    return state
```

2.4 Pemilihan Fungsi Neighbor

Fungsi tetangga menentukan perubahan apa saja yang boleh dilakukan terhadap sebuah state dalam satu langkah pencarian. Implementasinya terdapat pada file neighbor.py dan mengikuti tiga jenis gerakan yang diizinkan spesifikasi, yaitu menukar dua kendaraan, memindahkan sebuah kendaraan, dan mengubah orientasi kendaraan.

1. Menukar dua kendaraan (`swap_vehicles`). Gerakan ini selalu melibatkan setidaknya satu kendaraan yang berada di geladak. Jika kedua kendaraan berada di geladak, keduanya bertukar koordinat dengan tetap memakai orientasinya masing-masing. Jika salah satu kendaraan berada di luar kapal, kendaraan yang di geladak dikeluarkan, lalu kendaraan dari luar menempati koordinatnya. Pertukaran dua kendaraan yang sama-sama di luar kapal tidak dimasukkan karena tidak mengubah apa pun. Apabila kendaraan yang dipindahkan lebih besar dari ruang di koordinat tujuan, koordinatnya digeser secukupnya agar tidak keluar dari geladak.
2. Memindahkan sebuah kendaraan (`move_vehicle`). Kendaraan dipindahkan ke koordinat acak di geladak dengan orientasinya saat ini. Untuk kendaraan yang sudah berada di geladak, terdapat peluang 30% kendaraan tersebut dikeluarkan dari kapal. Kami menafsirkan pengeluaran ini sebagai pemindahan kendaraan ke luar geladak. Gerakan ini diperlukan karena tanpanya jumlah kendaraan di geladak tidak akan pernah bisa berkurang: pertukaran selalu memasukkan satu kendaraan untuk setiap kendaraan yang keluar, pemindahan dari luar selalu menambah kendaraan, dan perubahan orientasi tidak mengubah jumlah. Padahal, *state* awal yang acak bisa saja memuat lebih banyak kendaraan daripada yang sanggup ditampung kapal.

3. Mengubah orientasi kendaraan (`rotate_vehicle`). Gerakan ini hanya diterapkan pada kendaraan yang berada di geladak, karena memutar kendaraan di luar kapal tidak memengaruhi nilai objective sama sekali. Setelah diputar, koordinat kendaraan disesuaikan agar tetap berada di dalam geladak. Jika kendaraan tidak muat di geladak setelah diputar, gerakan dibatalkan dan state tidak berubah.

Gerakan-gerakan tersebut disediakan dalam dua bentuk sesuai kebutuhan algoritma. Fungsi `random_neighbor` memilih satu jenis gerakan dan satu kendaraan secara acak, lalu menghasilkan satu tetangga. Jika tetangga yang dihasilkan ternyata sama persis dengan state asal, fungsi akan mencoba ulang hingga 20 kali. Pengulangan ini mencegah algoritma membuang iterasi pada langkah yang tidak mengubah apa pun. Fungsi ini digunakan oleh *Stochastic Hill-Climbing, Simulated Annealing,* dan mutasi *Genetic Algorithm* yang hanya membutuhkan satu tetangga acak pada setiap iterasi.

Sementara itu, fungsi `all_neighbors` mendaftar seluruh tetangga yang dapat dicapai dalam satu gerakan, yaitu semua pasangan pertukaran, semua perubahan orientasi yang valid, semua pemindahan ke setiap koordinat yang muat, serta semua pengeluaran kendaraan dari kapal. Fungsi ini digunakan oleh *Steepest Ascent Hill-Climbing* dan *Hill-Climbing with Sideways Move* yang harus membandingkan seluruh tetangga untuk memilih yang terbaik. Banyaknya tetangga bergantung pada jumlah kendaraan dan ukuran geladak. Semakin banyak kendaraan dan semakin luas geladak, semakin banyak pula tetangga yang harus dievaluasi pada setiap iterasi, dengan penyumbang terbesar berasal dari pemindahan kendaraan ke berbagai koordinat.

Tetangga yang dihasilkan tidak harus valid. Sebuah gerakan boleh saja menyebabkan kendaraan bertumpuk atau total berat melebihi kapasitas. Kelompok kami memilih pendekatan ini agar algoritma dapat melewati state yang melanggar batasan sebagai jalan menuju *state* valid yang lebih baik, sementara komponen penalti pada *objective function* memastikan bahwa state seperti itu selalu dinilai lebih rendah daripada *state* yang valid.

Tabel 5. Implementasi Fungsi *Neighbor*

Kategori	Nama	Deskripsi
Parameter	`state`	State saat ini yang akan dicari tetangganya.
	`data`	Data input berisi ukuran geladak dan daftar kendaraan.
	`rng`	Pembangkit bilangan acak untuk memilih jenis gerakan, kendaraan, dan koordinat tujuan, sehingga tetangga yang sama dapat dibangkitkan ulang dengan seed yang sama.
	`vid, vid_a, vid_b`	Id kendaraan yang dikenai gerakan.
	`max_try`	Batas percobaan untuk mendapatkan tetangga yang berbeda dari state asal.
	`k`	Banyak tetangga acak yang diambil pada sample_neighbors.
Variabel	`new`	Salinan state yang sudah diubah oleh gerakan.
	`move_type`	Jenis gerakan yang dipilih secara acak, yaitu swap, move, atau rotate.
	`in_ship`	*Daftar id kendaraan yang sedang berada di geladak.*
	`result`	Daftar seluruh tetangga beserta keterangan gerakannya pada all_neighbors.
Konstanta	`MOVE_OUT_PROBABILITY`	Peluang (0,3) sebuah kendaraan di geladak dikeluarkan dari kapal saat gerakan move.
Method	`swap_vehicles`	Menukar posisi dua kendaraan. Jika salah satunya di luar kapal, kendaraan yang di dalam keluar dan kendaraan yang di luar masuk ke posisinya.
	`move_vehicle`	Memindahkan kendaraan ke koordinat acak di geladak, atau mengeluarkannya dari kapal dengan peluang MOVE_OUT_PROBABILITY.
	`rotate_vehicle`	Mengubah orientasi kendaraan di geladak.

		Jika setelah diputar tidak muat, state tidak berubah.
	`move_to`	Menempatkan kendaraan di koordinat dan orientasi tertentu.
	`move_out`	Mengeluarkan kendaraan dari kapal.
	`random_neighbor`	Menghasilkan satu tetangga acak yang pasti berbeda dari state asal. Dipakai Stochastic Hill-Climbing dan Simulated Annealing.
	`sample_neighbors`	Menghasilkan k tetangga acak sekaligus.
	`all_neighbors`	Menghasilkan seluruh tetangga dari state. Dipakai Steepest Ascent dan Sideways Move.
	`copy_state`	Membuat salinan state agar state asal tidak ikut berubah.
	`_fit_inside`	Menyesuaikan orientasi dan koordinat agar kendaraan tetap berada di dalam geladak.

Algoritma

```
import random
from objective import get_size

MOVE_OUT_PROBABILITY = 0.3


def copy_state(state):
    return {vid: dict(s) for vid, s in state.items()}


def _deck(data):
                          return      data["Ship"]["Dimensions"]["w"],
data["Ship"]["Dimensions"]["l"]


def _vehicle_map(data):
    return {v["id"]: v for v in data["Vehicles"]}


def _fit_inside(vehicle, orientation, x, y, W, L):
        for ori in (orientation, "Vertical" if orientation ==
"Horizontal" else "Horizontal"):
```

```
        width, length = get_size(vehicle, ori)
        if width <= W and length <= L:
            return ori, min(max(x, 0), W - width), min(max(y, 0), L -
length)
    return None


def swap_vehicles(state, data, vid_a, vid_b):
    W, L = _deck(data)
    vehicles = _vehicle_map(data)
    new = copy_state(state)
    a, b = state[vid_a], state[vid_b]

    for vid, target in ((vid_a, b), (vid_b, a)):
        if target["in_ship"]:
                                placed  =  _fit_inside(vehicles[vid],
new[vid]["Orientation"], target["x"], target["y"], W, L)
            if placed:
                ori, x, y = placed
                       new[vid] = {"in_ship": True, "x": x, "y": y,
"Orientation": ori}
                continue
             new[vid] = {"in_ship": False, "x": None, "y": None,
"Orientation": new[vid]["Orientation"]}
    return new


def move_vehicle(state, data, vid, rng=random):
    W, L = _deck(data)
    vehicle = _vehicle_map(data)[vid]
    new = copy_state(state)
    s = new[vid]

    if s["in_ship"] and rng.random() < MOVE_OUT_PROBABILITY:
            new[vid] = {"in_ship": False, "x": None, "y": None,
"Orientation": s["Orientation"]}
        return new

    placed = _fit_inside(vehicle, s["Orientation"], 0, 0, W, L)
    if placed is None:
        return new
    ori, _, _ = placed
    width, length = get_size(vehicle, ori)

    for _ in range(20):
        x = rng.randint(0, W - width)
        y = rng.randint(0, L - length)
         if not (s["in_ship"] and x == s["x"] and y == s["y"] and ori
== s["Orientation"]):
```

```
            break
    new[vid] = {"in_ship": True, "x": x, "y": y, "Orientation": ori}
    return new

def move_to(state, vid, x, y, orientation):
    new = copy_state(state)
      new[vid] = {"in_ship": True, "x": x, "y": y, "Orientation":
orientation}
    return new


def move_out(state, vid):
    new = copy_state(state)
         new[vid] = {"in_ship": False, "x": None, "y": None,
"Orientation": state[vid]["Orientation"]}
    return new


def rotate_vehicle(state, data, vid):
    W, L = _deck(data)
    vehicle = _vehicle_map(data)[vid]
    new = copy_state(state)
    s = new[vid]
     new_ori = "Vertical" if s["Orientation"] == "Horizontal" else
"Horizontal"

    if not s["in_ship"]:
        new[vid]["Orientation"] = new_ori
        return new

    width, length = get_size(vehicle, new_ori)
    if width > W or length > L:
        return new

    x = min(s["x"], W - width)
    y = min(s["y"], L - length)
      new[vid] = {"in_ship": True, "x": x, "y": y, "Orientation":
new_ori}
    return new


def _one_neighbor(state, data, rng):
    ids = list(state.keys())
    in_ship = [vid for vid in ids if state[vid]["in_ship"]]
    move_type = rng.choice(["swap", "move", "rotate"])

    if move_type in ("swap", "rotate") and not in_ship:
        move_type = "move"
    if move_type == "swap" and len(ids) < 2:
```

```
        move_type = "move"

    if move_type == "swap":
        vid_a = rng.choice(in_ship)
        vid_b = rng.choice([vid for vid in ids if vid != vid_a])
           return swap_vehicles(state, data, vid_a, vid_b), ("swap",
vid_a, vid_b)

    if move_type == "rotate":
        vid = rng.choice(in_ship)
        return rotate_vehicle(state, data, vid), ("rotate", vid)

    vid = rng.choice(ids)
    return move_vehicle(state, data, vid, rng), ("move", vid)


def random_neighbor(state, data, rng=random, max_try=20):
    for _ in range(max_try):
        new, move = _one_neighbor(state, data, rng)
        if new != state:
            return new, move
    return new, move

def sample_neighbors(state, data, k, rng=random):
    return [random_neighbor(state, data, rng) for _ in range(k)]

def all_neighbors(state, data):
    W, L = _deck(data)
    vehicles = _vehicle_map(data)
    ids = list(state.keys())
    result = []

    for a in ids:
        if not state[a]["in_ship"]:
            continue
        for b in ids:
            if b == a or (state[b]["in_ship"] and b < a):
                continue
            new = swap_vehicles(state, data, a, b)
            if new != state:
                result.append((new, ("swap", a, b)))

    for vid in ids:
        if state[vid]["in_ship"]:
            new = rotate_vehicle(state, data, vid)
            if new != state:
                result.append((new, ("rotate", vid)))

    for vid in ids:
```

```
        s = state[vid]
          placed = _fit_inside(vehicles[vid], s["Orientation"], 0, 0,
W, L)
        if placed is not None:
            ori, _, _ = placed
            width, length = get_size(vehicles[vid], ori)
            for x in range(W - width + 1):
                for y in range(L - length + 1):
                        if s["in_ship"] and (x, y, ori) == (s["x"],
s["y"], s["Orientation"]):
                        continue
                    result.append((move_to(state, vid, x, y, ori),
("move", vid, x, y)))
        if s["in_ship"]:
            result.append((move_out(state, vid), ("move_out", vid)))

    return result
```

2.5 Heuristik yang Digunakan

Selain algoritma local search itu sendiri, kelompok kami menambahkan beberapa heuristik agar pencarian lebih terarah dan tidak membuang iterasi. Heuristik di sini berarti aturan bantu yang tidak menjamin hasil terbaik, tetapi membuat pencarian lebih efisien. Seluruhnya tetap berada dalam lingkup local search karena tidak mengubah cara kerja dasar setiap algoritma.

1. **Penalti besar untuk pelanggaran batasan.** State yang tidak valid tidak dibuang, tetapi diberi pengurangan nilai yang lebih besar daripada total ShippingFee seluruh kendaraan. Dengan cara ini, state valid apa pun selalu bernilai lebih tinggi daripada state tidak valid, sehingga pencarian terdorong memperbaiki pelanggaran terlebih dahulu sebelum menambah keuntungan.
2. **Pemecah seri berdasarkan ETA dan posisi.** Dua state dengan total ShippingFee yang sama dibedakan oleh dua komponen kecil. Komponen pertama memberi nilai sedikit lebih tinggi pada kendaraan dengan ETA lebih pendek. Komponen kedua memberi nilai sedikit lebih tinggi pada kendaraan yang diletakkan lebih rapat ke pojok geladak, sehingga ruang kosong terkumpul di satu sisi. Kedua komponen ini dibuat sangat kecil sehingga tidak pernah mengalahkan perbedaan ShippingFee.

3. **Peluang terbatas untuk mengeluarkan kendaraan.** Saat memindahkan kendaraan secara acak, peluang kendaraan dipindahkan ke luar kapal dibatasi 30%. Tanpa batas ini, terlalu banyak langkah acak akan mengosongkan kapal dan menurunkan ShippingFee.
4. **Menghindari tetangga yang identik.** Fungsi tetangga acak mencoba ulang apabila langkah yang dihasilkan tidak mengubah state, misalnya memutar kendaraan yang tidak muat setelah diputar. Rotasi juga hanya diterapkan pada kendaraan di dalam kapal. Hal ini mencegah Stochastic Hill-Climbing, Simulated Annealing, dan mutasi pada Genetic Algorithm membuang iterasi untuk langkah yang tidak berpengaruh.
5. **Catatan susunan yang sudah dikunjungi pada sideways move.** Hill-Climbing with Sideways Move menyimpan susunan geladak yang sudah dilewati selama berada di daerah datar (plateau). Algoritma hanya berpindah ke susunan bernilai sama yang belum pernah dikunjungi, dan berhenti jika semuanya sudah dikunjungi. Ini mencegah pencarian bolak-balik antara dua susunan yang sama, misalnya saat dua kendaraan berukuran sama terus ditukar.
6. **Pemilihan acak saat terjadi seri.** Pada Sideways Move dan Random Restart, jika ada beberapa tetangga terbaik dengan nilai yang sama, salah satunya dipilih secara acak, bukan selalu yang pertama ditemukan. Dengan begitu, pencarian tidak selalu condong ke arah yang sama.

2.6 Implementasi Algoritma *Local Search*

2.6.1 Steepest Ascent Hill Climbing

Steepest Ascent Hill-Climbing adalah varian *hill-climbing* yang selalu memeriksa seluruh tetangga sebelum melangkah. Pada setiap iterasi, algoritma menghitung nilai *objective* dari semua *state* yang bisa dicapai dengan satu gerakan, lalu berpindah ke tetangga dengan nilai tertinggi. Dengan cara ini, setiap langkah selalu merupakan perbaikan terbesar yang tersedia. Kekurangannya, algoritma tidak memiliki cara untuk keluar dari *local optimum*, karena pencarian langsung berhenti begitu tidak ada tetangga yang lebih baik.

Pencarian dimulai dengan membangkitkan *state* awal acak melalui `generate_random_state` menggunakan *seed* yang diberikan. Pada setiap iterasi, fungsi `all_neighbors` menghasilkan seluruh tetangga dari *state* saat ini, kemudian setiap tetangga dihitung nilai *objective*-nya untuk mencari yang terbaik. Jika terdapat beberapa tetangga dengan nilai tertinggi yang sama, algoritma memilih yang pertama kali ditemukan. Apabila nilai tetangga terbaik lebih tinggi daripada *state* saat ini, algoritma berpindah ke tetangga tersebut dan iterasi dilanjutkan. Apabila tidak, algoritma dianggap telah mencapai *local optimum* dan pencarian dihentikan. Pencarian juga berhenti jika jumlah iterasi mencapai `max_iterations`. Selama proses berlangsung, algoritma mencatat nilai *objective* di setiap iterasi, jumlah tetangga yang telah dievaluasi, dan alasan berhenti.

Implementasi algoritma *Steepest Ascent Hill-Climbing* terdapat dalam file `hc_steepest_ascent.py`.

Tabel 6. Implementasi *Steepest Ascent Hill-Climbing*

Kategori	Nama	Deskripsi
Parameter	`data`	Data input berisi ukuran geladak, kapasitas kapal, dan daftar kendaraan.
	`seed`	Bilangan pengatur pembangkit acak untuk *state* awal.
	`max_iterations`	Batas maksimum iterasi pencarian.
Variabel	`current_state`	*State* yang sedang ditempati algoritma.
	`initial_state`	Salinan *state* awal untuk keperluan visualisasi.
	`current_score`	Nilai *objective* dari `current_state`.
	`neighbors`	Daftar seluruh tetangga dari `current_state` beserta keterangan gerakannya.
	`neighbor_state`, `neighbor_move`	Tetangga yang sedang dievaluasi dan gerakan yang menghasilkannya.

	`neighbor_score`	Nilai *objective* dari `neighbor_state`.
	`best_state`, `best_score`	Tetangga terbaik pada iterasi saat ini beserta nilai *objective*-nya.
	`score_list`	Daftar nilai *objective* pada setiap iterasi, untuk plot.
	`iterations`	Jumlah iterasi hingga pencarian berhenti.
	`neighbor_evaluated`	Jumlah seluruh tetangga yang sudah dievaluasi selama pencarian.
	`stop_reason`	Alasan pencarian berhenti, yaitu `local_optimum`, `no_neighbor`, atau `max_iterations`.
	`start_time, duration`	Waktu mulai dan total durasi proses pencarian.
	`final_detail`	Rincian perhitungan *objective* dari *state* akhir.
Method	`steepest_ascent`	Menjalankan *Steepest Ascent Hill-Climbing* dan mengembalikan *state* awal, *state* akhir, riwayat nilai *objective*, jumlah iterasi, jumlah tetangga yang dievaluasi, alasan berhenti, serta durasi.
	`generate_random_state`	Membangkitkan *state* awal acak (dari `util.py`).
	`all_neighbors`	Membangkitkan seluruh tetangga dari sebuah *state* (dari `neighbor.py`).
	`calculate_objective`	Menghitung nilai *objective* sebuah *state* (dari `objective.py`).
	`copy_state`	Membuat salinan *state* (dari `neighbor.py`).

Algoritma

```
import time
from neighbor import all_neighbors, copy_state
from objective import calculate_objective
from util import generate_random_state
```

```
def steepest_ascent(data, seed=None, max_iterations=1000):
    current_state = generate_random_state(data, seed)
    initial_state = copy_state(current_state)
         current_score  =  calculate_objective(current_state,  data,
detail=False)
    score_list = [current_score]
    iterations = 0
    neighbor_evaluated = 0
    start_time = time.perf_counter()
    stop_reason = "max_iterations"

    for i in range (max_iterations):
        neighbors = all_neighbors(current_state, data)

        if not neighbors:
            stop_reason = "no_neighbor"
            break

        best_state = None
        best_score = None

        for neighbor_state, neighbor_move in neighbors:
                 neighbor_score  =  calculate_objective(neighbor_state,
data, detail=False)
            neighbor_evaluated += 1
            if best_score is None or neighbor_score > best_score :
                best_state = neighbor_state
                best_score = neighbor_score

        if best_score > current_score :
            current_state = best_state
            current_score = best_score
            iterations += 1
            score_list.append(current_score)
        else :
            stop_reason = "local_optimum"
            break

    duration = time.perf_counter() - start_time
          final_detail  =  calculate_objective(current_state,  data,
detail=True)

    return {
        "algorithm" : "steepest_ascent",
        "seed" : seed,
        "max_iterations" : max_iterations,
        "initial_state" : initial_state,
        "final_state" : current_state,
        "initial_objective" : score_list[0],
```

```
        "final_objective" : current_score,
        "final_total_fee" : final_detail["total_fee"],
        "final_valid" : final_detail["valid"],
        "objective_history" : score_list,
        "iterations" : iterations,
        "duration_seconds" : duration,
        "extra" : {
            "neighbors_evaluated" : neighbor_evaluated,
            "stop_reason" : stop_reason
        }
    }
```

2.6.2 *Stochastic Hill-Climbing*

Stochastic Hill-Climbing adalah varian *hill-climbing* yang tidak memeriksa seluruh tetangga pada setiap langkah. Algoritma hanya membangkitkan satu tetangga secara acak, lalu berpindah ke tetangga tersebut apabila nilainya lebih tinggi daripada *state* saat ini. Karena hanya satu tetangga yang dievaluasi per iterasi, setiap iterasi berjalan sangat cepat, tetapi arah pencarian ditentukan oleh keberuntungan dalam memilih tetangga, bukan oleh perbandingan seluruh kemungkinan langkah.

Pencarian dimulai dengan membangkitkan *state* awal acak melalui `generate_random_state` menggunakan *seed* yang diberikan, kemudian menghitung nilai *objective*-nya. Pada setiap iterasi, fungsi `random_neighbor` menghasilkan satu tetangga acak yang berbeda dari *state* saat ini, menggunakan salah satu dari tiga gerakan yang diizinkan. Jika nilai *objective* tetangga lebih tinggi, *state* saat ini diganti dengan tetangga tersebut dan penghitung kegagalan dikembalikan ke nol. Jika tidak lebih tinggi, tetangga ditolak dan penghitung kegagalan bertambah satu. Pencarian berhenti ketika penghitung kegagalan mencapai batas `patience`, yang menandakan bahwa algoritma sudah terlalu lama tidak menemukan perbaikan dan kemungkinan besar berada di *local optimum*, atau ketika jumlah iterasi mencapai `max_iterations`. Setiap tetangga yang dievaluasi, baik diterima maupun ditolak, dihitung sebagai satu iterasi.

Implementasi algoritma S*tochastic Hill-Climbing* terdapat dalam file `hc_stochastic.py`.

Tabel 7. Implementasi *Stochastic Hill-Climbing*

Kategori	Nama	Deskripsi
Parameter	`data`	Data input berisi ukuran geladak, kapasitas kapal, dan daftar kendaraan.
	`seed`	Bilangan pengatur pembangkit acak untuk *state* awal dan pemilihan tetangga.
	`max_iterations`	Batas maksimum iterasi pencarian.
	`patience`	Batas jumlah tetangga berturut-turut yang tidak lebih baik sebelum pencarian dihentikan.
Variabel	`rng`	Pembangkit bilangan acak yang dibuat dari *seed*.
	`current_state`	*State* yang sedang ditempati algoritma.
	`initial_state`	Salinan *state* awal untuk keperluan visualisasi.
	`current_score`	Nilai *objective* dari `current_state`.
	`neighbor_state`	Tetangga acak yang dibangkitkan pada iterasi saat ini.
	`neighbor_move`	Keterangan gerakan yang menghasilkan `neighbor_state`.
	`neighbor_score`	Nilai *objective* dari `neighbor_state`.
	`score_list`	Daftar nilai *objective* `current_state` pada setiap iterasi, untuk plot.
	`worse_move_count`	Jumlah tetangga berturut-turut yang tidak lebih baik dari `current_state`.
	`iterations`	Jumlah iterasi yang sudah dilakukan.
	`stop_reason`	Alasan pencarian berhenti, yaitu `patience` atau `max_iterations`.
	`accepted_moves`	Jumlah langkah (*neighbor*) yang diterima karena skor *objective*-nya lebih baik daripada *state* saat ini.

	`start_time, duration`	Waktu mulai dan total durasi proses pencarian.
	`final_detail`	Rincian perhitungan *objective* dari *state* akhir.
Method	`stochastic_hill_climbing`	Menjalankan *Stochastic Hill-Climbing* dan mengembalikan *state* awal, *state* akhir, riwayat nilai *objective*, jumlah iterasi, serta durasi.
	`generate_random_state`	Membangkitkan *state* awal acak (dari `util.py`).
	`random_neighbor`	Membangkitkan satu tetangga acak yang berbeda dari *state* asal (dari `neighbor.py`).
	`calculate_objective`	Menghitung nilai *objective* sebuah *state* (dari `objective.py`).
	`copy_state`	Membuat salinan *state* (dari `neighbor.py`).

Algoritma

```
import random
import time
from objective import calculate_objective
from neighbor import random_neighbor, copy_state
from util import generate_random_state

def stochastic_hill_climbing(data, seed=None, max_iterations=1000,
patience=100):
    rng = random.Random(seed)
    current_state = generate_random_state(data, seed)
    initial_state = copy_state(current_state)
        current_score = calculate_objective(current_state, data,
detail=False)
    score_list = [current_score]
    worse_move_count = 0
    iterations = 0
    start_time = time.perf_counter()
    stop_reason = "max_iterations"
    accepted_moves = 0

    for i in range (max_iterations):
                                neighbor_state, neighbor_move =
random_neighbor(current_state, data, rng)
```

```
            neighbor_score = calculate_objective(neighbor_state, data,
detail=False)

        if neighbor_score > current_score :
            current_state = neighbor_state
            current_score = neighbor_score
            worse_move_count = 0
            accepted_moves += 1
        else :
            worse_move_count += 1

        score_list.append(current_score)
        iterations += 1

        if worse_move_count >= patience :
            stop_reason = "patience"
            break

    duration = time.perf_counter() - start_time
          final_detail   =   calculate_objective(current_state,   data,
detail=True)

    return {
        "algorithm" : "stochastic_hill_climbing",
        "seed" : seed,
        "max_iterations" : max_iterations,
        "patience" : patience,
        "initial_state" : initial_state,
        "final_state" : current_state,
        "initial_objective" : score_list[0],
        "final_objective" : current_score,
        "final_total_fee" : final_detail["total_fee"],
        "final_valid" : final_detail["valid"],
        "objective_history" : score_list,
        "iterations" : iterations,
        "duration_seconds" : duration,
        "extra" : {
            "stop_reason" : stop_reason,
            "accepted_moves" : accepted_moves,
            "final_worse_count" : worse_move_count,
            }
    }
```

2.6.3 *Hill Climbing with Sideways Move*

Hill-Climbing with Sideways Move adalah varian hill-climbing yang tetap mau berpindah ketika tetangga terbaik memiliki nilai yang sama dengan state saat ini. Perpindahan seperti ini disebut sideways move. Tujuannya adalah agar pencarian tidak langsung berhenti ketika berada di plateau, yaitu daerah yang nilainya datar, sehingga algoritma berkesempatan menemukan jalan naik di sisi lain plateau tersebut. Agar pencarian tidak berjalan tanpa akhir di daerah datar, jumlah sideways move dibatasi oleh parameter max_sideways.

Pencarian dimulai dari state awal acak hasil seed. Pada setiap iterasi, seluruh tetangga dibangkitkan dengan fungsi all_neighbors dan dihitung nilai objective-nya, kemudian dipilih tetangga dengan nilai tertinggi. Apabila terdapat beberapa tetangga dengan nilai tertinggi yang sama, salah satunya dipilih secara acak. Keputusan perpindahan mengikuti tiga kondisi berikut:

1. Jika nilai tetangga terbaik lebih tinggi daripada state saat ini, algoritma berpindah ke tetangga tersebut. Langkah ini dihitung sebagai uphill move.
2. Jika nilai tetangga terbaik sama dengan state saat ini, algoritma melakukan sideways move selama jumlah sideways move belum mencapai max_sideways. Jika batas tersebut sudah tercapai, pencarian dihentikan.
3. Jika nilai tetangga terbaik lebih rendah, algoritma sudah berada di local optimum dan pencarian dihentikan.

Selama berada di plateau, algoritma mencatat susunan geladak yang sudah dikunjungi dan tidak akan memilih susunan tersebut lagi. Catatan ini diperlukan karena menukar dua kendaraan berukuran sama menghasilkan state baru dengan nilai yang persis sama, padahal susunan geladaknya tidak berubah. Tanpa catatan ini, algoritma dapat menghabiskan seluruh jatah sideways move hanya untuk menukar dua kendaraan yang sama secara bolak-balik. Catatan tersebut dikosongkan setiap kali terjadi uphill move. Apabila semua tetangga yang bernilai sama sudah pernah dikunjungi, pencarian juga dihentikan. Jumlah sideways move dihitung secara total selama satu kali pencarian.

Implementasi algoritma Hill-Climbing with Sideways Move terdapat dalam file hc_sideways_move.py

Tabel 8. Implementasi Hill-Climbing with Sideways Move

Kategori	Nama	Deskripsi
Parameter	`data`	Data input berisi ukuran geladak, kapasitas kapal, dan daftar kendaraan.
	`initial_state`	State awal pencarian. Jika tidak diberikan, state awal dibangkitkan dari seed.
	`seed`	Bilangan pengatur pembangkit acak untuk state awal dan pemilihan di antara tetangga terbaik yang bernilai sama.
	`max_sideways`	Batas maksimum sideways move. Pencarian dihentikan ketika batas ini tercapai.
	`max_iterations`	Batas pengaman jumlah iterasi.
Konstanta	`TIE_TOLERANCE`	Batas selisih nilai objective agar dua nilai dianggap sama (10^{-9}).
Variabel	`rng`	Pembangkit bilangan acak yang dibuat dari seed.
	`vehicles`	Dictionary untuk mencari data kendaraan berdasarkan id.
	`current`	State yang sedang ditempati algoritma.
	`current_value`	Nilai objective dari current.
	`neighbors`	Daftar seluruh tetangga dari current beserta keterangan gerakannya.
	`scored`	Daftar tetangga beserta nilai objective masing-masing.
	`best_value`	Nilai objective tertinggi di antara seluruh tetangga.
	`best_candidates`	Daftar tetangga yang nilainya sama dengan best_value.
	`fresh`	Daftar tetangga terbaik yang susunan

		geladaknya belum pernah dikunjungi selama di plateau.
	`best_state, best_move`	Tetangga terpilih dan gerakan yang menghasilkannya.
	`plateau_visited`	Himpunan susunan geladak yang sudah dikunjungi selama berada di plateau.
	`history`	Daftar nilai objective pada setiap iterasi, untuk plot.
	`moves`	Daftar jenis langkah (uphill atau sideways) dan gerakan yang diambil pada setiap iterasi.
	`iteration`	Jumlah iterasi hingga pencarian berhenti.
	`uphill_count`	Jumlah uphill move yang dilakukan.
	`sideways_count`	Jumlah sideways move yang dilakukan.
	`stop_reason`	Alasan pencarian berhenti, yaitu local_optimum, max_sideways, plateau_exhausted, no_neighbor, atau max_iterations.
	`duration`	Total durasi proses pencarian.
Method	`hill_climbing_sideways`	Menjalankan Hill-Climbing with Sideways Move dan mengembalikan state awal, state akhir, riwayat nilai objective, jumlah iterasi, jumlah sideways move, alasan berhenti, serta durasi.
	`layout_key`	Membuat ringkasan susunan geladak berupa himpunan kendaraan yang diangkut dan himpunan area yang terisi, sehingga dua state yang hanya berbeda karena pertukaran kendaraan berukuran sama dianggap sama.
	`all_neighbors`	Membangkitkan seluruh tetangga dari sebuah state (dari neighbor.py).
	`calculate_objective`	Menghitung nilai objective sebuah state (dari objective.py).

	`generate_random_state`	Membangkitkan state awal acak (dari util.py).
	`copy_state`	Membuat salinan state (dari neighbor.py).
	`get_size`	Menentukan lebar dan panjang kendaraan sesuai orientasinya (dari objective.py).

Algoritma

```
import random
import time

from objective import calculate_objective, get_size
from neighbor import all_neighbors, copy_state
from util import generate_random_state

TIE_TOLERANCE = 1e-9


def layout_key(state, vehicles):
    loaded = frozenset(vid for vid, s in state.items() if
s["in_ship"])
    rects = frozenset(
        (s["x"], s["y"]) + get_size(vehicles[vid],
s["Orientation"])
        for vid, s in state.items() if s["in_ship"]
    )
    return loaded, rects


def hill_climbing_sideways(data, initial_state=None, seed=None,
max_sideways=100, max_iterations=10000):
    rng = random.Random(seed)
    vehicles = {v["id"]: v for v in data["Vehicles"]}
    if initial_state is None:
        initial_state = generate_random_state(data, seed=seed)

    current = copy_state(initial_state)
    current_value = calculate_objective(current, data)

    history = [current_value]
    moves = []
    iteration = 0
    sideways_count = 0
    uphill_count = 0
    stop_reason = "max_iterations"
    plateau_visited = {layout_key(current, vehicles)}
```

```
    start = time.perf_counter()
    while iteration < max_iterations:
        neighbors = all_neighbors(current, data)
        if not neighbors:
            stop_reason = "no_neighbor"
            break

        scored = [(calculate_objective(state, data), state, move)
for state, move in neighbors]
        best_value = max(value for value, _, _ in scored)
        best_candidates = [(s, m) for v, s, m in scored if abs(v -
best_value) <= TIE_TOLERANCE]

        if best_value > current_value + TIE_TOLERANCE:
            best_state, best_move = rng.choice(best_candidates)
            uphill_count += 1
            move_kind = "uphill"
            plateau_visited = {layout_key(best_state, vehicles)}
        elif abs(best_value - current_value) <= TIE_TOLERANCE:
            if sideways_count >= max_sideways:
                stop_reason = "max_sideways"
                break
            fresh = [(s, m) for s, m in best_candidates if
layout_key(s, vehicles) not in plateau_visited]
            if not fresh:
                stop_reason = "plateau_exhausted"
                break
            best_state, best_move = rng.choice(fresh)
            sideways_count += 1
            move_kind = "sideways"
            plateau_visited.add(layout_key(best_state, vehicles))
        else:
            stop_reason = "local_optimum"
            break

        current, current_value = best_state, best_value
        iteration += 1
        history.append(current_value)
        moves.append((move_kind, best_move))
    duration = time.perf_counter() - start

    return {
        "initial_state": initial_state,
        "final_state": current,
        "initial_detail": calculate_objective(initial_state, data,
detail=True),
        "final_detail": calculate_objective(current, data,
detail=True),
        "final_objective": current_value,
```

```
        "history": history,
        "moves": moves,
        "iterations": iteration,
        "uphill_moves": uphill_count,
        "sideways_moves": sideways_count,
        "max_sideways": max_sideways,
        "stop_reason": stop_reason,
        "duration": duration,
        "seed": seed,
    }
```

2.6.4 *Random Restart Hill-Climbing*

Random Restart Hill-Climbing mengatasi kelemahan *hill-climbing* yang mudah terjebak di local optimum dengan cara menjalankan pencarian berulang kali dari titik awal yang berbeda. Setiap kali pencarian berhenti, algoritma memulai pencarian baru dari state awal acak yang lain. Karena setiap titik awal cenderung berakhir di local optimum yang berbeda, peluang menemukan solusi yang lebih baik meningkat seiring bertambahnya jumlah restart. Hasil akhir yang dikembalikan adalah state terbaik dari seluruh pencarian.

Setiap pencarian di dalam algoritma ini menggunakan *Steepest Ascent Hill-Climbing.* Pada setiap iterasi, seluruh tetangga dibangkitkan dengan `all_neighbors`, kemudian algoritma berpindah ke tetangga dengan nilai tertinggi. Pencarian berhenti ketika tidak ada tetangga yang lebih baik daripada *state* saat ini. Pencarian pertama dimulai dari state awal hasil *seed*, sama seperti algoritma lain, sehingga hasilnya dapat dibandingkan secara adil. Setelah pencarian tersebut berhenti, algoritma melakukan *restart* dengan membangkitkan state awal acak yang baru, lalu menjalankan *Steepest Ascent* kembali. Proses ini diulang hingga banyak *restart* mencapai `max_restart`, sehingga total pencarian yang dilakukan adalah `max_restart` ditambah satu. Selama proses berlangsung, algoritma menyimpan state terbaik, jumlah iterasi pada setiap pencarian, serta nilai *objective* pada setiap iterasi dari seluruh pencarian untuk keperluan plot.

Implementasi algoritma *Random Restart Hill-Climbing* terdapat dalam *file* `hc_random_restart.py`.

Tabel 9. Implementasi *Random Restart Hill-Climbing*

Kategori	Nama	Deskripsi
Parameter	`data`	Data input berisi ukuran geladak, kapasitas kapal, dan daftar kendaraan.
	`initial_state`	State awal pencarian pertama. Jika tidak diberikan, state awal dibangkitkan dari seed.
	`seed`	Bilangan pengatur pembangkit acak untuk state awal, state awal setiap restart, dan pemilihan di antara tetangga terbaik yang bernilai sama.
	`max_restart`	Batas maksimum banyak restart. Pencarian dihentikan ketika batas ini tercapai.
	`max_iterations`	Batas pengaman jumlah iterasi pada setiap pencarian.
Konstanta	`TIE_TOLERANCE`	Batas selisih nilai objective agar dua nilai dianggap sama (10^{-9}).
Variabel	`rng`	Pembangkit bilangan acak yang dibuat dari seed.
	`start_state`	State awal dari pencarian yang sedang berjalan.
	`best_state`, `best_value`	State terbaik dari seluruh pencarian beserta nilai objective-nya.
	`final_state`, `final_value`	State akhir dan nilai objective dari satu kali pencarian.
	`run_history`	Daftar nilai objective pada setiap iterasi dalam satu kali pencarian.
	`run_iterations`	Jumlah iterasi dalam satu kali pencarian.
	`runs`	Daftar rincian setiap pencarian, berisi state awal, state akhir, nilai objective, fee, jumlah iterasi, dan durasi.

	`history`	Gabungan nilai objective pada setiap iterasi dari seluruh pencarian, untuk plot.
	`best_history`	Nilai objective terbaik yang sudah ditemukan sampai setiap iterasi.
	`running_best`	Nilai terbaik sementara saat menyusun `best_history`.
	`restart_count`	Banyak restart yang sudah dilakukan.
	`duration`	Total durasi proses pencarian.
Variabel (steepest_ascent)	`current, current_value`	State yang sedang ditempati beserta nilai objective-nya.
	`neighbors`	Daftar seluruh tetangga dari `current`.
	`scored`	Daftar tetangga beserta nilai objective masing-masing.
	`best_value`	Nilai objective tertinggi di antara seluruh tetangga.
	`best_candidates`	Daftar tetangga yang nilainya sama dengan `best_value`.
	`iteration`	Jumlah iterasi hingga pencarian berhenti.
Method	`random_restart_hill_climbing`	Menjalankan Random Restart Hill-Climbing dan mengembalikan state terbaik, banyak restart, iterasi per restart, riwayat nilai objective, serta durasi.
	`steepest_ascent`	Menjalankan satu kali *Steepest Ascent Hill-Climbing* hingga mencapai *local optimum.*
	`all_neighbors`	Membangkitkan seluruh tetangga dari sebuah state (dari `neighbor.py`).
	`calculate_objective`	Menghitung nilai objective sebuah state (dari `objective.py`).
	`generate_random_state`	Membangkitkan state awal acak, baik untuk pencarian pertama maupun setiap restart (dari `util.py`).

	`copy_state`	Membuat salinan state (dari `neighbor.py`).

Algoritma

```
import random
import time

from objective import calculate_objective
from neighbor import all_neighbors, copy_state
from util import generate_random_state

TIE_TOLERANCE = 1e-9


def steepest_ascent(initial_state, data, rng, max_iterations=10000):
    current = copy_state(initial_state)
    current_value = calculate_objective(current, data)
    history = [current_value]
    iteration = 0

    while iteration < max_iterations:
        neighbors = all_neighbors(current, data)
        if not neighbors:
            break
        scored = [(calculate_objective(state, data), state) for
state, _ in neighbors]
        best_value = max(value for value, _ in scored)
        if best_value <= current_value + TIE_TOLERANCE:
            break
        best_candidates = [state for value, state in scored if
abs(value - best_value) <= TIE_TOLERANCE]
        current, current_value = rng.choice(best_candidates),
best_value
        iteration += 1
        history.append(current_value)

    return current, current_value, history, iteration


def random_restart_hill_climbing(data, initial_state=None, seed=None,
max_restart=10, max_iterations=10000):
    rng = random.Random(seed)
    if initial_state is None:
        initial_state = generate_random_state(data, seed=seed)

    best_state, best_value = None, float("-inf")
    runs = []
    history = []
```

```
    best_history = []
    restart_count = 0
    start_state = initial_state

    start = time.perf_counter()
    while True:
        run_start = time.perf_counter()
        final_state, final_value, run_history, run_iterations =
steepest_ascent(start_state, data, rng, max_iterations)

        running_best = best_value
        for value in run_history:
            running_best = max(running_best, value)
            history.append(value)
            best_history.append(running_best)

        if final_value > best_value:
            best_state, best_value = final_state, final_value

        runs.append({
            "restart": restart_count,
            "initial_state": start_state,
            "final_state": final_state,
            "final_objective": final_value,
            "final_fee": calculate_objective(final_state, data,
detail=True)["total_fee"],
            "iterations": run_iterations,
            "history": run_history,
            "duration": time.perf_counter() - run_start,
        })

        if restart_count >= max_restart:
            break
        restart_count += 1
        start_state = generate_random_state(data, seed=rng.randint(0,
10**9))
    duration = time.perf_counter() - start

    return {
        "initial_state": initial_state,
        "final_state": best_state,
        "initial_detail": calculate_objective(initial_state, data,
detail=True),
        "final_detail": calculate_objective(best_state, data,
detail=True),
        "final_objective": best_value,
        "history": history,
        "best_history": best_history,
        "runs": runs,
```

```
        "restarts": restart_count,
        "max_restart": max_restart,
        "iterations_per_restart": [r["iterations"] for r in runs],
        "total_iterations": sum(r["iterations"] for r in runs),
        "best_restart": max(runs, key=lambda r:
r["final_objective"])["restart"],
        "duration": duration,
        "seed": seed,
    }
```

2.6.5 *Simulated Annealing*

Simulated annealing adalah algoritma local search yang berbeda dengan hill-climbing, boleh berpindah ke tetangga yang lebih buruk dengan peluang tertentu. Peluang ini ditentukan oleh suhu (T), yaitu parameter yang diturunkan secara bertahap selama pencarian. Saat suhu masih tinggi, tetangga yang lebih buruk lebih mudah diterima, sehingga algoritma bisa keluar dari local optimum dan menjelajahi daerah lain. Seiring suhu turun, algoritma makin jarang menerima langkah yang memburuk, hingga pada akhirnya hampir hanya menerima perbaikan.

Pencarian dimulai dari state awal acak hasil seed, dengan suhu awal sebesar `initial_temp`. Pada setiap iterasi, fungsi `random_neighbor` menghasilkan satu tetangga acak, lalu dihitung selisih nilainya terhadap state saat ini, yaitu ΔE = nilai tetangga − nilai state saat ini. Karena permasalahan ini adalah maksimasi, tetangga yang lebih baik ($\Delta E > 0$) selalu diterima, sedangkan tetangga yang lebih buruk ($\Delta E \leq 0$) diterima dengan peluang $e^{\frac{\Delta E}{T}}$, dengan T adalah suhu saat ini. Semakin buruk tetangga dan semakin rendah suhu, semakin kecil peluangnya diterima. Setelah keputusan diambil, suhu diturunkan dengan dikalikan `cooling_rate`. Algoritma juga menyimpan state terbaik yang pernah ditemui, karena state yang sedang ditempati bisa saja lebih buruk dari yang pernah dicapai sebelumnya. Pencarian berhenti ketika suhu turun di bawah `min_temp` atau jumlah iterasi mencapai `max_iterations`. Setelah itu, state saat ini dikembalikan ke state terbaik, sehingga state akhir dan titik terakhir kurva menunjukkan

nilai objective yang sama. Dengan parameter default, suhu mencapai `min_temp` setelah sekitar 8.500 iterasi, sehingga pencarian berhenti sebelum `max_iterations`.

Untuk mengukur frekuensi stuck di local optima, kelompok kami mendefinisikan satu kejadian stuck sebagai keadaan ketika tetangga ditolak sebanyak 50 kali berturut-turut. Satu rangkaian penolakan hanya dihitung sebagai satu kejadian, berapa pun panjangnya. Frekuensi stuck kemudian dihitung sebagai jumlah kejadian stuck dibagi jumlah iterasi.

Implementasi algoritma *Simulated Annealing* terdapat dalam *file* *`simulated_annealing`*`.py`.

Tabel 10. Implementasi *Simulated Annealing*

Kategori	Nama	Deskripsi
Parameter	`data`	Data input berisi ukuran geladak, kapasitas kapal, dan daftar kendaraan.
	`initial_temp`	Suhu awal pencarian (default 50)
	`cooling_rate`	Faktor pengali suhu setelah setiap iterasi (default 0,999).
	`max_iterations`	Batas maksimum iterasi pencarian (default 10.000). Pencarian dihentikan ketika batas ini tercapai.
	`min_temp`	Suhu minimum. Pencarian berhenti jika suhu turun di bawah nilai ini (default 0,01)
	`seed`	Bilangan pengatur pembangkit acak untuk state awal, pemilihan tetangga, dan keputusan penerimaan.
Konstanta	`STUCK_TRESHOLD`	Banyak penolakan berturut-turut yang dihitung sebagai satu kejadian stuck (50).
Variabel	`rng`	Pembangkit bilangan acak yang dibuat dari seed.
	`initial_state`	Salinan state awal untuk keperluan visualisasi.

	`current_state, current_score`	State yang sedang ditempati beserta nilai objective-nya.
	`best_state, best_score`	State terbaik yang pernah ditemui beserta nilai objective-nya.
	`temp`	Suhu saat ini.
	`neighbor_state, neighbor_score`	Tetangga acak pada iterasi saat ini beserta nilai objective-nya.
	`delta`	Selisih nilai objective tetangga terhadap state saat ini (ΔE)
	`prob`	Peluang penerimaan tetangga, yaitu 1 jika $\Delta E > 0$ dan $e^{\frac{\Delta E}{T}}$ jika tidak.
	`history`	Daftar nilai objective state saat ini pada setiap iterasi, ditambah satu titik terakhir berupa nilai state terbaik, untuk plot.
	`prob_history`	Daftar nilai prob pada setiap iterasi, untuk plot $e^{\frac{\Delta E}{T}}$.
	`temp_history`	Daftar suhu pada setiap iterasi.
	`rejected_streak`	Banyak penolakan berturut-turut sejak penerimaan terakhir.
	`stuck_count`	Total tetangga yang ditolak selama pencarian.
	`stuck_events`	Banyak kejadian stuck selama pencarian.
	`iterations`	Jumlah iterasi yang sudah dilakukan.
	`Start, duration`	Waktu mulai dan total durasi proses pencarian.
	`final_detail`	RIncian perhitungan objective dari state akhir (state terbaik).
Method	`simulated_annealing`	Menjalankan simulated annealing dan mengembalikan state awal, state terbaik, riwayat nilai objective, riwayat peluang penerimaan, jumlah kejadian stuck, serta durasi.

	`random_neighbor`	Berusaha membangkitkan satu tetangga acak yang berbeda dari state asal, dengan maksimal 20 percobaan (dari `neighbor.py`)
	`math.exp`	Menghitung nilai $e^{\frac{\Delta E}{T}}$.
	`calculate_objective`	Menghitung nilai objective sebuah state (dari `objective.py`).
	`generate_random_state`	Membangkitkan state awal acak (dari `util.py`).
	`copy_state`	Membuat salinan state (dari `neighbor.py`).

Algoritma

```
import math
import random
import time

from objective import calculate_objective
from neighbor import copy_state, random_neighbor
from util import generate_random_state

STUCK_TRESHOLD = 50

def simulated_annealing(data, initial_temp=50, cooling_rate=0.999,
max_iterations=10000, min_temp=0.01, seed=None):
    rng = random.Random(seed)
    current_state = generate_random_state(data, seed)
    initial_state = copy_state(current_state)
    current_score = calculate_objective(current_state, data)
    best_state, best_score = copy_state(current_state), current_score

    temp = initial_temp
    history = [current_score]
    prob_history = []
    temp_history = []
    rejected_streak = 0
    stuck_count = 0
    stuck_events = 0
    iterations = 0

    start = time.perf_counter()
    while temp > min_temp and iterations < max_iterations:
        neighbor_state, _ = random_neighbor(current_state, data, rng)
```

```
        neighbor_score = calculate_objective(neighbor_state, data)
        delta = neighbor_score - current_score

        prob = 1.0 if delta > 0 else math.exp(delta / temp)
        prob_history.append(prob)
        temp_history.append(temp)

        if delta > 0 or rng.random() < prob:
            current_state, current_score = neighbor_state,
neighbor_score
            rejected_streak = 0
            if current_score > best_score:
                best_state, best_score = copy_state(current_state),
current_score
        else:
            rejected_streak += 1
            stuck_count += 1
            if rejected_streak == STUCK_TRESHOLD:
                stuck_events += 1

        history.append(current_score)
        temp *= cooling_rate
        iterations += 1
    current_state, current_score = best_state, best_score
    history.append(current_score)
    duration = time.perf_counter() - start

    final_detail = calculate_objective(current_state, data,
detail=True)
    return {
        "algorithm": "Simulated Annealing",
        "seed": seed,
        "initial_temp": initial_temp,
        "cooling_rate": cooling_rate,
        "initial_state": initial_state,
        "final_state": current_state,
        "initial_objective": history[0],
        "final_objective": current_score,
        "final_total_fee": final_detail["total_fee"],
        "final_valid": final_detail["valid"],
        "objective_history": history,
        "prob_history": prob_history,
        "temp_history": temp_history,
        "iterations": iterations,
        "stuck_count": stuck_count,
        "stuck_events": stuck_events,
        "duration_seconds": duration,
    }
```

2.6.6 *Genetic Algorithm*

Genetic algorithm adalah metode pencarian berbasis populasi. Berbeda dengan algoritma sebelumnya yang hanya mengelola satu state, algoritma ini mengelola sekumpulan state sekaligus yang disebut populasi, dan setiap state di dalamnya disebut individu. Pada setiap generasi, individu yang baik dipertahankan dan dikombinasikan untuk menghasilkan individu baru, sehingga sifat-sifat dari state yang baik dapat diwariskan dan digabungkan. Nilai objective digunakan langsung sebagai ukuran kualitas (fitness) setiap individu.

Pencarian diawali dengan membentuk populasi awal sebanyak `population_size` individu. Individu pertama adalah state awal hasil seed, sedangkan sisanya dibangkitkan secara acak. Karena itu, state awal yang ditampilkan pada hasil eksperimen adalah individu terbaik pada populasi awal, yang tidak selalu sama dengan state awal pada algoritma lain. Setiap individu kemudian dihitung nilai objective-nya dan populasi diurutkan dari nilai tertinggi. Pada setiap generasi, algoritma menjalankan tiga tahap berikut:

1. **Seleksi**: setengah populasi dengan nilai tertinggi dipertahankan ke generasi berikutnya, sedangkan setengah sisanya dibuang. Cara ini dipilih karena nilai objective dapat bernilai negatif akibat penalti, sehingga metode seperti roulette wheel yang membutuhkan nilai positif tidak dapat digunakan secara langsung. Karena individu terbaik selalu dipertahankan, nilai maksimum populasi tidak pernah turun dari satu generasi ke generasi berikutnya.
2. **Crossover**: dua induk dipilih secara acak dari individu yang dipertahankan. Anak dibentuk dengan cara setiap kendaraan mewarisi status penempatannya (di dalam atau di luar kapal, koordinat, dan orientasi) dari salah satu induk, dengan peluang masing-masing 50%. Karena status tiap kendaraan diwariskan secara terpisah, anak dapat melanggar batasan (kendaraan bertumpuk atau berat melebihi kapasitas). Pelanggaran ini tidak diperbaiki, tetapi dihukum oleh penalti pada

objective function sehingga anak tersebut cenderung tersingkir pada seleksi berikutnya.

3. **Mutasi**: dengan peluang sebesar `mutation_rate`, anak dikenai satu gerakan tetangga acak melalui `random_neighbor`. Mutasi menjaga keragaman populasi agar tidak semua individu menjadi serupa.

Proses crossover dan mutasi diulang hingga jumlah individu kembali mencapai `population_size`. Satu generasi dihitung sebagai satu iterasi, dan algoritma mencatat nilai *objective* maksimum dan rata-rata populasi, baik pada populasi awal (generasi 0) maupun pada setiap generasi. Setelah iterations generasi, individu dengan nilai tertinggi dikembalikan sebagai state akhir.

Implementasi algoritma *Genetic Algorithm* terdapat dalam *file* *`genetic_algorithm`*`.py`.

Tabel 11. Implementasi *Genetic Algorithm*

Kategori	Nama	Deskripsi
Parameter	`data`	Data input berisi ukuran geladak, kapasitas kapal, dan daftar kendaraan.
	`population_size`	Jumlah individu dalam populasi (default 50).
	`iterations`	Banyak generasi yang dijalankan (default 200).
	`seed`	Bilangan pengatur pembangkit acak untuk populasi awal, pemilihan induk, crossover, dan mutasi.
	`mutation_rate`	Peluang seorang anak dikenai mutasi (default 0,5).
Variabel	`rng`	Pembangkit bilangan acak yang dibuat dari seed.
	`states`	Daftar state pada populasi awal.
	`population`	Daftar pasangan nilai objective dan individu, diurutkan dari nilai tertinggi.

	`survivors`	Setengah populasi dengan nilai tertinggi yang dipertahankan ke generasi berikutnya.
	`new_population`	Populasi generasi berikutnya, berisi survivors dan anak-anak baru.
	`parent_a, parent_b`	Dua induk yang dipilih secara acak dari survivors.
	`child`	Individu baru hasil crossover dan mutasi.
	`generation`	Nomor generasi yang sedang berjalan.
	`history`	Dictionary hasil pencarian, berisi state awal, state akhir, parameter, riwayat nilai objective, serta durasi.
	`iteration_list`	Daftar nomor generasi (0 sampai iterations), untuk sumbu x plot.
	`max_objectives`	Nilai objective maksimum populasi pada setiap generasi, untuk plot.
	`avg_objectives`	Nilai objective rata-rata populasi pada setiap generasi, untuk plot.
	`best_state, best_score`	Individu terbaik pada generasi terakhir beserta nilai objective-nya.
	`start_time`	Waktu mulai proses pencarian.
Method	`genetic_algorithm`	Menjalankan genetic algorithm dan mengembalikan state akhir beserta riwayat nilai maksimum dan rata-rata populasi, jumlah populasi, banyak iterasi, serta durasi.
	`crossover`	Membentuk satu anak dari dua induk dengan mewariskan penempatan setiap kendaraan dari salah satu induk secara acak.
	`mutate`	Menerapkan satu gerakan tetangga acak pada anak dengan peluang `mutation_rate`.
	`generate_random_stat`	Membangkitkan individu acak untuk

	`e`	populasi awal (dari util.py)
	`random_neighbor`	Berusaha membangkitkan satu tetangga acak, dipakai sebagai mutasi (dari neighbor.py)
	`calculate_objective`	Menghitung nilai objective sebuah individu (dari objective.py).
	`copy_state`	Membuat salinan state (dari `neighbor.py`).

Algoritma

```
import random
import time

from objective import calculate_objective
from neighbor import copy_state, random_neighbor
from util import generate_random_state


def crossover(parent_a, parent_b, rng):
    child = {}
    for vid in parent_a:
        source = parent_a if rng.random() < 0.5 else parent_b
        child[vid] = dict(source[vid])
    return child

def mutate(state, data, mutation_rate, rng):
    if rng.random() < mutation_rate:
        state, _ = random_neighbor(state, data, rng)
    return state

def genetic_algorithm(data, population_size=50, iterations=200,
seed=None, mutation_rate=0.5):
    rng = random.Random(seed)
    start_time = time.perf_counter()
    states = [generate_random_state(data, seed=seed)]
    while len(states) < population_size:
        states.append(generate_random_state(data,
seed=rng.randrange(10**9)))
    population = [(calculate_objective(s, data), s) for s in states]
    population.sort(key=lambda p: p[0], reverse=True)

    history = {
        "algorithm": "Genetic Algorithm",
        "population_size": population_size,
        "iterations": iterations,
```

```
        "seed": seed,
        "initial_state": copy_state(population[0][1]),
        "initial_objective": population[0][0],
        "iteration_list": [0],
        "max_objectives": [population[0][0]],
        "avg_objectives": [sum(p[0] for p in population) /
population_size],
    }

    for generation in range(1, iterations + 1):
        survivors = population[:population_size // 2]
        new_population = list(survivors)

        while len(new_population) < population_size:
            parent_a = rng.choice(survivors)[1]
            parent_b = rng.choice(survivors)[1]
            child = mutate(crossover(parent_a, parent_b, rng), data,
mutation_rate, rng)
            new_population.append((calculate_objective(child, data),
child))

        population = sorted(new_population, key=lambda p: p[0],
reverse=True)
        history["iteration_list"].append(generation)
        history["max_objectives"].append(population[0][0])
        history["avg_objectives"].append(sum(p[0] for p in
population) / population_size)

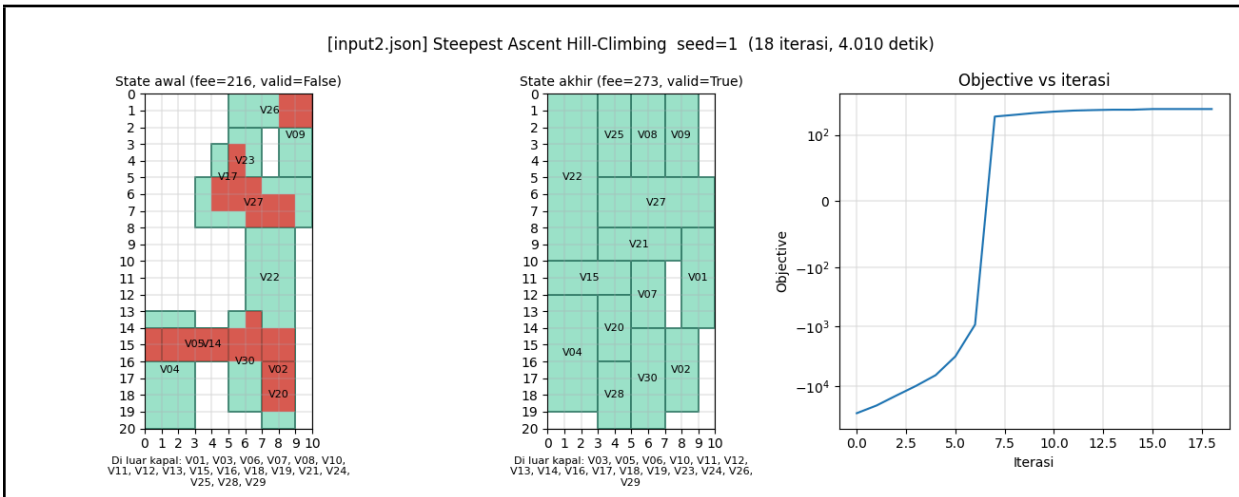
    best_score, best_state = population[0]
    history["final_state"] = copy_state(best_state)
    history["final_objective"] = best_score
    history["final_detail"] = calculate_objective(best_state, data,
detail=True)
    history["final_total_fee"] = history["final_detail"]["total_fee"]
    history["final_valid"] = history["final_detail"]["valid"]
    history["objective_history"] = history["max_objectives"]
    history["duration_seconds"] = time.perf_counter() - start_time
    return history["final_state"], history
```

2.7 Hasil Eksperimen

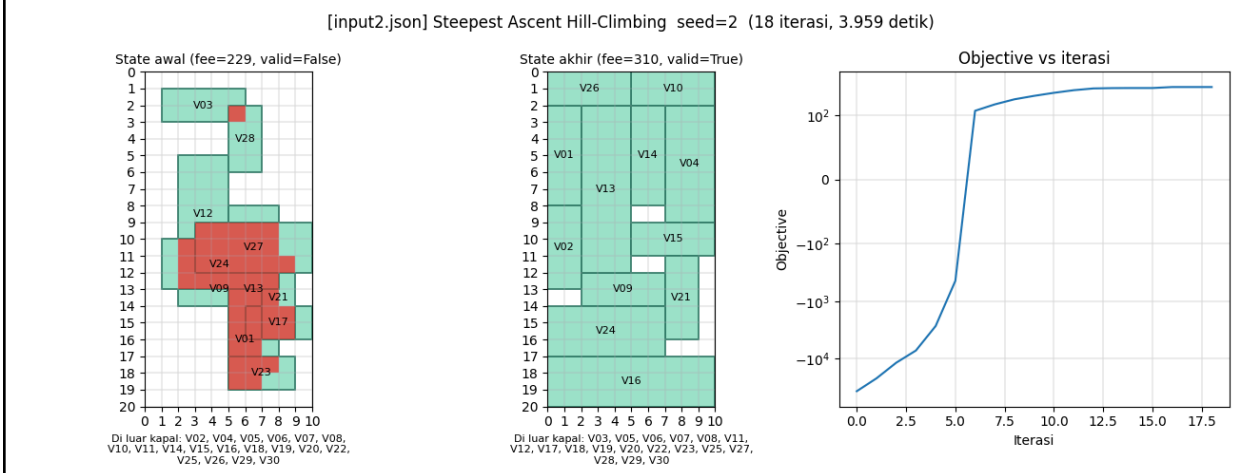
2.7.1 Hasil Eksperimen *Steepest Ascent Hill Climbing*

Tabel 12. Hasil Eksperimen *Steepest Ascent Hill Climbing*

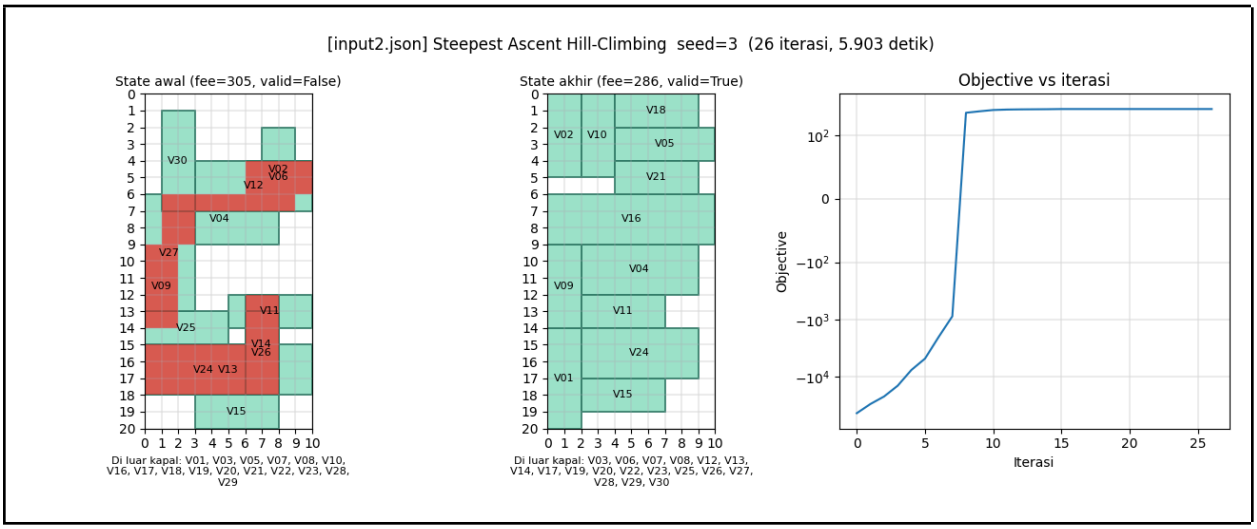
Percobaan 1		
Durasi	4.010s	
Maksimum Iterasi	1000	
Iterasi hingga berhenti	18	
Tetangga dievaluasi	71709	
Alasan Berhenti	Local optimum	
State Awal	**Fee**	216 juta
	Berat (Max. 500)	370
	Objective	-28598.9992
	Status	Tidak valid (51 sel bertumpuk)
State Akhir	**Fee**	273 juta
	Berat (Max. 500)	412
	Objective	273.0478
	Status	Valid



Percobaan 2		
Durasi	3.959s	
Maksimum Iterasi	1000	
Iterasi hingga berhenti	18	
Tetangga dievaluasi	70930	
Alasan Berhenti	Local optimum	
State Awal	**Fee**	229 juta
	Berat (Max. 500)	388
	Objective	-36495.9915
	Status	Tidak valid (65 sel bertumpuk)
State Akhir	**Fee**	310 juta
	Berat (Max. 500)	473
	Objective	310.0544
	Status	Valid



Percobaan 3		
Durasi	5.903s	
Maksimum Iterasi	1000	
Iterasi hingga berhenti	26	
Tetangga dievaluasi	102022	
Alasan Berhenti	Local optimum	
State Awal	**Fee**	305 juta
	Berat (Max. 500)	490
	Objective	-43764.9954
	Status	Tidak valid (78 sel bertumpuk)
State Akhir	**Fee**	286 juta
	Berat (Max. 500)	378
	Objective	286.0493
	Status	Valid

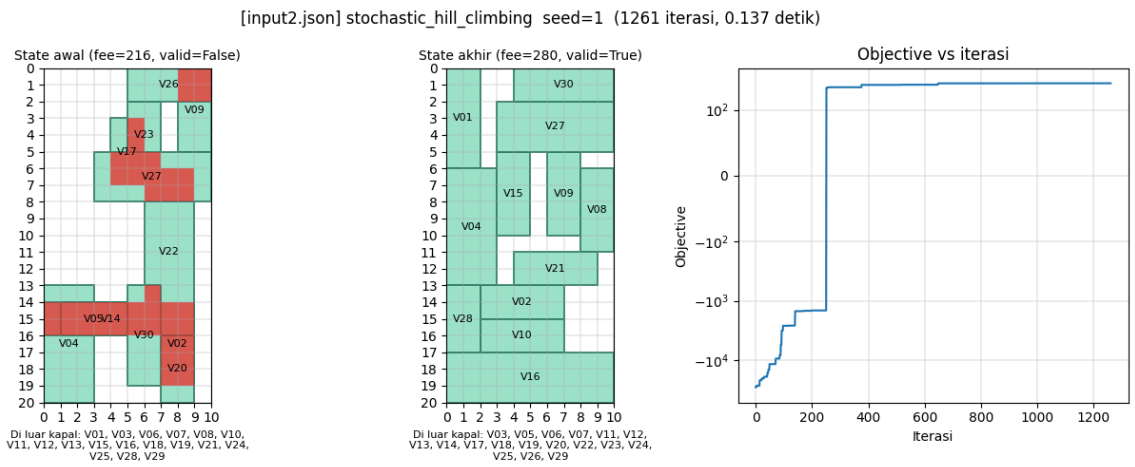


2.7.2 Hasil Eksperimen *Stochastic Hill-Climbing*

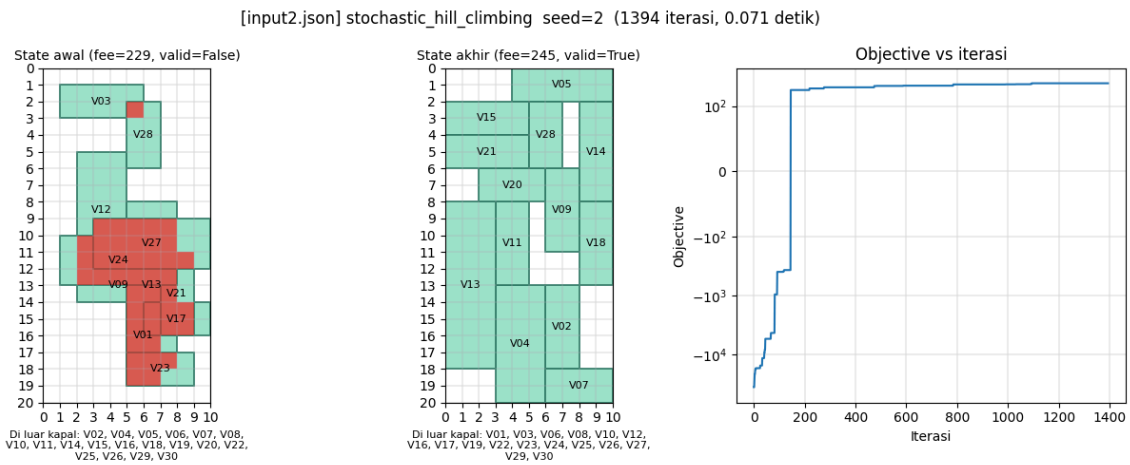
Tabel 13. Hasil Eksperimen Stochastic Hill-Climbing

Percobaan 1		
Durasi	0.137s	
Maksimum Iterasi	3000	
Patience	300	
Iterasi hingga berhenti	1261	
Alasan berhenti	*patience*	
Langkah diterima	43	
State Awal	**Fee**	216 juta
	Berat (Max. 500)	370
	Objective	-28598.9992
	Status	Tidak valid (51 sel bertumpuk)
State Akhir	**Fee**	280 juta
	Berat (Max. 500)	365

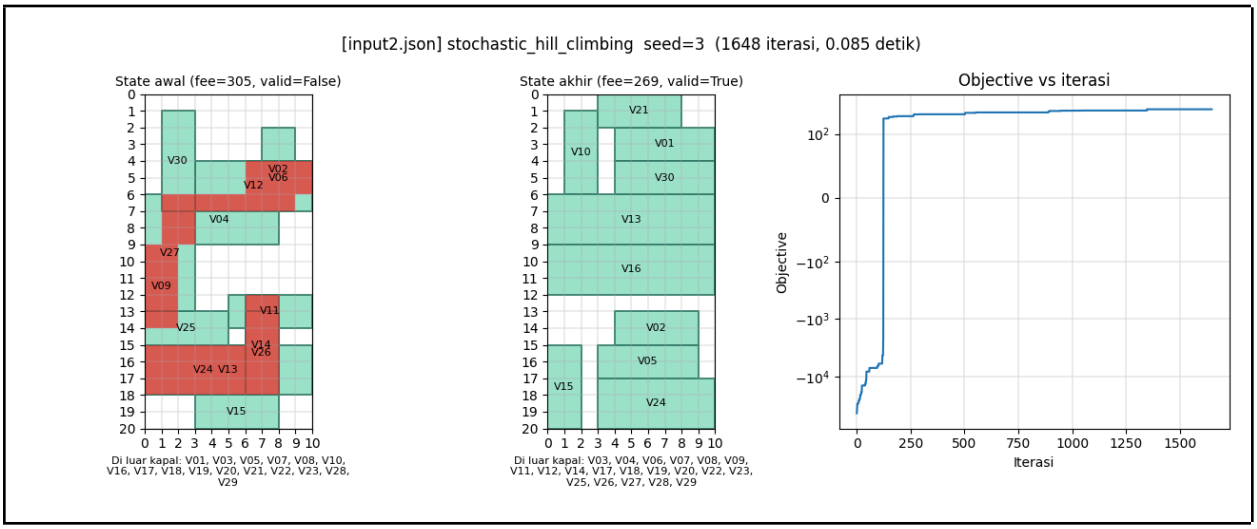
	Objective	280.0394
	Status	Valid



Percobaan 2		
Durasi	0.071s	
Maksimum Iterasi	3000	
Patience	300	
Iterasi hingga berhenti	1394	
Alasan berhenti	*patience*	
Langkah diterima	39	
State Awal	**Fee**	229 juta
	Berat (Max. 500)	388
	Objective	-36495.9915
	Status	Tidak valid (65 sel bertumpuk)
State Akhir	**Fee**	245 juta
	Berat (Max. 500)	347
	Objective	245.0468
	Status	Valid



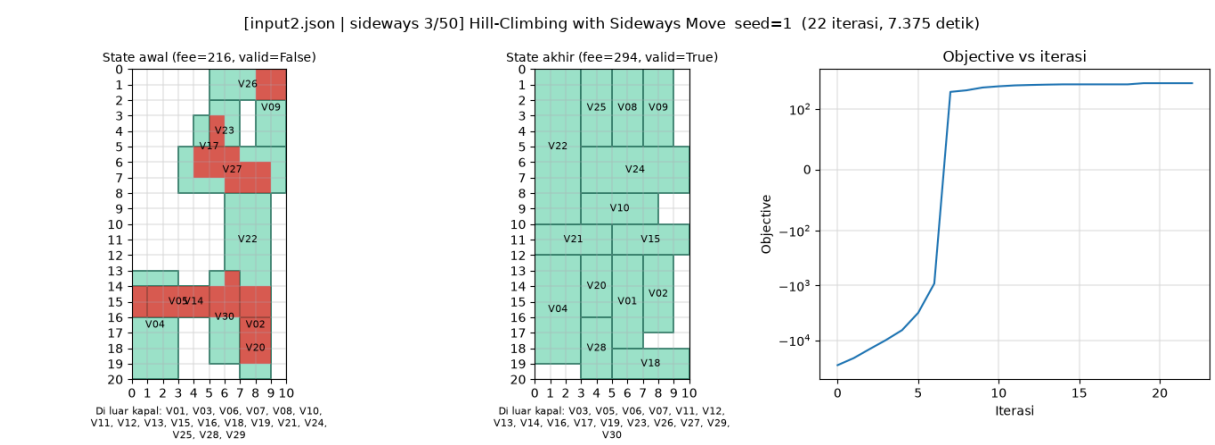
Percobaan 3		
Durasi	0.085s	
Maksimum Iterasi	3000	
Patience	300	
Iterasi hingga berhenti	1648	
Alasan berhenti	*patience*	
Langkah diterima	46	
State Awal	**Fee**	305 juta
	Berat (Max. 500)	490
	Objective	-43764.9954
	Status	Tidak valid (78 sel bertumpuk)
State Akhir	**Fee**	269 juta
	Berat (Max. 500)	418
	Objective	269.0325
	Status	Valid



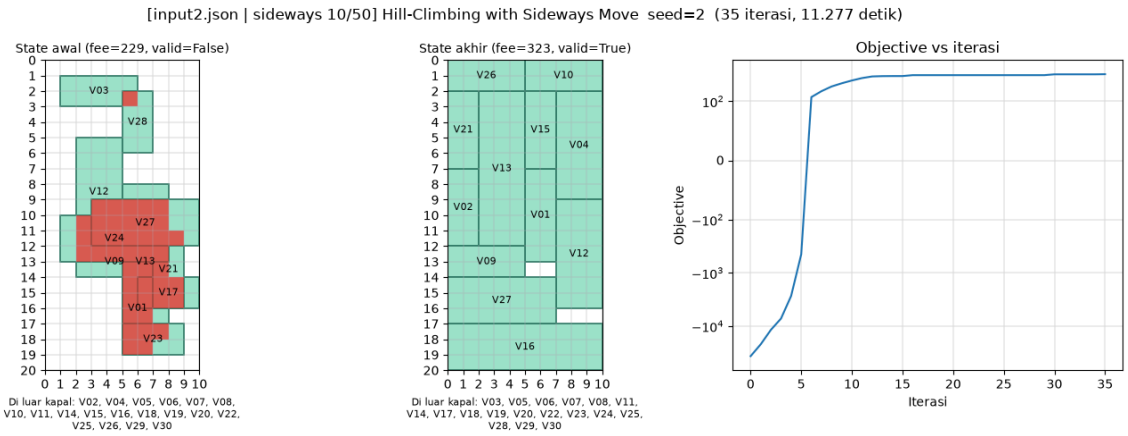
2.7.3 Hasil Eksperimen *Hill Climbing with Sideways Move*

Tabel 14. Hasil Eksperimen *Hill Climbing with Sideways Move*

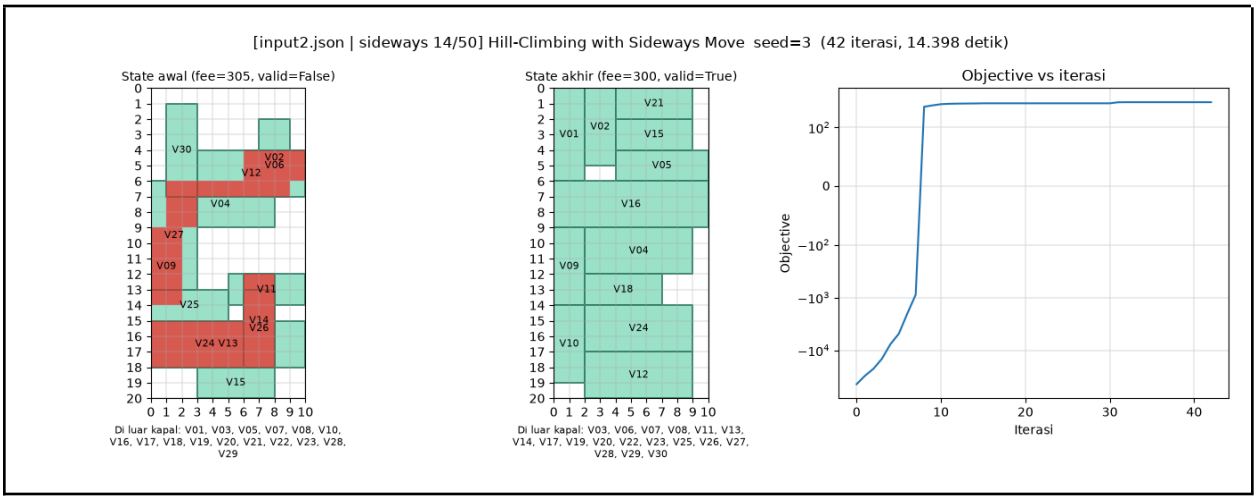
Percobaan 1		
Durasi	7.37 s	
Maksimum Sideways Move	50	
Iterasi hingga berhenti	22 (uphill 19, sideways 3)	
Alasan Berhenti	Plateau habis dijelajahi	
State Awal	**Fee**	216 juta
	Berat (Max. 500)	370
	Objective	-28598.9992
	Status	Tidak valid (51 sel bertumpuk)
State Akhir	**Fee**	294 juta
	Berat (Max. 500)	416
	Objective	294.0383
	Status	Valid



Percobaan 2		
Durasi	11.28s	
Maksimum Sideways Move	50	
Iterasi hingga berhenti	35 (uphill 25, sideways 10)	
Alasan Berhenti	Plateau habis dijelajahi	
State Awal	**Fee**	229 juta
	Berat (Max. 500)	388
	Objective	-36495.9915
	Status	Tidak valid (65 sel bertumpuk)
State Akhir	**Fee**	323 juta
	Berat (Max. 500)	496
	Objective	323.0583
	Status	Valid



Percobaan 3		
Durasi	14.40 s	
Maksimum Sideways Move	50	
Iterasi hingga berhenti	42 (uphill 28, sideways 14)	
Alasan Berhenti	Plateau habis dijelajahi	
State Awal	**Fee**	305 juta
	Berat (Max. 500)	490
	Objective	-43764.9954
	Status	Tidak valid (78 sel bertumpuk)
State Akhir	**Fee**	300 juta
	Berat (Max. 500)	411
	Objective	300.0525
	Status	Valid

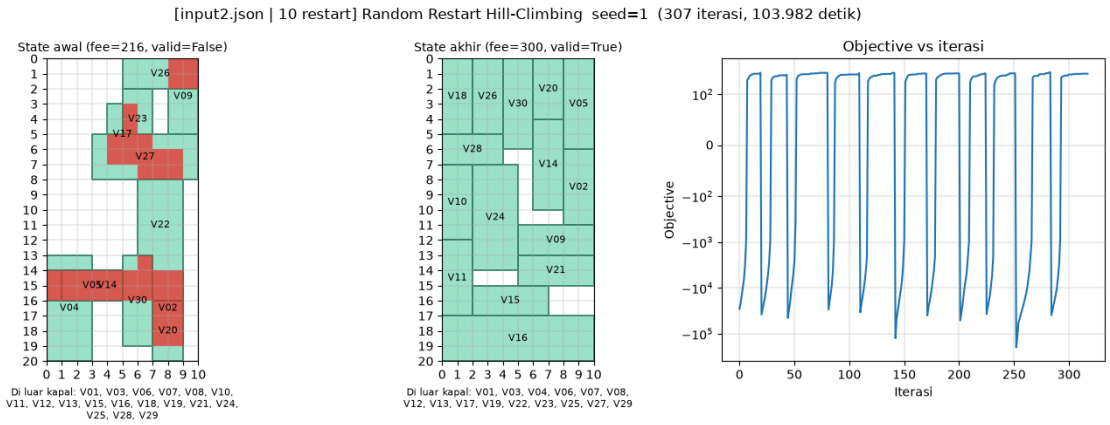


2.7.4 Hasil Eksperimen Random Restart Hill-Climbing

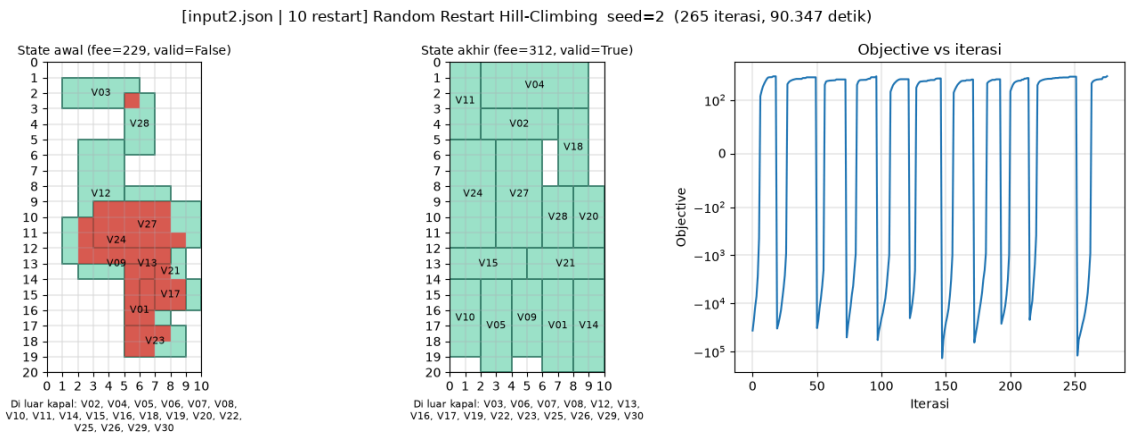
Tabel 15. Hasil Eksperimen *Random Restart Hill-Climbing*

Percobaan 1		
Durasi	103.98 s	
Maksimum Restart	10	
Banyak Restart	10	
Total Iterasi	307	
Iterasi per Restart	19, 23, 36, 28, 31, 28, 29, 23, 26, 31, 33	
Fee Akhir per Restart	294, 265, 293, 289, 292, 283, 285, 284, 285, 300, 280	
Solusi Terbaik dari	Restart ke-9	
State Awal	**Fee**	216 juta
	Berat (Max. 500)	370
	Objective	-28598.9992
	Status	Tidak valid (51 sel bertumpuk)
State Akhir	**Fee**	300 juta
	Berat (Max. 500)	377

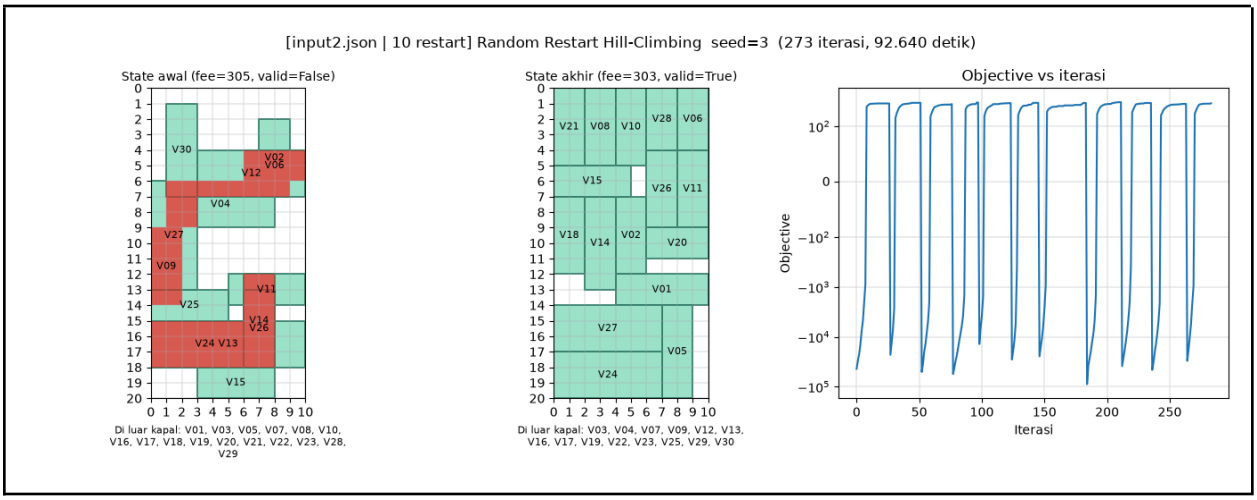
	Objective	300.0293
	Status	Valid



Percobaan 2		
Durasi	90.35s	
Maksimum Restart	10	
Banyak Restart	10	
Total Iterasi	265	
Iterasi per Restart	18, 30, 22, 23, 24, 24, 24, 20, 21, 36, 23	
Fee Akhir per Restart	310, 296, 267, 310, 270, 280, 266, 269, 285, 305, 312	
Solusi Terbaik dari	Restart ke-10	
State Awal	**Fee**	229 juta
	Berat (Max. 500)	388
	Objective	-36495.9915
	Status	Tidak valid (65 sel bertumpuk)
State Akhir	**Fee**	312 juta
	Berat (Max. 500)	389
	Objective	312.0424
	Status	Valid



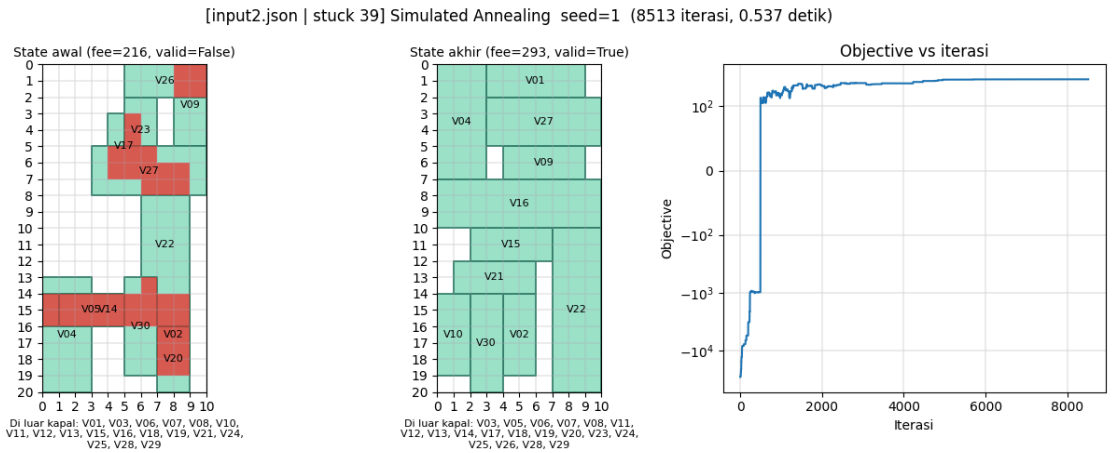
Percobaan 3		
Durasi	92.64s	
Maksimum Restart	10	
Banyak Restart	10	
Total Iterasi	273	
Iterasi per Restart	26, 24, 24, 20, 25, 21, 37, 27, 23, 27, 19	
Fee Akhir per Restart	286, 294, 278, 299, 290, 294, 292, 303, 292, 280, 288	
Solusi Terbaik dari	Restart ke-7	
State Awal	**Fee**	305 juta
	Berat (Max. 500)	490
	Objective	-43764.9954
	Status	Tidak valid (78 sel bertumpuk)
State Akhir	**Fee**	303 juta
	Berat (Max. 500)	368
	Objective	303.0404
	Status	Valid

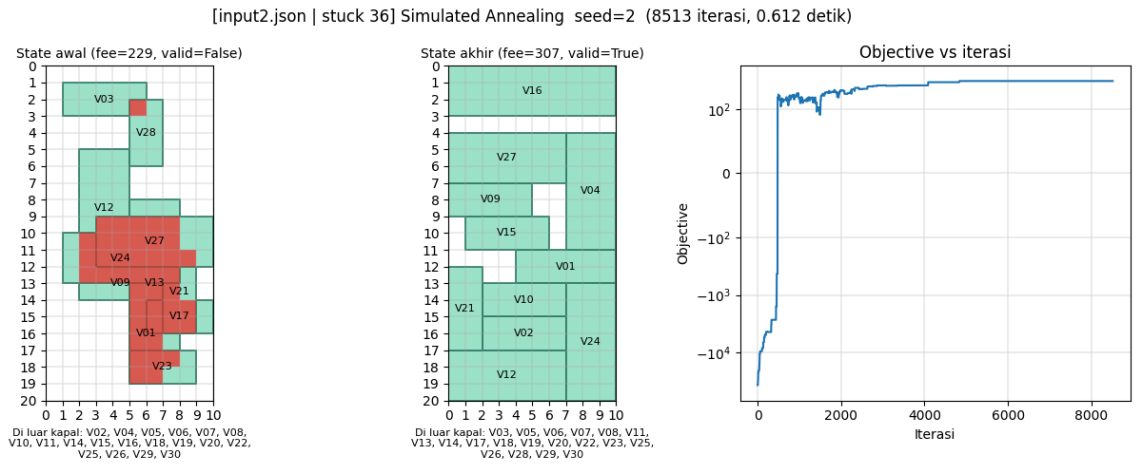


2.7.5 Hasil Eksperimen *Simulated Annealing*

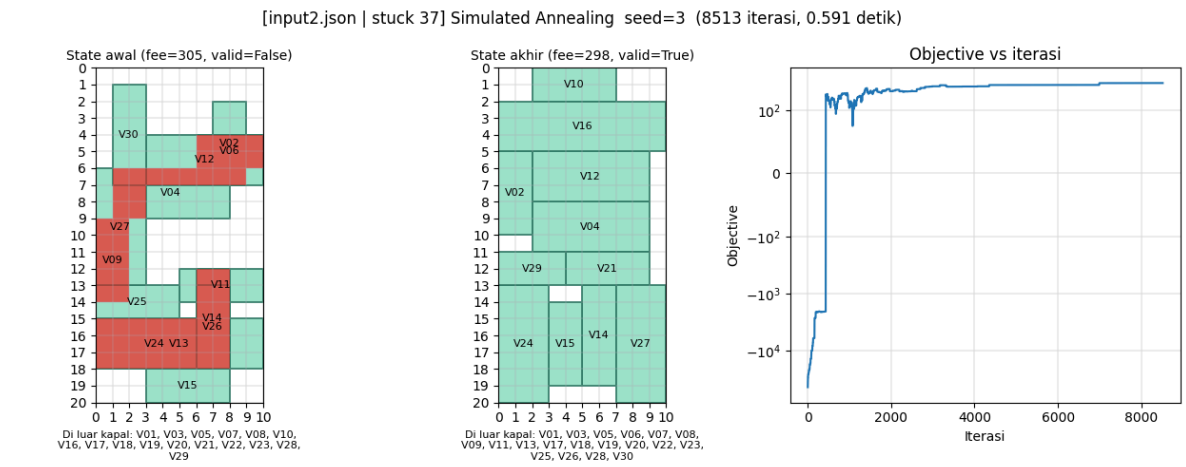
Tabel 16. Hasil Eksperimen *Simulated Annealing*

Percobaan 1		
Durasi	0.537 s	
Iterasi	8.513	
Alasan berhenti	Suhu mencapai min_temp	
Kejadian stuck	39	
Frekuensi stuck	0,0046	
Tetangga ditolak	7.919	
State Awal	**Fee**	216 juta
	Berat (Max. 500)	370
	Objective	-28598.9992
	Status	Tidak valid (51 sel bertumpuk)
State Akhir	**Fee**	293 juta
	Berat (Max. 500)	458
	Objective	293.0408

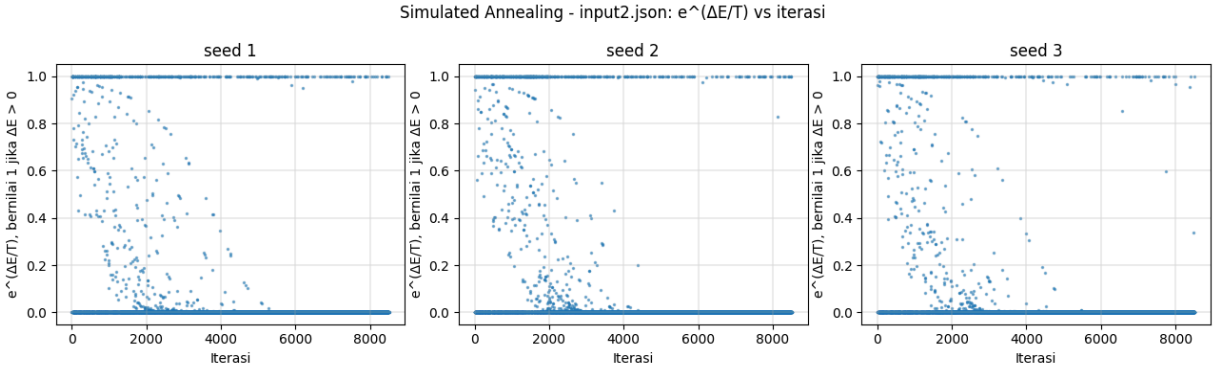
	Status	Valid
		
Percobaan 2		
Durasi	0.612 s	
Iterasi	8.513	
Alasan berhenti	Suhu mencapai min_temp	
Kejadian stuck	36	
Frekuensi stuck	0,0042	
Tetangga ditolak	7.796	
State Awal	**Fee**	229 juta
	Berat (Max. 500)	388
	Objective	-36495.9915
	Status	Tidak valid (65 sel bertumpuk)
State Akhir	**Fee**	307 juta
	Berat (Max. 500)	431
	Objective	307.0414
	Status	Valid



Percobaan 3		
Durasi	0.591 s	
Iterasi	8.513	
Alasan berhenti	Suhu mencapai min_temp	
Kejadian stuck	37	
Frekuensi stuck	0,0043	
Tetangga ditolak	7.903	
State Awal	**Fee**	305 juta
	Berat (Max. 500)	490
	Objective	-43764.9954
	Status	Tidak valid (78 sel bertumpuk)
State Akhir	**Fee**	298 juta
	Berat (Max. 500)	420
	Objective	298.0431
	Status	Valid



Plot $e^{\frac{\Delta E}{T}}$ terhadap Iterasi



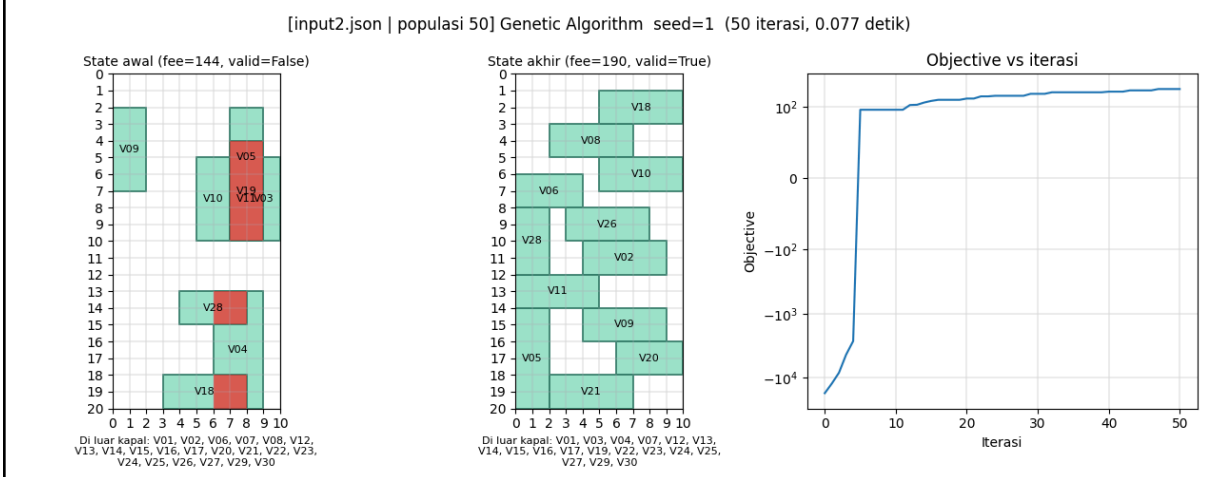
2.7.6 Hasil Eksperimen *Genetic Algorithm*

2.7.6.1 Skenario 1: Populasi tetap, iterasi divariasikan

Tabel 17. Hasil Eksperimen Skenario 1 Konfigurasi 1 *Genetic Algorithm*

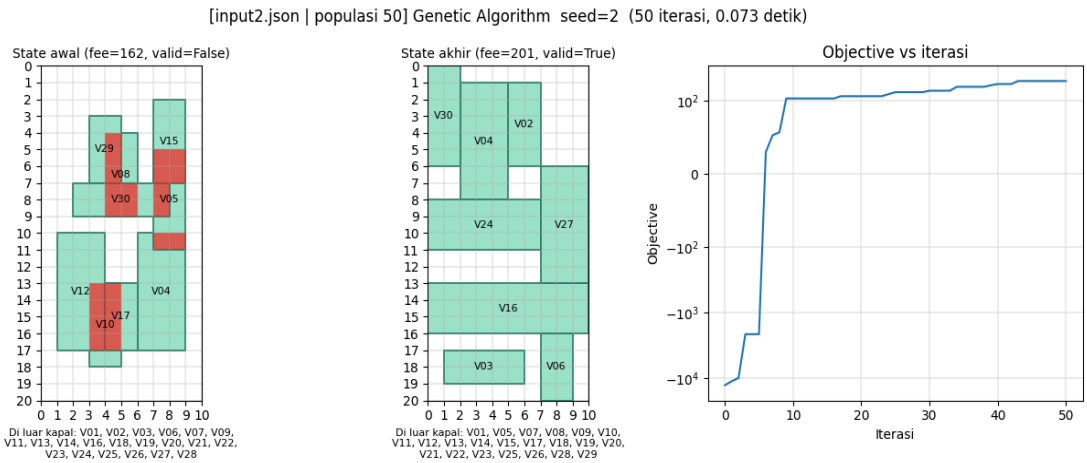
Populasi	50
Iterasi	50
Percobaan 1	
Durasi	0.077 s

Jumlah populasi	50	
Banyak iterasi	50	
Mutation rate	0,5	
State Awal	**Fee**	144 juta
	Berat (Max. 500)	180
	Objective	-17371.0098
	Status	Tidak valid (31 sel bertumpuk)
State Akhir	**Fee**	190 juta
	Berat (Max. 500)	186
	Objective	189.9993
	Status	Valid



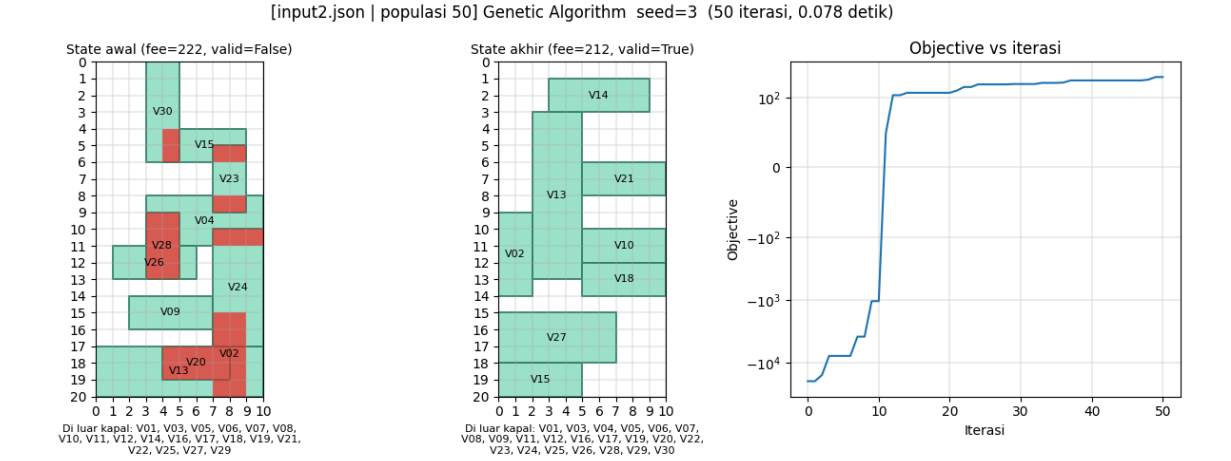
Percobaan 2		
Durasi	0.073 s	
Jumlah populasi	50	
Banyak iterasi	50	
Mutation rate	0,5	
State Awal	**Fee**	162 juta

	Berat (Max. 500)	221
	Objective	-12832.9767
	Status	Tidak valid (23 sel bertumpuk)
State Akhir	**Fee**	201 juta
	Berat (Max. 500)	322
	Objective	201.0021
	Status	Valid

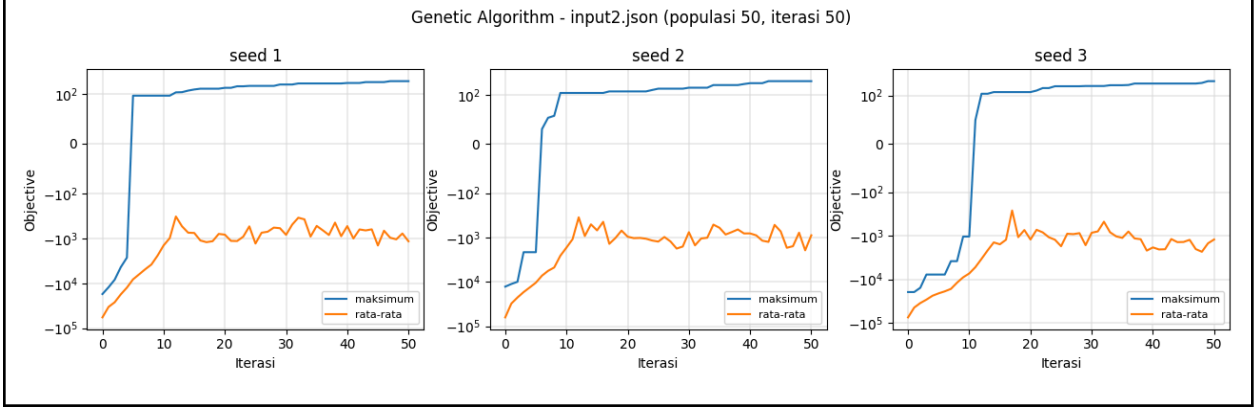


Percobaan 3		
Durasi	0.078 s	
Jumlah populasi	50	
Banyak iterasi	50	
Mutation rate	0,5	
State Awal	**Fee**	222 juta
	Berat (Max. 500)	348
	Objective	-19552.9894
	Status	Tidak valid (35 sel bertumpuk)

State Akhir	**Fee**	212 juta
	Berat (Max. 500)	290
	Objective	212.0192
	Status	Valid



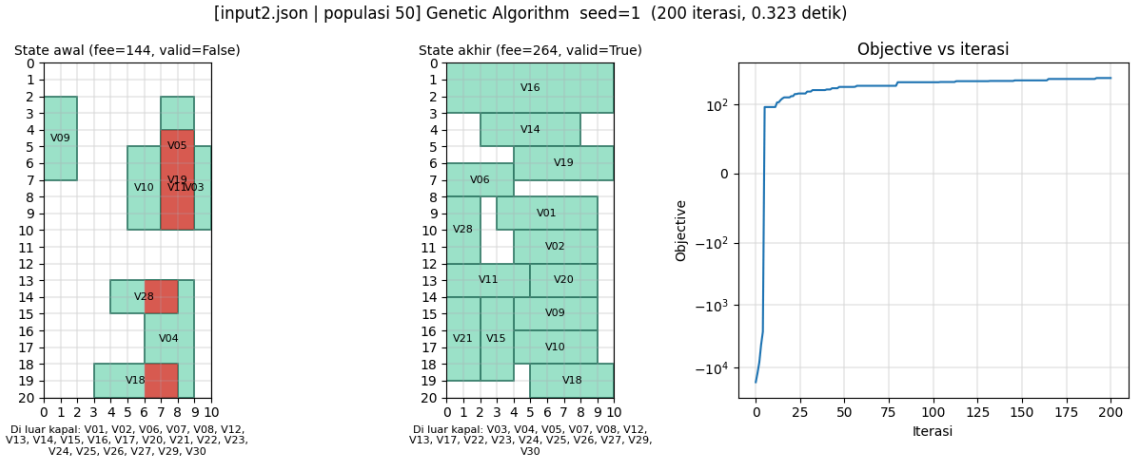
Plot Objective Maksimum dan Rata-rata



Tabel 18. Hasil Eksperimen Skenario 1 Konfigurasi 2 *Genetic Algorithm*

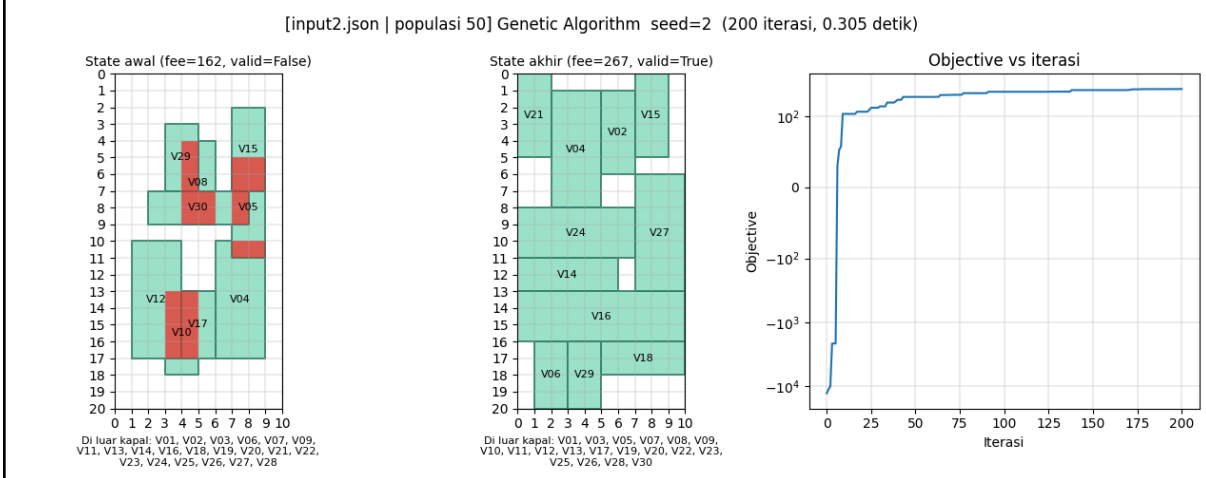
Populasi	50
Iterasi	200
Percobaan 1	

Durasi	0.323 s	
Jumlah populasi	50	
Banyak iterasi	200	
Mutation rate	0,5	
State Awal	**Fee**	144 juta
	Berat (Max. 500)	180
	Objective	-17371.0098
	Status	Tidak valid (31 sel bertumpuk)
State Akhir	**Fee**	264 juta
	Berat (Max. 500)	316
	Objective	264.0153
	Status	Valid



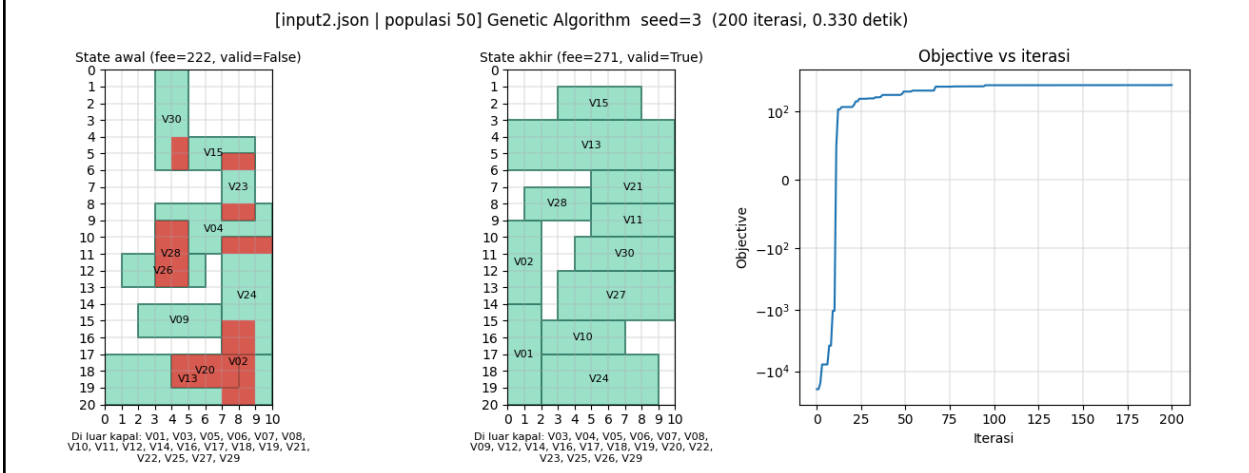
Percobaan 2		
Durasi	0.305 s	
Jumlah populasi	50	
Banyak iterasi	200	
Mutation rate	0,5	

State Awal	**Fee**	162 juta
	Berat (Max. 500)	221
	Objective	-12832.9767
	Status	Tidak valid (23 sel bertumpuk)
State Akhir	**Fee**	267 juta
	Berat (Max. 500)	378
	Objective	267.0369
	Status	Valid

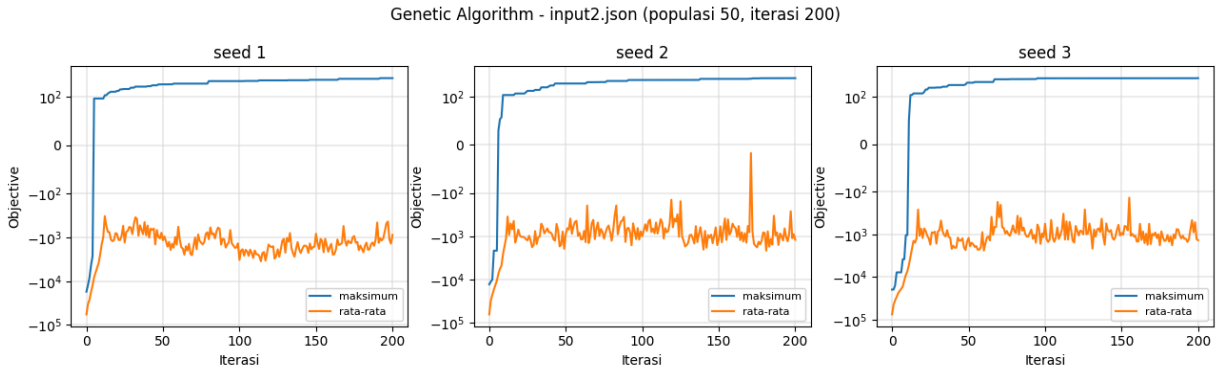


Percobaan 3		
Durasi	0.330 s	
Jumlah populasi	50	
Banyak iterasi	200	
Mutation rate	0,5	
State Awal	**Fee**	222 juta
	Berat (Max. 500)	348
	Objective	-19552.9894
	Status	Tidak valid (35 sel

		bertumpuk)
State Akhir	**Fee**	271 juta
	Berat (Max. 500)	392
	Objective	271.0263
	Status	Valid



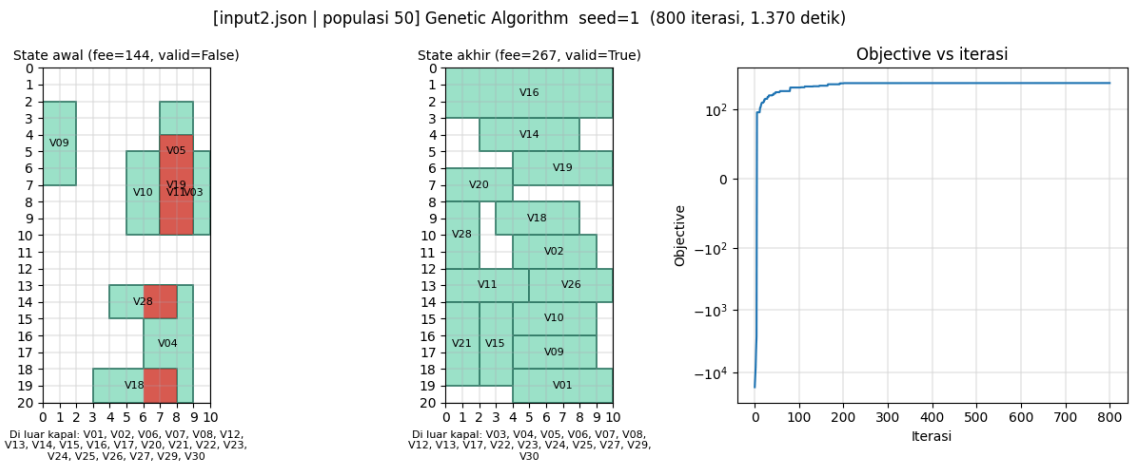
Plot Objective Maksimum dan Rata-rata



Tabel 19. Hasil Eksperimen Skenario 1 Konfigurasi 3 *Genetic Algorithm*

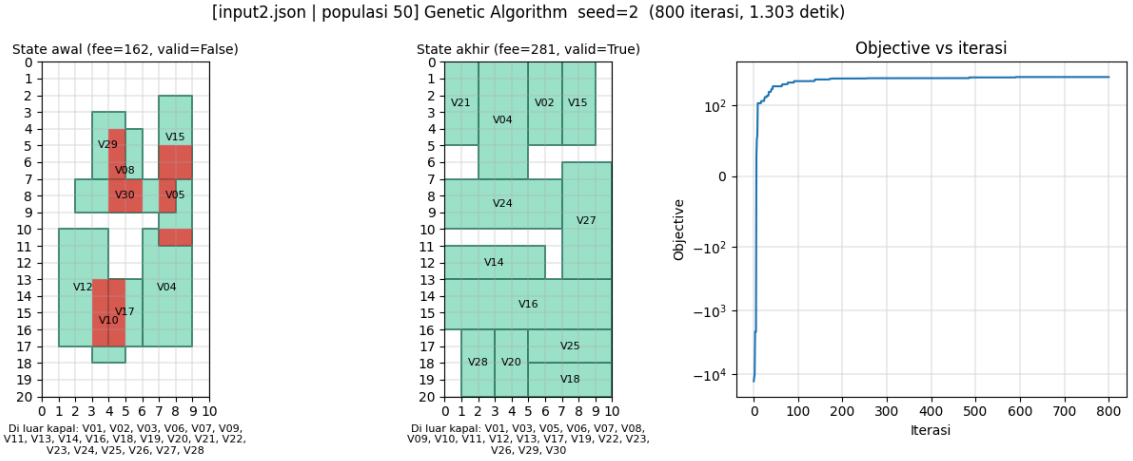
Populasi	50
Iterasi	800

Percobaan 1		
Durasi	1.370 s	
Jumlah populasi	50	
Banyak iterasi	800	
Mutation rate	0,5	
State Awal	**Fee**	144 juta
	Berat (Max. 500)	180
	Objective	-17371.0098
	Status	Tidak valid (31 sel bertumpuk)
State Akhir	**Fee**	267 juta
	Berat (Max. 500)	320
	Objective	267.0167
	Status	Valid



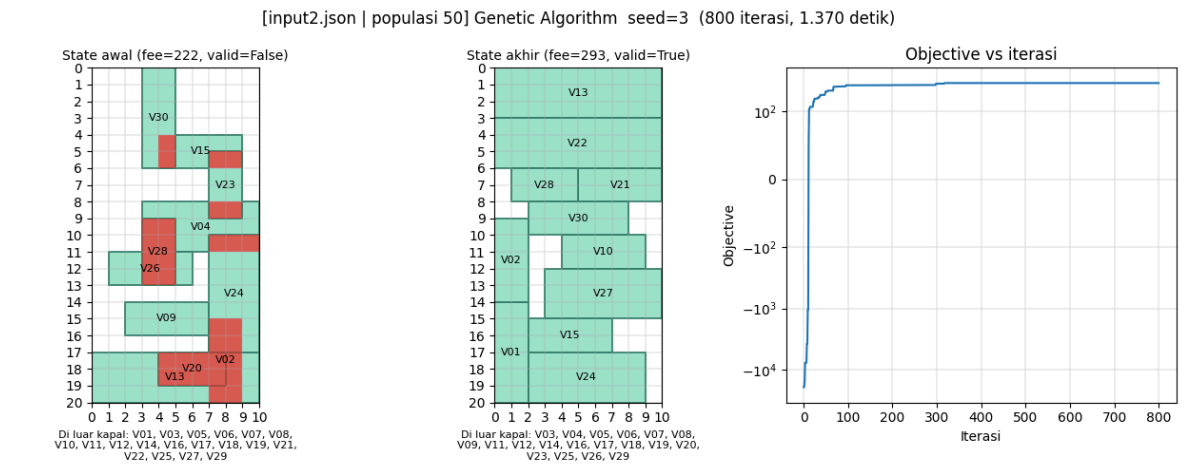
Percobaan 2	
Durasi	1.303 s
Jumlah populasi	50
Banyak iterasi	800

Mutation rate	0,5	
State Awal	**Fee**	162 juta
	Berat (Max. 500)	221
	Objective	-12832.9767
	Status	Tidak valid (23 sel bertumpuk)
State Akhir	**Fee**	281 juta
	Berat (Max. 500)	390
	Objective	281.0300
	Status	Valid

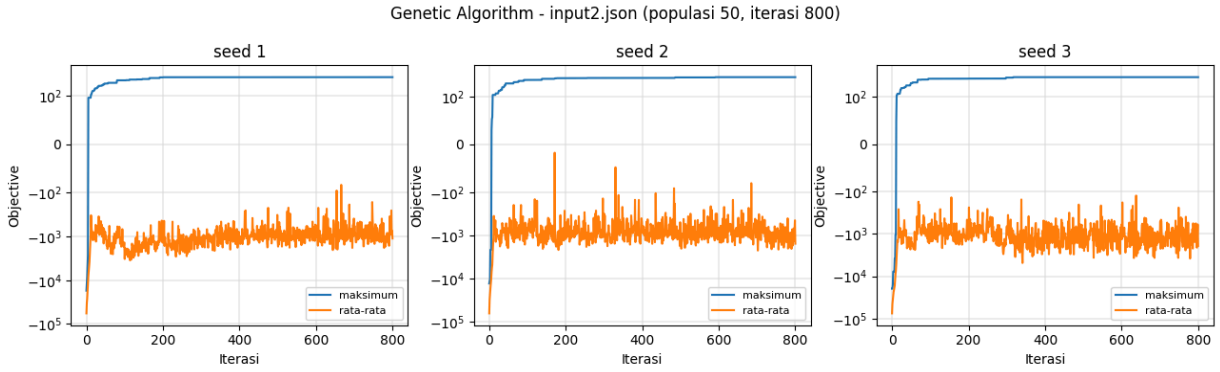


Percobaan 3		
Durasi	1.370 s	
Jumlah populasi	50	
Banyak iterasi	800	
Mutation rate	0,5	
State Awal	**Fee**	222 juta
	Berat (Max. 500)	348
	Objective	-19552.9894

	Status	Tidak valid (35 sel bertumpuk)
State Akhir	**Fee**	293 juta
	Berat (Max. 500)	493
	Objective	293.0311
	Status	Valid



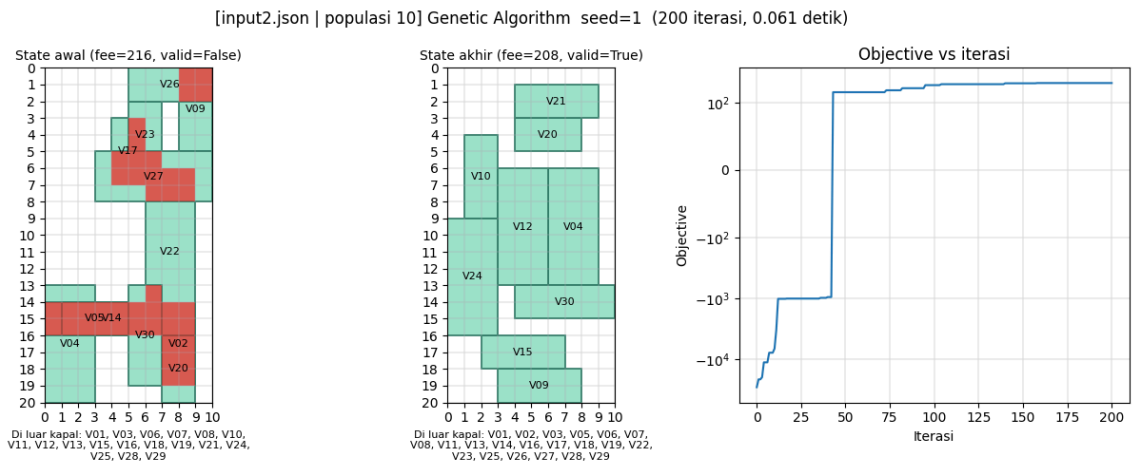
Plot Objective Maksimum dan Rata-rata



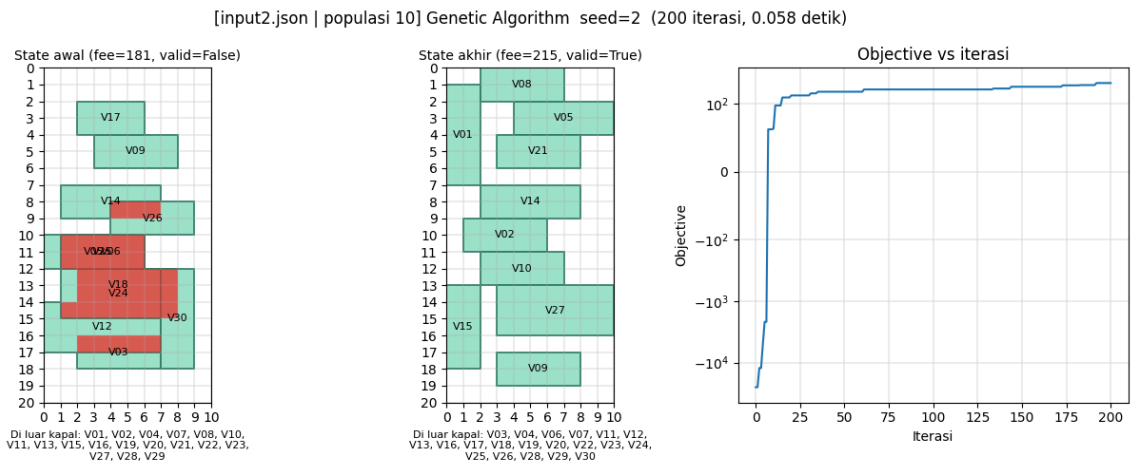
2.7.6.2 Skenario 2: Populasi divariasikan, iterasi tetap

Tabel 20. Hasil Eksperimen Skenario 2 Konfigurasi 1 *Genetic Algorithm*

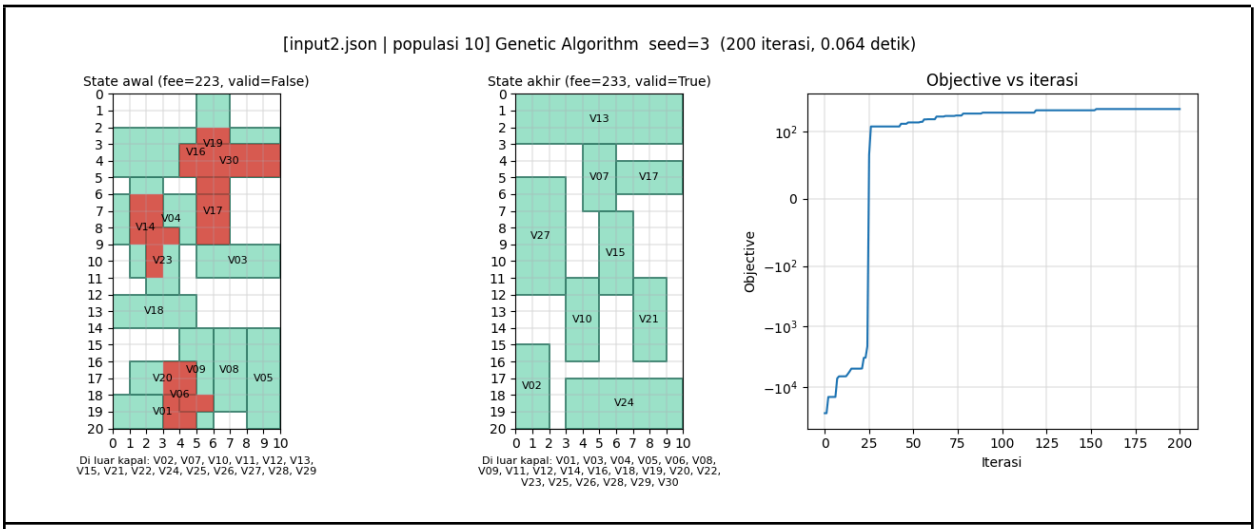
Populasi	10	
Iterasi	200	
Percobaan 1		
Durasi	0.061 s	
Jumlah populasi	10	
Banyak iterasi	200	
Mutation rate	0,5	
State Awal	**Fee**	216 juta
	Berat (Max. 500)	370
	Objective	-28598.9992
	Status	Tidak valid (51 sel bertumpuk)
State Akhir	**Fee**	208 juta
	Berat (Max. 500)	270
	Objective	208.0428
	Status	Valid



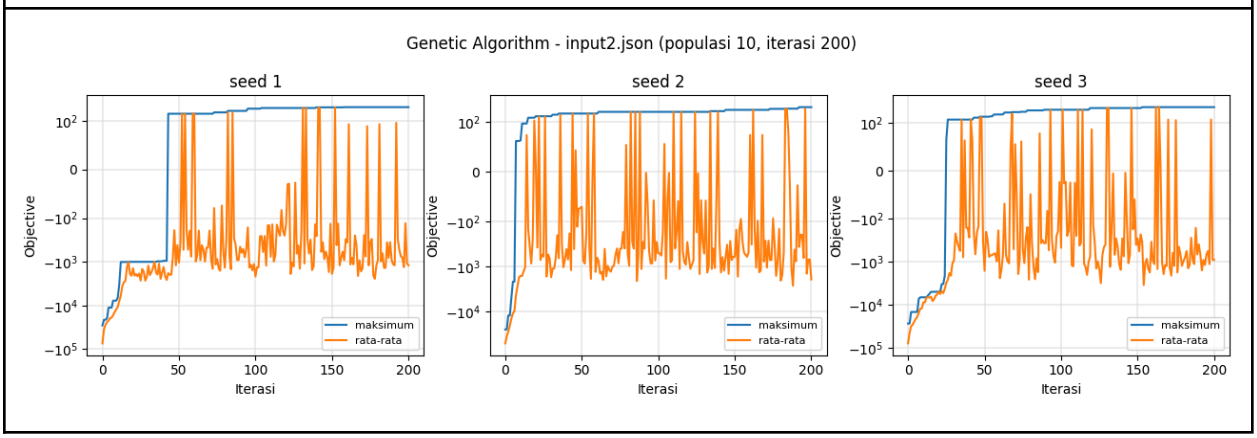
Percobaan 2		
Durasi	0.058 s	
Jumlah populasi	10	
Banyak iterasi	200	
Mutation rate	0,5	
State Awal	**Fee**	181 juta
	Berat (Max. 500)	269
	Objective	-25244.0207
	Status	Tidak valid (45 sel bertumpuk)
State Akhir	**Fee**	215 juta
	Berat (Max. 500)	242
	Objective	215.0397
	Status	Valid



Percobaan 3		
Durasi	0.064 s	
Jumlah populasi	10	
Banyak iterasi	200	
Mutation rate	0,5	
State Awal	**Fee**	223 juta
	Berat (Max. 500)	352
	Objective	-26897.0038
	Status	Tidak valid (48 sel bertumpuk)
State Akhir	**Fee**	233 juta
	Berat (Max. 500)	338
	Objective	233.0369
	Status	Valid



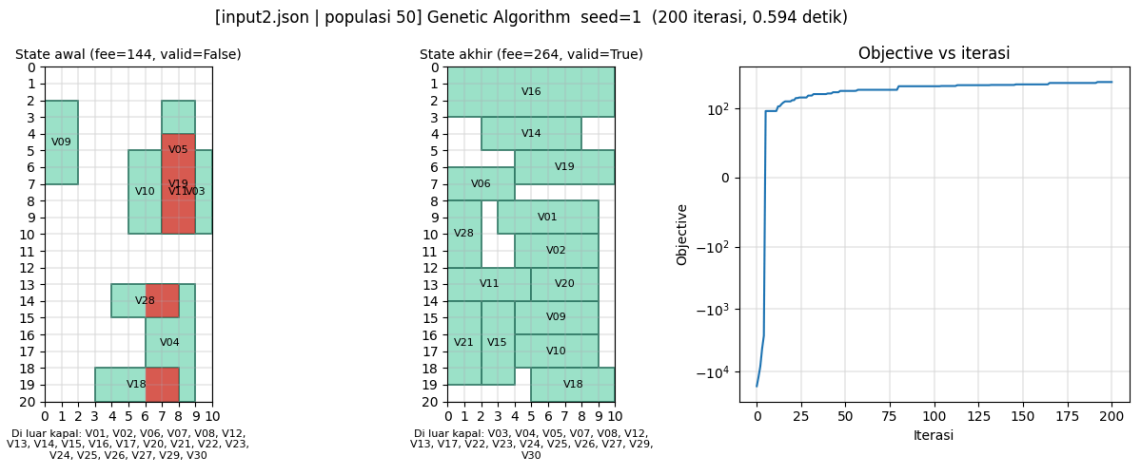
Plot Objective Maksimum dan Rata-rata



Tabel 21. Hasil Eksperimen Skenario 2 Konfigurasi 2 *Genetic Algorithm*

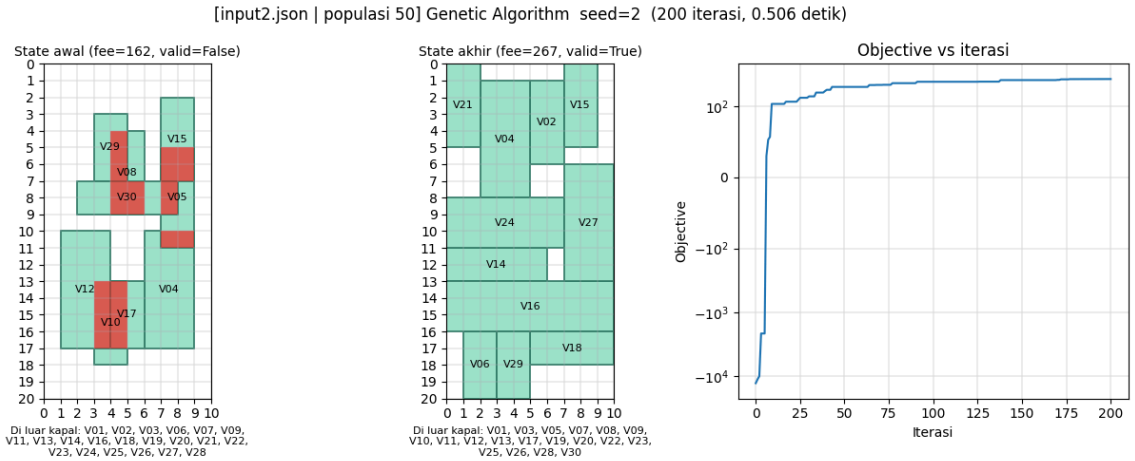
Populasi	50
Iterasi	200
Percobaan 1	
Durasi	0.594 s
Jumlah populasi	50
Banyak iterasi	200
Mutation rate	0,5

State Awal	**Fee**	144 juta
	Berat (Max. 500)	180
	Objective	-17371.0098
	Status	Tidak valid (31 sel bertumpuk)
State Akhir	**Fee**	264 juta
	Berat (Max. 500)	316
	Objective	264.0153
	Status	Valid



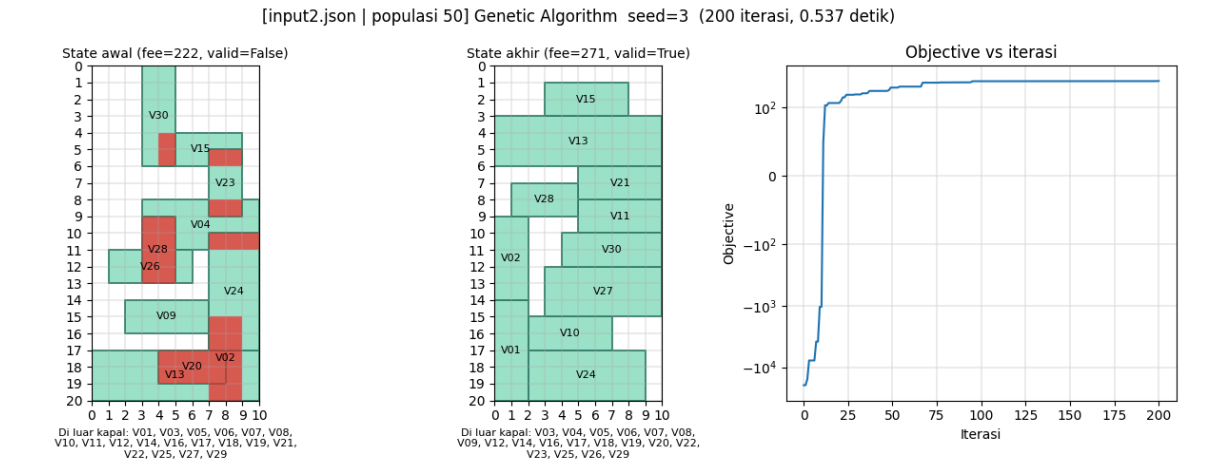
Percobaan 2		
Durasi	0.506 s	
Jumlah populasi	50	
Banyak iterasi	200	
Mutation rate	0,5	
State Awal	**Fee**	162 juta
	Berat (Max. 500)	221
	Objective	-12832.9767
	Status	Tidak valid (23 sel

		bertumpuk)
State Akhir	**Fee**	267 juta
	Berat (Max. 500)	378
	Objective	267.0369
	Status	Valid

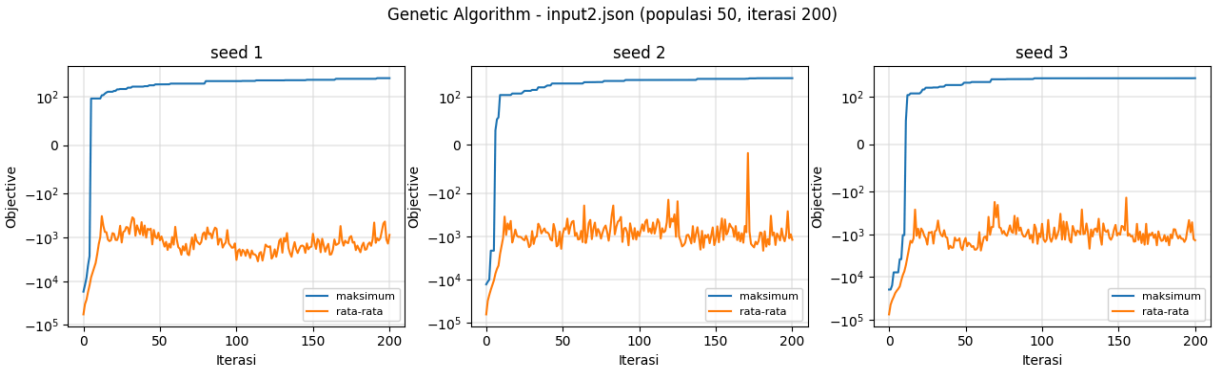


Percobaan 3		
Durasi	0.537 s	
Jumlah populasi	50	
Banyak iterasi	200	
Mutation rate	0,5	
State Awal	**Fee**	222 juta
	Berat (Max. 500)	348
	Objective	-19552.9894
	Status	Tidak valid (35 sel bertumpuk)
State Akhir	**Fee**	271 juta
	Berat (Max. 500)	392
	Objective	271.0263

	Status	Valid



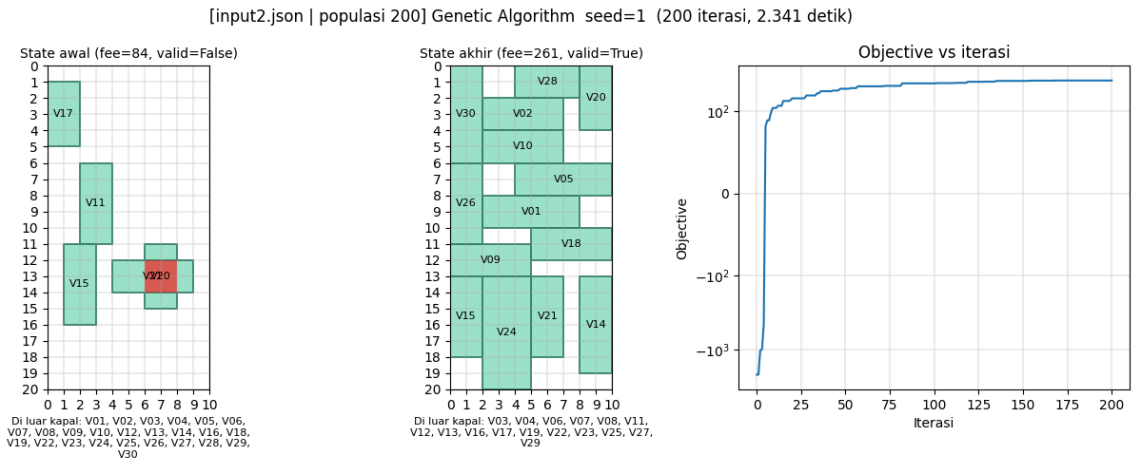
Plot Objective Maksimum dan Rata-rata



Tabel 22. Hasil Eksperimen Skenario 2 Konfigurasi 3 *Genetic Algorithm*

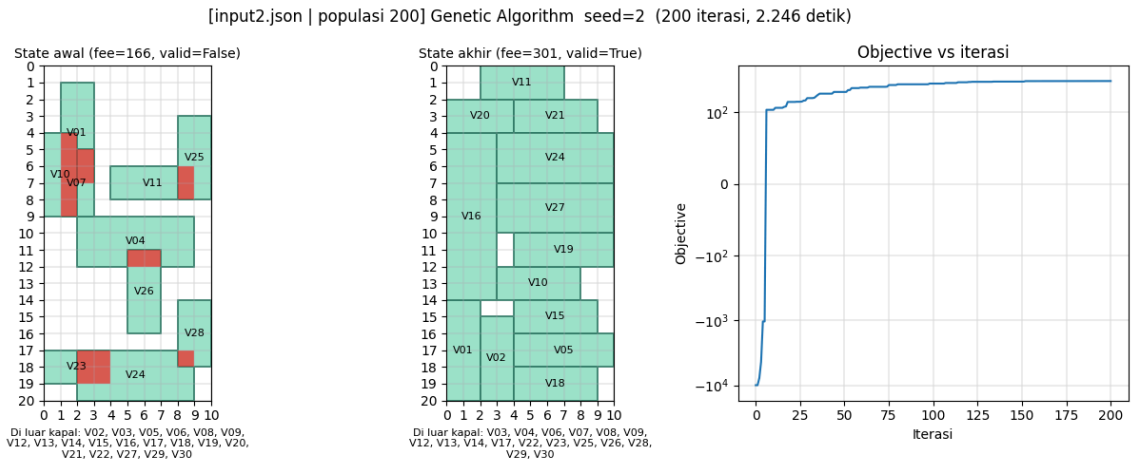
Populasi	200
Iterasi	200
Percobaan 1	
Durasi	2.341 s
Jumlah populasi	200
Banyak iterasi	200

Mutation rate	0,5	
State Awal	**Fee**	84 juta
	Berat (Max. 500)	81
	Objective	-2175.9721
	Status	Tidak valid (4 sel bertumpuk)
State Akhir	**Fee**	261 juta
	Berat (Max. 500)	301
	Objective	261.0431
	Status	Valid



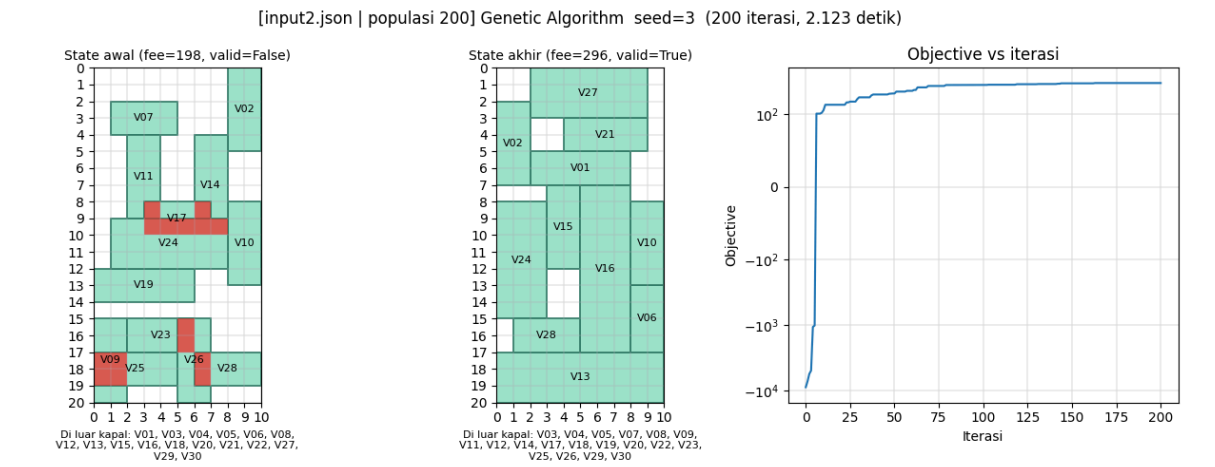
Percobaan 2		
Durasi	2.246 s	
Jumlah populasi	200	
Banyak iterasi	200	
Mutation rate	0,5	
State Awal	**Fee**	166 juta
	Berat (Max. 500)	236
	Objective	-10003.9807

	Status	Tidak valid (18 sel bertumpuk)
State Akhir	**Fee**	301 juta
	Berat (Max. 500)	405
	Objective	301.0284
	Status	Valid

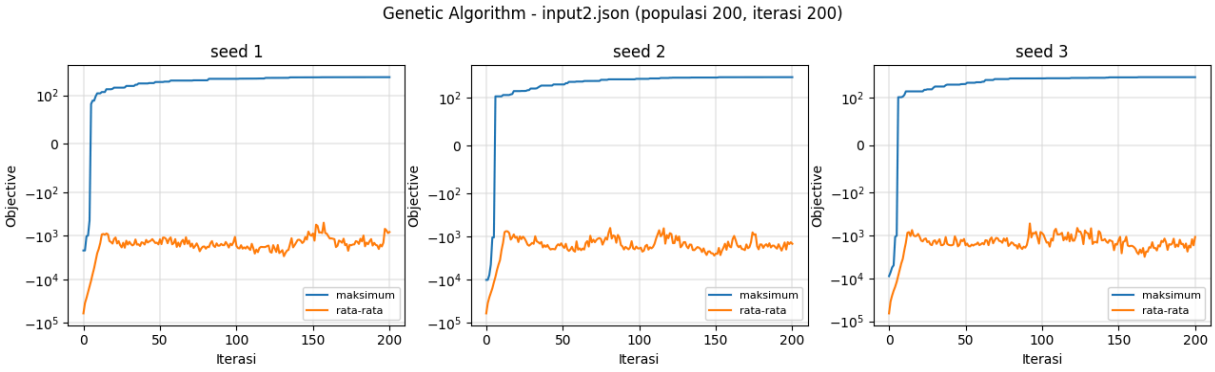


Percobaan 3		
Durasi	2.123 s	
Jumlah populasi	200	
Banyak iterasi	200	
Mutation rate	0,5	
State Awal	**Fee**	198 juta
	Berat (Max. 500)	245
	Objective	-8842.0158
	Status	Tidak valid (16 sel bertumpuk)
State Akhir	**Fee**	296 juta
	Berat (Max. 500)	451

	Objective	296.0322
	Status	Valid



Plot Objective Maksimum dan Rata-rata



2.8 Analisis Hasil Eksperimen

2.8.1 Analisis Pertanyaan 1

Tabel 23. Analisis Eksperimen Pertanyaan 1

Pertanyaan 1. Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian?	
Algoritma	**Penjelasan**

Steepest Ascent Hill Climbing	*Steepest Ascent Hill-Climbing* memeriksa seluruh tetangga pada setiap iterasi dan selalu berpindah ke tetangga dengan nilai tertinggi. Dengan cara ini, setiap langkah merupakan perbaikan terbesar yang tersedia dari *state* saat ini, sehingga pencarian bergerak cepat menuju puncak terdekat. Namun, algoritma langsung berhenti begitu tidak ada tetangga yang lebih baik, tanpa mekanisme apa pun untuk keluar dari *local optimum.* Pada eksperimen input2, ketiga percobaan berhenti karena mencapai *local optimum* dalam waktu singkat, yaitu hanya 18 hingga 26 iterasi. Hasil akhirnya berada di bawah *Sideways Move, Random Restart*, dan *Simulated Annealing* pada sebagian besar percobaan, sehingga *Steepest Ascent* belum mendekati *global optima.* Hal ini terjadi karena ruang pencarian input2 memiliki banyak *local optimum*, dan *Steepest Ascent* hanya mampu mencapai *local optimum* yang paling dekat dengan *state* awalnya. Karena algoritma tidak memiliki unsur acak lain selain *state* awal, kualitas hasil akhirnya sepenuhnya ditentukan oleh titik awal pencarian. Hal ini terlihat dari percobaan kedua yang menghasilkan nilai jauh lebih tinggi dibandingkan dua percobaan lainnya, karena *state* awalnya kebetulan mengarah ke *local optimum* yang lebih baik.
Stochastic Hill-Climbing	*Stochastic Hill-Climbing* hanya membangkitkan satu tetangga acak pada setiap iterasi dan langsung berpindah apabila tetangga tersebut lebih baik. Algoritma tidak membandingkan seluruh kemungkinan langkah, sehingga perbaikan yang diambil belum tentu yang terbesar. Pencarian dihentikan ketika tidak ditemukan perbaikan dalam sejumlah iterasi berturut-turut yang ditentukan oleh parameter `patience`. Pada eksperimen input2, *Stochastic Hill-Climbing* menghasilkan total `ShippingFee` akhir terendah di antara algoritma *hill-climbing* dan *Simulated Annealing* pada percobaan kedua dan ketiga, sehingga merupakan salah satu algoritma yang paling jauh dari *global optima.* Ketiga percobaan berhenti karena batas `patience` tercapai, dengan hanya 39 hingga 46 langkah yang diterima dari 1.261 hingga 1.648 iterasi. Hal ini menunjukkan bahwa sebagian besar tetangga acak yang dibangkitkan tidak memberikan perbaikan, terutama ketika *state* sudah cukup baik. Karena berhenti berdasarkan `patience`, algoritma bisa saja berhenti sebelum benar-benar mencapai *local optimum*, sebab masih mungkin ada tetangga

	yang lebih baik tetapi tidak sempat terpilih secara acak. Hasilnya juga dipengaruhi keberuntungan dalam memilih tetangga yaitu pada percobaan pertama, *Stochastic* justru menghasilkan nilai sedikit lebih tinggi dari *Steepest Ascent*, sedangkan pada percobaan kedua hasilnya jauh lebih rendah meskipun dimulai dari *state* awal yang sama.
Hill Climbing with Sideways Move	*Hill-Climbing with Sideways Move* bekerja seperti *Steepest Ascent*, tetapi tetap mau berpindah ketika tetangga terbaik memiliki nilai yang sama dengan *state* saat ini. Kemampuan ini ditujukan untuk melewati *plateau*, yaitu daerah datar yang membuat *Steepest Ascent* langsung berhenti, sehingga pencarian berpeluang menemukan jalan menuju *local optimum* yang lebih baik. Pada permasalahan penataan kendaraan, *plateau* muncul ketika dua susunan geladak berbeda menghasilkan nilai *objective* yang sama, misalnya saat kendaraan dipindah ke posisi lain tanpa mengubah total `ShippingFee` maupun komponen pemecah serinya. Pada eksperimen input2, *sideways move* terjadi pada ketiga percobaan, dan percobaan kedua menghasilkan susunan dengan total `ShippingFee` tertinggi di antara seluruh eksperimen, dengan kapasitas kapal yang hampir penuh. Namun, kemampuan algoritma ini dalam mendekati *global optima* tidak konsisten, karena dua percobaan lainnya berhenti di *local optimum* yang lebih rendah. Hal ini terjadi karena *plateau* pada permasalahan ini jarang dan sempit yaitu komponen ETA dan posisi pada *objective function* membuat dua *state* hampir tidak pernah memiliki nilai yang persis sama. Akibatnya, *sideways move* hanya membantu melewati daerah datar yang kecil, bukan keluar dari *local optimum* yang dalam. Ketiga percobaan juga berhenti karena seluruh tetangga bernilai sama sudah pernah dikunjungi, bukan karena batas *sideways move* tercapai. Dengan demikian, seberapa dekat hasilnya ke *global optima* sangat ditentukan oleh *state* awal.
Random Restart Hill Climbing	*Random Restart Hill-Climbing* mengatasi kelemahan *hill-climbing* yang mudah terjebak di *local optimum* dengan cara mengulang pencarian dari *state* awal acak yang berbeda. Setiap pencarian berakhir di *local optimum* masing-masing, dan algoritma mengambil hasil terbaik dari seluruh pencarian tersebut. Semakin banyak *restart* yang dilakukan, semakin banyak *local optimum* yang dijelajahi, sehingga peluang

	menemukan solusi yang dekat dengan *global optima* semakin besar. Pada eksperimen input2, total `ShippingFee` akhir dari setiap pencarian dalam satu percobaan sangat bervariasi, yang menunjukkan bahwa ruang pencarian input2 memiliki banyak *local optimum* dengan kualitas berbeda-beda. *Restart* terbukti menaikkan hasil dibandingkan pencarian pertama pada ketiga percobaan, dan hasil akhirnya lebih merata dibandingkan *Sideways Move* serta hasil terburuknya pun masih lebih baik daripada hasil terburuk *Sideways Move*. Meski begitu, *Random Restart* tidak menghasilkan solusi terbaik secara keseluruhan, karena setiap *restart* tetap memakai *Steepest Ascent* yang berhenti di *local optimum* terdekat dari titik awalnya. Kedekatan ke *global optima* pada algoritma ini lebih ditentukan oleh jumlah *restart* daripada oleh satu *state* awal. Pada percobaan kedua, solusi terbaik baru ditemukan pada *restart* terakhir, yang menandakan bahwa menambah jumlah *restart* masih berpotensi memperbaiki hasil.
Simulated Annealing	*Simulated Annealing* menerima tetangga yang lebih buruk dengan peluang yang bergantung pada suhu. Saat suhu masih tinggi, langkah yang memburuk cukup sering diterima, sehingga algoritma dapat keluar dari *local optimum* dan menjelajahi daerah lain. Seiring suhu turun, algoritma makin jarang menerima langkah yang memburuk dan akhirnya hanya menyempurnakan solusi yang sudah ada. Pada eksperimen input2, *Simulated Annealing* menghasilkan total `ShippingFee` yang cukup dekat dengan *Sideways Move* dan *Random Restart*, dengan selisih hanya beberapa juta pada setiap percobaan, dan lebih tinggi dari *Steepest Ascent* pada percobaan pertama dan ketiga. Hal ini menunjukkan bahwa mekanisme penerimaan langkah yang lebih buruk berhasil membawa pencarian keluar dari *local optimum* yang dangkal. Namun, *Simulated Annealing* tidak pernah menghasilkan nilai tertinggi pada satu percobaan pun. Hal ini terjadi karena setiap iterasi hanya mengevaluasi satu tetangga acak, sehingga ketika suhu sudah rendah, perbaikan kecil yang tersisa belum tentu ditemukan sebelum pencarian berhenti. Selain itu, pencarian selalu berhenti tepat saat suhu mencapai suhu minimum, sehingga jumlah iterasinya tetap walaupun solusi masih mungkin diperbaiki.

Genetic Algorithm	*Genetic Algorithm* tidak mengelola satu *state*, melainkan sekumpulan *state* yang disebut populasi. Pada setiap generasi, setengah populasi terbaik dipertahankan, kemudian dikombinasikan melalui *crossover* dan diberi mutasi untuk menghasilkan individu baru. Secara teori, cara ini memungkinkan algoritma menjelajahi beberapa daerah ruang pencarian sekaligus, sehingga tidak mudah terjebak di satu *local optimum.* Pada eksperimen input2, kedekatan *Genetic Algorithm* terhadap *global optima* sangat bergantung pada jumlah populasi dan banyak iterasi. Dengan populasi atau iterasi yang kecil, hasilnya menjadi yang terendah dari seluruh eksperimen, sedangkan dengan populasi 200 hasil terbaiknya mencapai 301 juta. Namun, rata-rata hasil *Genetic Algorithm* pada seluruh konfigurasi tetap berada di bawah *Steepest Ascent, Sideways Move, Random Restart*, dan *Simulated Annealing.* Hal ini kemungkinan terjadi karena pada permasalahan penataan geladak, posisi kendaraan saling bergantung satu sama lain. Crossover yang mewariskan posisi setiap kendaraan dari salah satu induk secara acak sering menghasilkan kendaraan yang bertumpuk, terutama pada generasi awal ketika induk-induknya masih berbeda jauh. Mutasi yang memindahkan kendaraan ke posisi acak juga dapat menimbulkan tumpang tindih. Akibatnya, banyak individu baru yang langsung tersingkir oleh penalti.

2.8.2 Analisis Pertanyaan 2

Tabel 24. Perbandingan Total `ShippingFee` Akhir Tiap Algoritma

Algoritma	Seed 1	Seed 2	Seed 3	Rata-rata
Steepest Ascent Hill Climbing	273	310	286	289,7
Stochastic Hill-Climbing	280	245	269	264,7
Hill Climbing with Sideways Move	294	323	300	305,7
Random Restart Hill Climbing	300	312	303	305,0
Simulated Annealing	293	307	298	299,3
Genetic Algorithm **(populasi 50, iterasi 50)**	190	201	212	201,0

Genetic Algorithm **(populasi 50, iterasi 200)**	264	267	271	267,3
Genetic Algorithm **(populasi 50, iterasi 800)**	267	281	293	280,3
Genetic Algorithm **(populasi 10, iterasi 200)**	208	215	233	218,7
Genetic Algorithm **(populasi 50, iterasi 200)**	264	267	271	267,3
Genetic Algorithm **(populasi 200, iterasi 200)**	261	301	296	286,0

Tabel 25. Analisis Eksperimen Pertanyaan 2

Pertanyaan 2. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?	
Algoritma	**Penjelasan**
Steepest Ascent Hill Climbing	Berdasarkan Tabel 24, **rata-rata hasil *Steepest Ascent Hill-Climbing* berada di urutan keempat**, di bawah *Sideways Move, Random Restart*, dan *Simulated Annealing,* tetapi di atas *Stochastic Hill-Climbing* dan seluruh konfigurasi *Genetic Algorithm.* Dibandingkan *Stochastic* yang dimulai dari *state* awal yang sama, *Steepest Ascent* unggul pada percobaan kedua dan ketiga, tetapi sedikit kalah pada percobaan pertama. Keunggulan ini berasal dari cara *Steepest Ascent* yang selalu memilih perbaikan terbesar dari seluruh tetangga. Namun, algoritma ini langsung berhenti begitu tidak ada tetangga yang lebih baik, sehingga kalah dari algoritma yang memiliki mekanisme untuk melanjutkan pencarian, seperti *sideways move, restart,* dan penerimaan langkah yang lebih buruk pada *Simulated Annealing.* Pada percobaan ketiga, hasilnya bahkan berada di bawah *Genetic Algorithm* dengan populasi 200 dan dengan 800 iterasi.
Stochastic Hill-Climbing	Berdasarkan Tabel 24, ***Stochastic Hill-Climbing*** **menghasilkan rata-rata total `ShippingFee` terendah di antara algoritma hill-climbing dan *Simulated Annealing***. Pada percobaan kedua, hasilnya lebih rendah 65 juta dibandingkan *Steepest Ascent* yang dimulai dari *state* awal yang sama. Hal ini terjadi karena algoritma hanya mengevaluasi satu tetangga acak pada setiap iterasi dan berhenti setelah 300 tetangga berturut-turut tidak memberikan perbaikan, sehingga pencarian dapat

	berhenti sebelum mencapai *local optimum*. Dibandingkan *Genetic Algorithm*, rata-rata hasil *Stochastic* lebih tinggi dari konfigurasi dengan populasi 10 dan dengan 50 iterasi, tetapi lebih rendah dari tiga konfigurasi lainnya.
Hill Climbing with Sideways Move	Berdasarkan Tabel 24, ***Hill-Climbing with Sideways Move* menghasilkan rata-rata total `ShippingFee` tertinggi di antara seluruh algoritma,** sekaligus hasil terbaik dari seluruh eksperimen `input2` pada percobaan kedua. Dibandingkan *Steepest Ascent, Sideways Move* menghasilkan nilai yang lebih tinggi pada ketiga percobaan, karena kemampuannya berpindah ke tetangga yang bernilai sama memberi kesempatan untuk melanjutkan pencarian ketika *Steepest Ascent* sudah berhenti. Hasilnya juga lebih tinggi dari *Simulated Annealing* dan seluruh konfigurasi *Genetic Algorithm* pada ketiga percobaan. Dibandingkan *Random Restart*, rata-rata keduanya hampir sama, tetapi *Sideways Move* unggul pada percobaan kedua dan kalah pada percobaan pertama dan ketiga. Kelemahannya, selisih antara hasil terbaik dan terburuk cukup besar, karena algoritma hanya melakukan satu kali pencarian sehingga sangat bergantung pada state awal.
Random Restart Hill Climbing	Berdasarkan Tabel 24, **rata-rata hasil *Random Restart Hill-Climbing* hampir sama dengan *Sideways Move*, tetapi lebih merata antar percobaan.** *Random Restart* menghasilkan nilai tertinggi pada percobaan pertama dan ketiga, dan hasil terburuknya masih lebih baik daripada hasil terburuk seluruh algoritma lain. Hal ini terjadi karena algoritma tidak bergantung pada satu *local optimum*, tetapi memilih yang terbaik dari sebelas pencarian yang dimulai dari *state* awal berbeda. Meski begitu, *Random Restart* tidak menghasilkan solusi terbaik secara keseluruhan, karena setiap *restart* tetap menggunakan *Steepest Ascent* yang berhenti di *local optimum* terdekat. Kelebihan algoritma ini adalah kestabilan hasilnya, sedangkan kekurangannya adalah biaya komputasi yang jauh lebih besar dibandingkan seluruh algoritma lain.
Simulated Annealing	Berdasarkan Tabel 24, ***Simulated Annealing* menghasilkan rata-rata total `ShippingFee` tertinggi ketiga, di bawah *Sideways Move* dan *Random Restart*, tetapi di atas *Steepest Ascent, Stochastic,* dan seluruh konfigurasi *Genetic Algorithm*.** Kemampuan menerima tetangga yang lebih buruk saat suhu masih tinggi memungkinkan algoritma keluar dari *local optimum* di awal pencarian, sehingga hasilnya jauh lebih baik dibandingkan *Stochastic Hill-Climbing* yang sama-sama hanya mengevaluasi satu tetangga acak per iterasi. *Simulated*

	Annealing juga menghasilkan nilai yang lebih tinggi dari *Genetic Algorithm* dengan populasi 200 pada ketiga percobaan, walaupun jumlah *state* yang dievaluasinya lebih sedikit. Kelebihan utama algoritma ini adalah kualitas hasil yang mendekati *Sideways Move* dan *Random Restart* dengan waktu komputasi yang jauh lebih singkat.
Genetic Algorithm	Berdasarkan Tabel 24, **kualitas hasil *Genetic Algorithm* sangat bergantung pada jumlah populasi dan banyak iterasi.** Pada konfigurasi terbaiknya, yaitu populasi 200 dan iterasi 200, rata-rata hasilnya berada di bawah *Sideways Move, Random Restart, Simulated Annealing,* dan *Steepest Ascent*, tetapi di atas *Stochastic Hill-Climbing.* Pada percobaan ketiga, konfigurasi populasi 200 dan konfigurasi 800 iterasi menghasilkan nilai yang lebih tinggi dari *Steepest Ascent*, sedangkan pada dua percobaan lainnya lebih rendah. Pada konfigurasi dengan populasi 10 atau dengan 50 iterasi, hasilnya menjadi yang terendah dari seluruh eksperimen. Hal ini kemungkinan terjadi karena anak hasil crossover, yang mewariskan posisi setiap kendaraan dari salah satu induk secara acak, dan hasil mutasi sering memiliki kendaraan yang bertumpuk, sehingga langsung tersingkir oleh penalti. Akibatnya, *Genetic Algorithm* membutuhkan lebih banyak evaluasi untuk mencapai kualitas yang sama dengan *Simulated Annealing*. Kelebihannya, hasil algoritma ini cenderung membaik seiring bertambahnya populasi dan iterasi.

2.8.3 Analisis Pertanyaan 3

Tabel 26. Perbandingan Durasi Pencarian Tiap Algoritma

Algoritma	Seed 1	Seed 2	Seed 3	Rata-rata
Steepest Ascent Hill Climbing	4,010	3,959	5,903	4,624
Stochastic Hill-Climbing	0,137	0,071	0,085	0,098
Hill Climbing with Sideways Move	7,37	11,28	14,40	11,017
Random Restart Hill Climbing	103,98	90,35	92,64	95,657
Simulated Annealing	0,537	0,612	0,591	0,580
Genetic Algorithm **(populasi 50, iterasi 50)**	0,077	0,073	0,078	0,076

Genetic Algorithm **(populasi 50, iterasi 200)**	0,323	0,305	0,330	0,319
Genetic Algorithm **(populasi 50, iterasi 800)**	1,370	1,303	1,370	1,348
Genetic Algorithm **(populasi 10, iterasi 200)**	0,061	0,058	0,064	0,061
Genetic Algorithm **(populasi 50, iterasi 200)**	0,594	0,506	0,537	0,546
Genetic Algorithm **(populasi 200, iterasi 200)**	2,341	2,246	2,123	2,237

Tabel 27. Analisis Eksperimen Pertanyaan 3

Pertanyaan 3. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?	
Algoritma	**Penjelasan**
Steepest Ascent Hill Climbing	**Durasi relatif sedang.** Berdasarkan data eksperimen pada Tabel 26, rata-rata durasi pencarian adalah 4,624 detik dengan 18 hingga 26 iterasi. Jumlah iterasinya sedikit, tetapi setiap iterasi harus membangkitkan dan mengevaluasi seluruh tetangga, yaitu hampir 4.000 tetangga per iterasi pada input2. Karena itu, Steepest Ascent jauh lebih lambat dibandingkan algoritma yang hanya mengevaluasi satu tetangga per iterasi.
Stochastic Hill-Climbing	**Durasi relatif sangat cepat.** Berdasarkan data eksperimen pada Tabel 26, rata-rata durasi pencarian adalah 0,098 detik dengan 1.261 hingga 1.648 iterasi, atau sekitar 47 kali lebih cepat dari Steepest Ascent. Walaupun iterasinya banyak, setiap iterasi hanya membangkitkan dan mengevaluasi satu tetangga acak, sehingga beban komputasinya sangat ringan.
Hill Climbing with Sideways Move	**Durasi relatif lambat.** Berdasarkan data eksperimen pada Tabel 26, rata-rata durasi pencarian adalah 11,017 detik dengan 22 hingga 42 iterasi, sekitar dua kali lebih lama dari Steepest Ascent. Setiap iterasinya sama beratnya dengan Steepest Ascent karena sama-sama mengevaluasi seluruh tetangga, tetapi jumlah iterasinya lebih banyak karena algoritma melanjutkan pencarian melalui sideways move ketika menemui tetangga yang bernilai sama.

Random Restart Hill Climbing	**Durasi paling lambat.** Berdasarkan data eksperimen pada Tabel 26, rata-rata durasi pencarian adalah 95,657 detik dengan total 265 hingga 307 iterasi untuk 10 restart. Durasi ini sekitar 20 kali lebih lama dari Steepest Ascent, karena dalam satu percobaan algoritma menjalankan Steepest Ascent secara penuh sebanyak 11 kali. Semakin banyak restart yang dilakukan, semakin lama pula durasi pencariannya.
Simulated Annealing	**Durasi relatif cepat.** Berdasarkan data eksperimen pada Tabel 26, rata-rata durasi pencarian adalah 0,580 detik dengan 8.513 iterasi pada setiap percobaan. Seperti Stochastic, algoritma ini hanya mengevaluasi satu tetangga acak per iterasi, tetapi jumlah iterasinya lebih banyak karena pencarian baru berhenti saat suhu mencapai suhu minimum. Durasinya relatif stabil antar percobaan, karena jumlah iterasinya selalu sama.
Genetic Algorithm	**Durasi relatif cepat,** tetapi bergantung pada parameter. Berdasarkan data eksperimen pada Tabel 26, rata-rata durasi pencarian berkisar dari 0,061 detik pada populasi 10 dan 200 iterasi, hingga 2,237 detik pada populasi 200 dan 200 iterasi. Algoritma ini tidak mengevaluasi seluruh tetangga, melainkan hanya menghitung nilai *objective* dari anak yang dihasilkan pada setiap generasi, sehingga tetap lebih cepat dari *Steepest Ascent, Sideways Move,* dan *Random Restart*. Durasinya bertambah seiring banyak iterasi dan jumlah populasi, karena keduanya menambah jumlah anak yang harus dievaluasi.

2.8.4 Analisis Pertanyaan 4

Tabel 28. Perbandingan Hasil *Fee* Tiap Algoritma

Algoritma	Seed 1	Seed 2	Seed 3	Selisih Tertinggi dan Terendah
Steepest Ascent Hill Climbing	273	310	286	37
Stochastic Hill-Climbing	280	245	269	35
Hill Climbing with Sideways Move	294	323	300	29
Random Restart Hill Climbing	300	312	303	12
Simulated Annealing	293	307	298	14
Genetic Algorithm **(populasi 50, iterasi 50)**	190	201	212	22

Genetic Algorithm **(populasi 50, iterasi 200)**	264	267	271	7
Genetic Algorithm **(populasi 50, iterasi 800)**	267	281	293	26
Genetic Algorithm **(populasi 10, iterasi 200)**	208	215	233	25
Genetic Algorithm **(populasi 50, iterasi 200)**	264	267	271	7
Genetic Algorithm **(populasi 200, iterasi 200)**	261	301	296	40

Tabel 29. Analisis Eksperimen Pertanyaan 4

Pertanyaan 4. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?	
Algoritma	**Penjelasan**
Steepest Ascent Hill Climbing	**Kurang konsisten.** Berdasarkan Tabel 28, ketiga percobaan selalu menghasilkan susunan yang valid, tetapi total `ShippingFee` akhirnya berbeda cukup jauh, dengan s**elisih 37 juta antara hasil tertinggi dan terendah**. Selisih ini merupakan yang terbesar di antara algoritma *hill-climbing* dan *Simulated Annealing*. *Steepest Ascent* selalu memilih langkah terbaik dan tidak memiliki unsur acak lain selain *state* awal, sehingga perbedaan hasil antar percobaan sepenuhnya disebabkan oleh *state* awal yang berbeda. Setiap *state* awal mengarahkan pencarian ke *local optimum* yang berbeda, dan algoritma tidak memiliki cara untuk keluar dari *local optimum* tersebut.
Stochastic Hill-Climbing	**Kurang konsisten.** Berdasarkan Tabel 28, ketiga percobaan menghasilkan susunan yang valid, tetapi **selisih antara hasil tertinggi dan terendah mencapai 35 juta**. Ketidakkonsistenan algoritma ini berasal dari dua sumber, yaitu *state* awal yang berbeda dan pemilihan tetangga yang acak pada setiap iterasi. Selain itu, pencarian dihentikan setelah 300 tetangga berturut-turut tidak memberikan perbaikan, sehingga titik berhentinya juga dipengaruhi keberuntungan. Pola hasilnya pun berbeda dari algoritma *hill-climbing* lain dan *Simulated Annealing* yaitu percobaan kedua menghasilkan nilai terendah pada *Stochastic*, padahal pada algoritma-algoritma tersebut percobaan yang sama justru menghasilkan nilai tertinggi.

Hill Climbing with Sideways Move	**Cukup konsisten.** Berdasarkan Tabel 28, ketiga percobaan menghasilkan susunan yang valid dengan **selisih 29 juta antara hasil tertinggi dan terendah,** lebih kecil dibandingkan *Steepest Ascent* dan *Stochastic*. *Sideways move* membantu pencarian melanjutkan perjalanan ketika menemui *plateau*, sehingga hasil akhirnya tidak sepenuhnya ditentukan oleh *local optimum* pertama yang ditemui. Namun, karena algoritma hanya melakukan satu kali pencarian dan *plateau* pada permasalahan ini sempit, hasil akhirnya masih cukup bergantung pada *state* awal. Hal ini terlihat dari percobaan kedua yang menghasilkan nilai jauh lebih tinggi dibandingkan dua percobaan lainnya.
Random Restart Hill Climbing	**Sangat konsisten**. Berdasarkan Tabel 28, ketiga percobaan menghasilkan susunan yang valid dengan **selisih hanya 12 juta antara hasil tertinggi dan terendah,** yang merupakan selisih terkecil di antara algoritma *hill-climbing* dan *Simulated Annealing*. Kestabilan ini sesuai dengan cara kerja *Random Restart* yang menjalankan pencarian dari sebelas *state* awal berbeda dalam satu percobaan dan memilih yang terbaik. Walaupun *fee* akhir dari masing-masing pencarian di dalamnya sangat bervariasi, mengambil yang terbaik dari banyak pencarian membuat hasil akhir setiap percobaan berada di rentang yang sempit. Dengan kata lain, pengaruh satu *state* awal yang kurang menguntungkan dapat ditutupi oleh *restart* lainnya.
Simulated Annealing	**Konsisten.** Berdasarkan Tabel 28, ketiga percobaan menghasilkan susunan yang valid dengan **selisih 14 juta antara hasil tertinggi dan terendah,** hampir sama dengan *Random Restart*. Selain hasil akhirnya, jumlah iterasi, frekuensi *stuck*, dan durasi pencariannya juga hampir sama pada ketiga percobaan. Hal ini terjadi karena pada awal pencarian, saat suhu masih tinggi, algoritma sering menerima tetangga yang lebih buruk, sehingga pengaruh *state* awal cepat berkurang. Ketika suhu sudah turun, setiap percobaan sudah berada di daerah ruang pencarian yang sama baiknya, lalu menyempurnakan hasilnya secara bertahap. Akibatnya, hasil akhir *Simulated Annealing* tidak terlalu bergantung pada *state* awal.
Genetic Algorithm	**Konsistensi bergantung pada konfigurasi.** Berdasarkan Tabel 28, seluruh percobaan pada setiap konfigurasi menghasilkan susunan yang valid, tetapi selisih hasil antar percobaan berubah-ubah sesuai parameter. Konfigurasi populasi 50 dan iterasi 200 menghasilkan selisih hanya 7 juta, yang merupakan selisih terkecil dari seluruh eksperimen `input2`. Sebaliknya, konfigurasi populasi 200 dan iterasi 200 menghasilkan selisih

	40 juta, terbesar dari seluruh eksperimen. Konfigurasi lainnya berada di antara keduanya, dengan selisih 22 hingga 26 juta. Hal ini menunjukkan bahwa tidak ada hubungan yang sederhana antara besarnya populasi atau banyaknya iterasi dengan konsistensi hasil. Populasi yang besar membuka peluang untuk menemukan solusi yang lebih baik, tetapi tidak menjamin solusi tersebut selalu ditemukan. Dalam jumlah iterasi yang terbatas, berhasil atau tidaknya sebuah percobaan tetap ditentukan oleh kualitas populasi awal serta unsur acak pada *crossover* dan mutasi, sehingga hasil antar percobaan dapat berbeda jauh.

2.8.5 Analisis Pertanyaan 5

Tabel 30. Analisis Eksperimen Pertanyaan 5

Pertanyaan 5. Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?
Penjelasan
Berdasarkan hasil eksperimen pada skenario 1, penambahan banyak iterasi dengan jumlah populasi tetap meningkatkan total `ShippingFee` akhir. Rata-rata hasil naik dari 201,0 juta pada 50 iterasi menjadi 267,3 juta pada 200 iterasi, lalu 280,3 juta pada 800 iterasi. Peningkatan ini terjadi karena setiap generasi memberi kesempatan tambahan bagi crossover dan mutasi untuk menghasilkan individu yang lebih baik, sementara individu terbaik selalu dipertahankan sehingga nilai maksimum populasi tidak pernah turun. Namun, peningkatannya makin kecil sehingga menaikkan iterasi empat kali lipat dari 50 ke 200 menambah rata-rata sekitar 66 juta, sedangkan dari 200 ke 800 hanya menambah sekitar 13 juta. Kemungkinan populasi makin lama makin seragam (kehilangan keragaman), sehingga crossover antar induk yang hampir sama tidak lagi menghasilkan perbaikan dan perbaikan hanya datang dari mutasi, akibatnya peningkatannya melambat. Hal ini sejalan dengan percobaan pertama yang hanya naik dari 264 ke 267 juta, meskipun dua percobaan lainnya masih naik (267 ke 281 dan 271 ke 293). Selisih antara hasil tertinggi dan terendah antar percobaan juga tidak berubah searah, yaitu 22, 7, dan 26 juta pada 50, 200, dan 800 iterasi. Sementara itu, durasi bertambah sebanding dengan banyak iterasi. Berdasarkan hasil eksperimen pada Skenario 2, penambahan jumlah populasi dengan banyak iterasi tetap juga meningkatkan rata-rata total ShippingFee akhir, dari 218,7 juta pada populasi 10, menjadi 267,3 juta pada populasi 50, dan 286,0 juta pada populasi 200. Populasi yang lebih besar kemungkinan menjaga keragaman individu sehingga crossover memiliki lebih banyak bahan untuk dikombinasikan. Populasi besar juga menghasilkan individu terbaik awal dengan pelanggaran yang lebih sedikit, karena dipilih dari lebih banyak state acak (rata-rata sekitar 13 sel bertumpuk pada populasi

200, dibanding sekitar 30 pada populasi 50). Namun, populasi yang lebih besar tidak menjamin hasil yang lebih baik pada setiap percobaan contohnya pada percobaan pertama, populasi 200 menghasilkan 261 juta, sedikit lebih rendah dari populasi 50 (264 juta). Selisih hasil antar percobaan juga tidak berubah searah, yaitu 25, 7, dan 40 juta pada populasi 10, 50, dan 200. Durasi pencarian bertambah dari sekitar 0,06 detik pada populasi 10 menjadi sekitar 2,2 detik pada populasi 200.

Secara keseluruhan, menambah iterasi maupun populasi cenderung memperbaiki hasil Genetic Algorithm, tetapi manfaatnya makin kecil dan durasinya makin lama. Pada anggaran evaluasi yang hampir sama, yaitu populasi 50 dengan 800 iterasi dan populasi 200 dengan 200 iterasi (masing-masing sekitar 20.000 anak baru), hasil rata-ratanya berdekatan (280,3 dan 286,0 juta), sehingga tidak ada bukti yang jelas bahwa salah satu parameter lebih berpengaruh dibanding yang lain.

BAB III
KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan keseluruhan hasil eksperimen pada permasalahan penataan kendaraan di geladak kapal, seluruh algoritma *local search* yang diimplementasikan, yaitu *Steepest Ascent Hill-Climbing, Stochastic Hill-Climbing, Hill-Climbing with Sideways Move, Random Restart Hill-Climbing, Simulated Annealing,* dan *Genetic Algorithm,* selalu berhasil mengubah *state* awal yang tidak valid menjadi susunan kendaraan yang valid. Hal ini menunjukkan bahwa *objective function* yang dirancang, dengan penalti besar untuk setiap pelanggaran dan total `ShippingFee` sebagai tujuan utama, berhasil mengarahkan pencarian untuk memenuhi seluruh batasan terlebih dahulu sebelum memaksimalkan keuntungan.

Dari segi kualitas hasil, *Hill-Climbing with Sideways Move* dan *Random Restart Hill-Climbing* menghasilkan rata-rata total `ShippingFee` tertinggi, disusul oleh *Simulated Annealing* dengan selisih yang kecil. Hasil terbaik dari seluruh eksperimen diperoleh *Sideways Move* dengan total `ShippingFee` 323 juta dan kapasitas kapal yang hampir penuh. *Steepest Ascent Hill-Climbing* menghasilkan nilai yang lebih rendah karena langsung berhenti di *local optimum* terdekat, sedangkan *Stochastic Hill-Climbing* menghasilkan nilai terendah di antara algoritma *hill-climbing* karena dapat berhenti sebelum benar-benar mencapai *local optimum*. Kualitas hasil *Genetic Algorithm* sangat bergantung pada parameter, dan pada konfigurasi terbaiknya pun rata-rata hasilnya masih berada di bawah *Steepest Ascent, Sideways Move, Random Restart,* dan *Simulated Annealing*, tetapi di atas *Stochastic Hill-Climbing.* Hal ini kemungkinan disebabkan oleh *crossover* dan mutasi yang sering menghasilkan kendaraan bertumpuk, sehingga banyak individu baru yang langsung tersingkir oleh penalti.

Dari segi durasi, algoritma yang hanya mengevaluasi satu tetangga atau sebagian populasi per iterasi, yaitu *Stochastic Hill-Climbing, Simulated Annealing*, dan *Genetic Algorithm*, berjalan lebih cepat dibandingkan algoritma yang mengevaluasi seluruh

tetangga. *Random Restart Hill-Climbing* menjadi algoritma paling lambat karena menjalankan *Steepest Ascent* berulang kali. *Simulated Annealing* memberikan keseimbangan terbaik antara kualitas dan waktu, karena menghasilkan nilai yang mendekati *Sideways Move* dan *Random Restart* dengan durasi kurang dari satu detik.

Dari segi konsistensi, *Random Restart Hill-Climbing* dan *Simulated Annealing* menghasilkan nilai yang paling stabil antar percobaan, karena keduanya tidak bergantung pada satu *state* awal saja. Sebaliknya, *Steepest Ascent* dan *Stochastic Hill-Climbing* paling kurang konsisten di antara algoritma selain *Genetic Algorithm* karena hasil akhirnya sangat ditentukan oleh *state* awal dan, khusus *Stochastic*, oleh pemilihan tetangga secara acak. Pada *Genetic Algorithm,* penambahan populasi maupun iterasi cenderung meningkatkan kualitas hasil dengan peningkatan yang makin kecil, tetapi tidak otomatis membuat hasil lebih konsisten. Selain itu, *sideways move* pada permasalahan ini jarang terjadi, karena komponen pemecah seri pada *objective function* membuat *plateau* menjadi sempit.

3.2 Saran

Berdasarkan hasil eksperimen dan analisis yang telah dilakukan, berikut beberapa saran untuk pengembangan lebih lanjut:

1. Pemilihan algoritma sebaiknya disesuaikan dengan kebutuhan. Jika kualitas dan kestabilan hasil lebih diutamakan dan waktu pencarian tidak menjadi masalah, *Random Restart Hill-Climbing* atau *Hill-Climbing with Sideways Move* dapat digunakan. Jika dibutuhkan hasil yang cukup baik dalam waktu singkat, *Simulated Annealing* lebih direkomendasikan.
2. Operator *crossover* pada *Genetic Algorithm* perlu dirancang ulang agar lebih sesuai dengan sifat permasalahan, misalnya dengan mewariskan sekelompok kendaraan yang berdekatan dari satu induk sekaligus, atau menambahkan tahap perbaikan yang memindahkan kendaraan bertumpuk ke posisi kosong. Dengan begitu, lebih banyak anak yang valid dan perbaikan populasi dapat berjalan lebih cepat.

3. Perhitungan *objective function* dapat dibuat lebih efisien dengan hanya menghitung ulang bagian geladak yang berubah akibat suatu gerakan, bukan seluruh geladak. Cara ini akan sangat mempercepat *Steepest Ascent, Sideways Move*, dan *Random Restart* yang mengevaluasi ribuan tetangga pada setiap iterasi.
4. Parameter setiap algoritma, seperti *patience* pada *Stochastic Hill-Climbing*, suhu awal dan laju pendinginan pada *Simulated Annealing*, jumlah *restart* pada *Random Restart,* serta *mutation rate* pada *Genetic Algorithm*, perlu diuji dengan lebih banyak variasi untuk menemukan nilai yang paling sesuai.
5. Pengujian dengan jumlah percobaan yang lebih banyak serta variasi data input yang lebih beragam, baik dari segi jumlah kendaraan, ukuran geladak, maupun kapasitas kapal, diperlukan untuk menilai konsistensi dan kemampuan setiap algoritma secara lebih akurat.

Pembagian Tugas

Nama	NIM	Pembagian Tugas
Zhafira Kayla Nafisa	18224018	Steepest Ascent, Stochastic
Kayla Fiyaza Nawal Zaghbi	18224054	Simulated Annealing, Genetic Algorithm
Nabilla Eka Putri Sunarto	18224056	Random Restart, Sideways Move

REFERENSI

Lodi, A., Martello, S., & Monaci, M. (2002). Two-dimensional packing problems: A survey. *European Journal of Operational Research, 141*(2), 241–252.

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671–680.

Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95.

Python Software Foundation. (n.d.). *random: Generate pseudo-random numbers*. Python 3 Documentation. https://docs.python.org/3/library/random.html

Tim Pengajar IF3070. (2026). *Beyond Classical Search*. https://edunexcontentprodhot.blob.core.windows.net/edunex/2026/97455-Foundations-of-Artificial-Intelligence-Parent-Class/467169-Beyond-Classical-Search/file/1788513395957_2026_IF3070_Materi03-BeyondClassicalSearch?sv=2024-11-04&spr=https&st=2026-09-04T09%3A16%3A36Z&se=2028-09-03T09%3A16%3A36Z&sr=b&sp=r&sig=iAGCxaVplBwFqZ6RLFFVeQxDdGVVRoC4G0SivJ6%2BpfM%3D&rsct=application%2Fpdf

LAMPIRAN

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM	:	- Zhafira Kayla Nafisa/18224018 - Kayla Fiyaza Nawal Zaghbi/18224054 - Nabilla Eka Putri Sunarto/18224056
Fakultas/Sekolah	:	Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah	:	IF3070
Nama Mata Kuliah	:	Dasar Inteligensi Artifisial
Judul Tugas/Proposal	:	Optimasi Penataan Kendaraan pada Geladak Kapal dengan Local Search

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Sedang
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Sedang
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Zhafira Kayla Nafisa
18224018



Kayla Fiyaza Nawal Zaghbi
18224054



Nabilla Eka Putri Sunarto
18224056